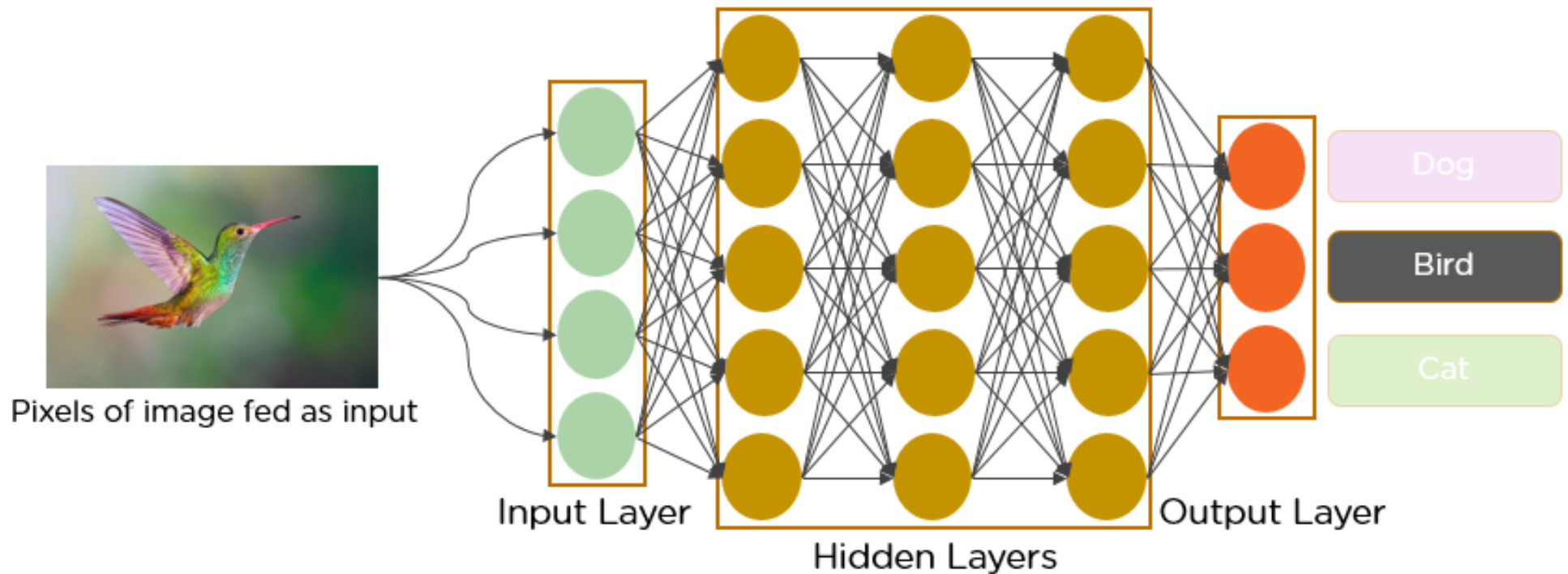


Lecture 3

Deep CNNs for Image



Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Saving and Reloading Models

- Call `tf.keras.Model.save` to save a model's architecture, weights, and training configuration in a single file/folder.
- Entire model can be saved in 3 file formats (.keras, SavedModel and HDF5).
- This allows you to export a model so it can be used without access to the original Python code.
 - Saving custom objects?
https://www.tensorflow.org/tutorials/keras/save_and_load
 - need to include them if .keras or .h5
- Since the optimizer-state is recovered, you can resume training from where left off.
- `.keras` default now (2.15), `SavedModel` format was default in TF2.13. Can choose.

saved model format

```
model.save('myModel')
```

```
new_model=tf.keras.models.load_model('myModel')
```

hdf5 format

```
model.save('myModel.h5')
```

```
new_model=tf.keras.models.load_model('myModel.h5',
```

```
custom_objects=custom_objects) 5
```

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

**“You need a lot of a data if you
want to train/use CNNs”**

Transfer Learning

“You need a lot of a data if you want to train/use CNNs”

Transfer Learning with Keras

- one appeal of deep nets is that they easily admit transfer
- take lower layers of a network good at one task (e.g. object recognition) and use them in a similar task (e.g. flower recognition)

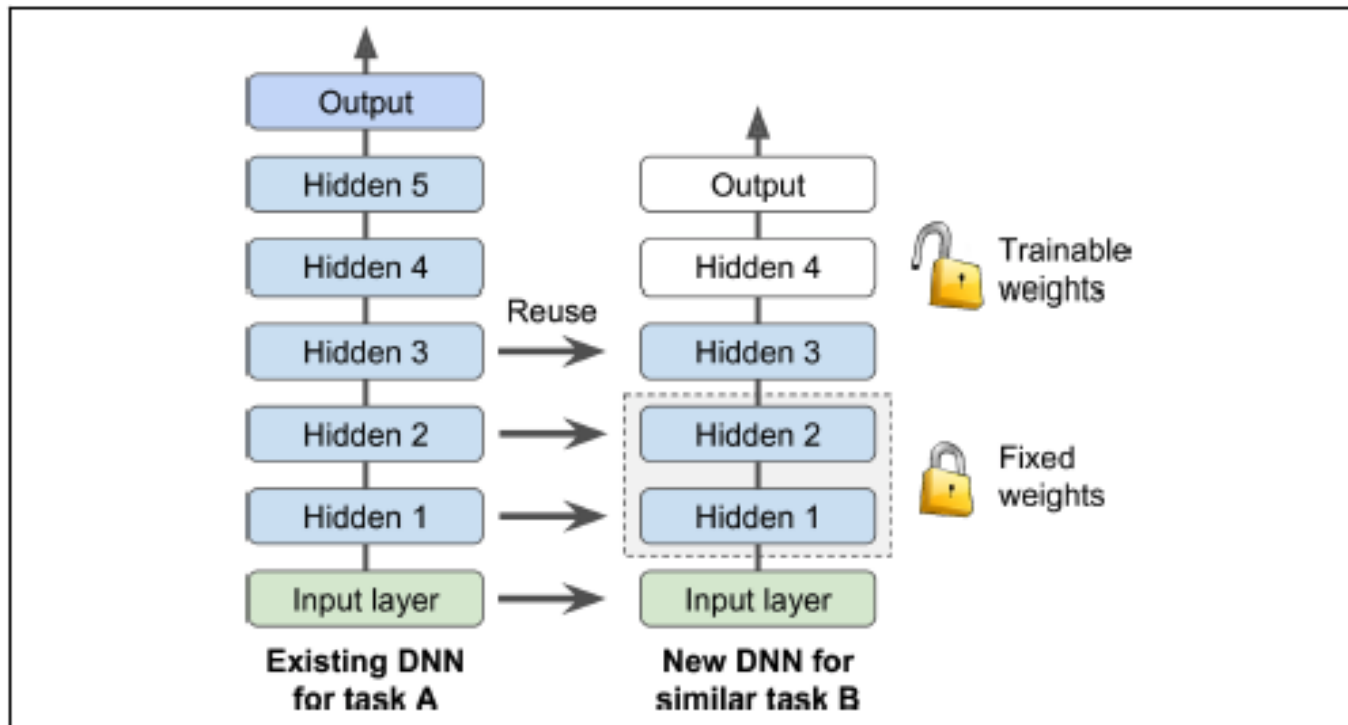


Figure 11-4. Reusing pretrained layers



If the input pictures of your new task don't have the same size as the ones used in the original task, you will usually have to add a preprocessing step to resize them to the size expected by the original model. More generally, transfer learning will work best when the inputs have similar low-level features.

Transfer Learning with CNNs



More specific

More generic

	Dataset similar to ImageNet	Dataset very different from ImageNet
very little data (10s to 100s)	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data (100s to 1000s)	Finetune a few layers	Finetune a larger number of layers

Transfer Learning with Keras

First, you need to load model A, and create a new model based on the model A's layers. Let's reuse all layers except for the output layer:

```
model_A = keras.models.load_model("my_model_A.h5")
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))
```

Note that `model_A` and `model_B_on_A` now share some layers. When you train `model_B_on_A`, it will also affect `model_A`. If you want to avoid that, you need to clone `model_A` before you reuse its layers. To do this, you must clone model A's architecture, then copy its weights (since `clone_model()` does not clone the weights):

```
model_A_clone = keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

start off freezing the weights in the lower layers...

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False
```

recompile to cause this freeze

```
model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd",
                     metrics=["accuracy"])
```

Transfer Learning with Keras

after frozen for a few epochs, can resume adapting them again too

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = keras.optimizers.SGD(lr=1e-4) # the default lr is 1e-3
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B, y_valid_B))
```

more detailed example demonstrating this practically after we review basic imagenet object recognition models

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. Convenient Dataset Commands

(a) batch, filter, take, map, shuffle, cache, prefetch

2. Converting your data to easily load into Data API w/ tfrecords

3. Using Keras Preprocessing Layers to Handle Simple Data Processing

4. How to Work with Heterogeneous Data

(a) Datasets of Dictionaries

i. Saving Them

ii. Reloading them & Preprocessing

iii. Making a Keras Model For them

7. Customizing Keras

(a) Custom Loss Functions

(b) Custom Layer

Convenient Data API Dataset Commands

1. batch - group data into blocks (batches) of same size
2. filter - keep only the part of the data that returns true for the given filter/test function
3. take - return k instances from the dataset
4. map - apply a function to each element of the dataset
5. shuffle - buffer shuffle the dataset
6. prefetch - always keep the next k elements in memory
7. cache - hold the entire dataset in memory

TensorFlow's Data API – Introduction

TensorFlow's data API makes operations to group, repeat, shuffle, and cache data easy. For example, suppose we want to repeat a data set three times, then group it into batches of size 7

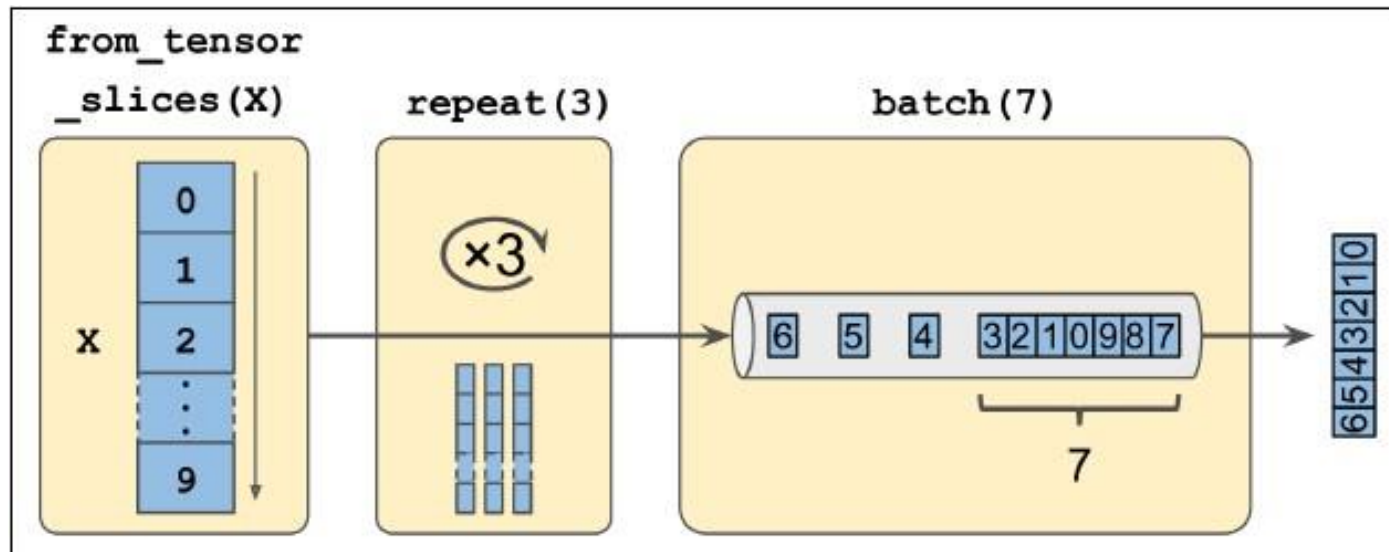


Figure 13-1. Chaining Dataset Transformations

```
>>> X = tf.range(10) # any data tensor
>>> dataset = tf.data.Dataset.from_tensor_slices(X)
>>> dataset

>>> dataset = dataset.repeat(3).batch(7)
>>> for item in dataset:
...     print(item)
```

note, repeat without a number builds an infinite repeating stream!!

TensorFlow's Data API – Mapping, Filtering, Shuffling

Once we have created a dataset, we can transform it elementwise as in

```
>>> dataset = dataset.map(lambda x: x * 2) # Items: [0,2,4,6,8,10,12]
```

Or perform an operation on the whole of it as in

```
>>> dataset = dataset.apply(tf.data.experimental.unbatch()) # Items: 0,2,4,...
```

Or pull out parts of it as in

```
>>> dataset = dataset.filter(lambda x: x < 10) # Items: 0 2 4 6 8 0 2 4 6...
```

A dataset can be buffer-shuffled (keep buffer full, selecting one at random, sending it and replacing it) with

```
>>> dataset = tf.data.Dataset.range(10).repeat(3) # 0 to 9, three times
>>> dataset = dataset.shuffle(buffer_size=5, seed=42).batch(7)
>>> for item in dataset:
...     print(item)
...
tf.Tensor([0 2 3 6 7 9 4], shape=(7,), dtype=int64)
tf.Tensor([5 0 1 1 8 6 5], shape=(7,), dtype=int64)
tf.Tensor([4 8 7 1 2 3 0], shape=(7,), dtype=int64)
tf.Tensor([5 4 2 7 8 9 9], shape=(7,), dtype=int64)
tf.Tensor([3 6], shape=(2,), dtype=int64)
```

TensorFlow's Data API – Pulling Lines from Text Files

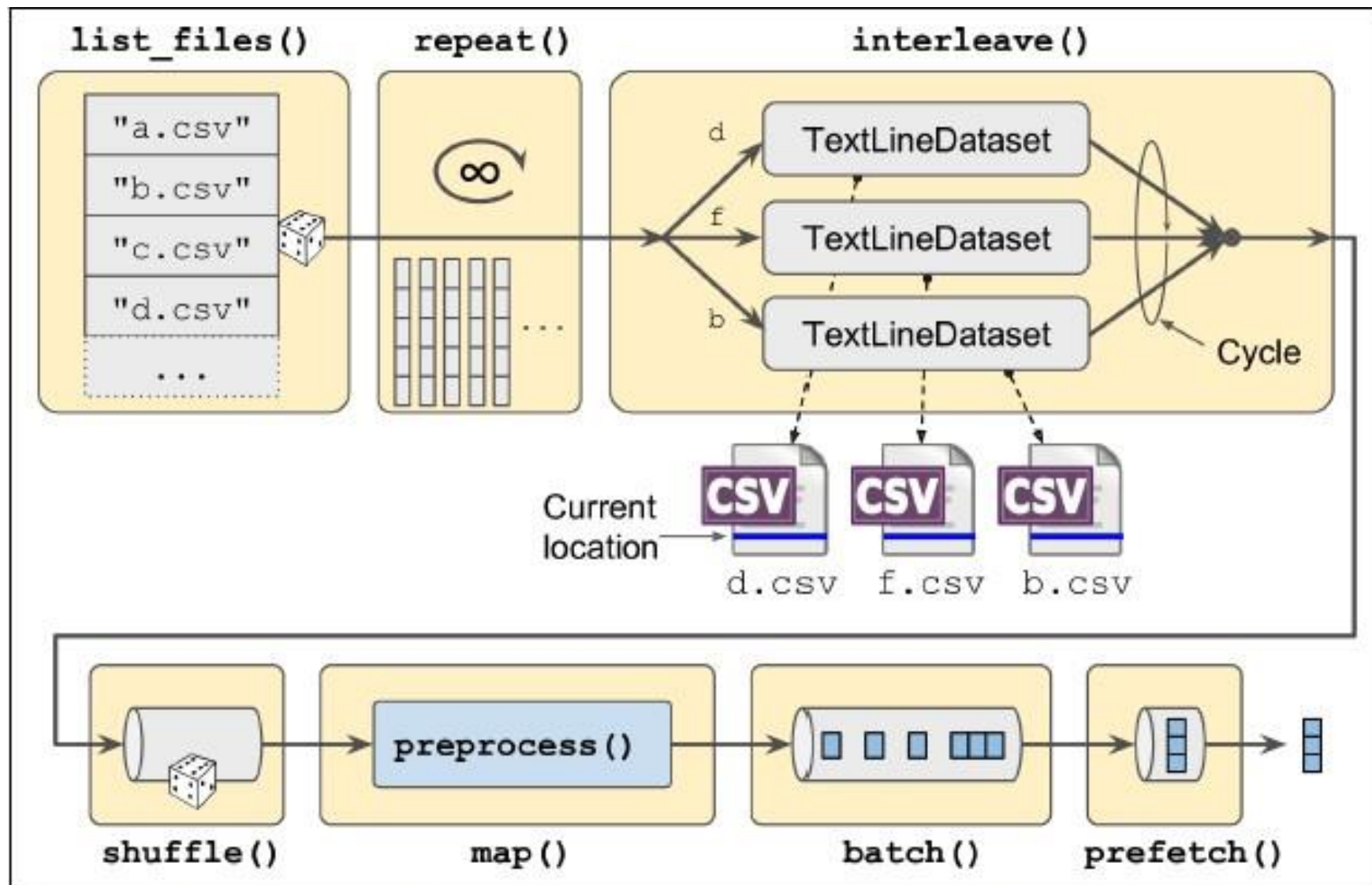


Figure 13-2. Loading and Preprocessing Data From Multiple CSV Files

TensorFlow's Data API – Pulling Lines from Text Files

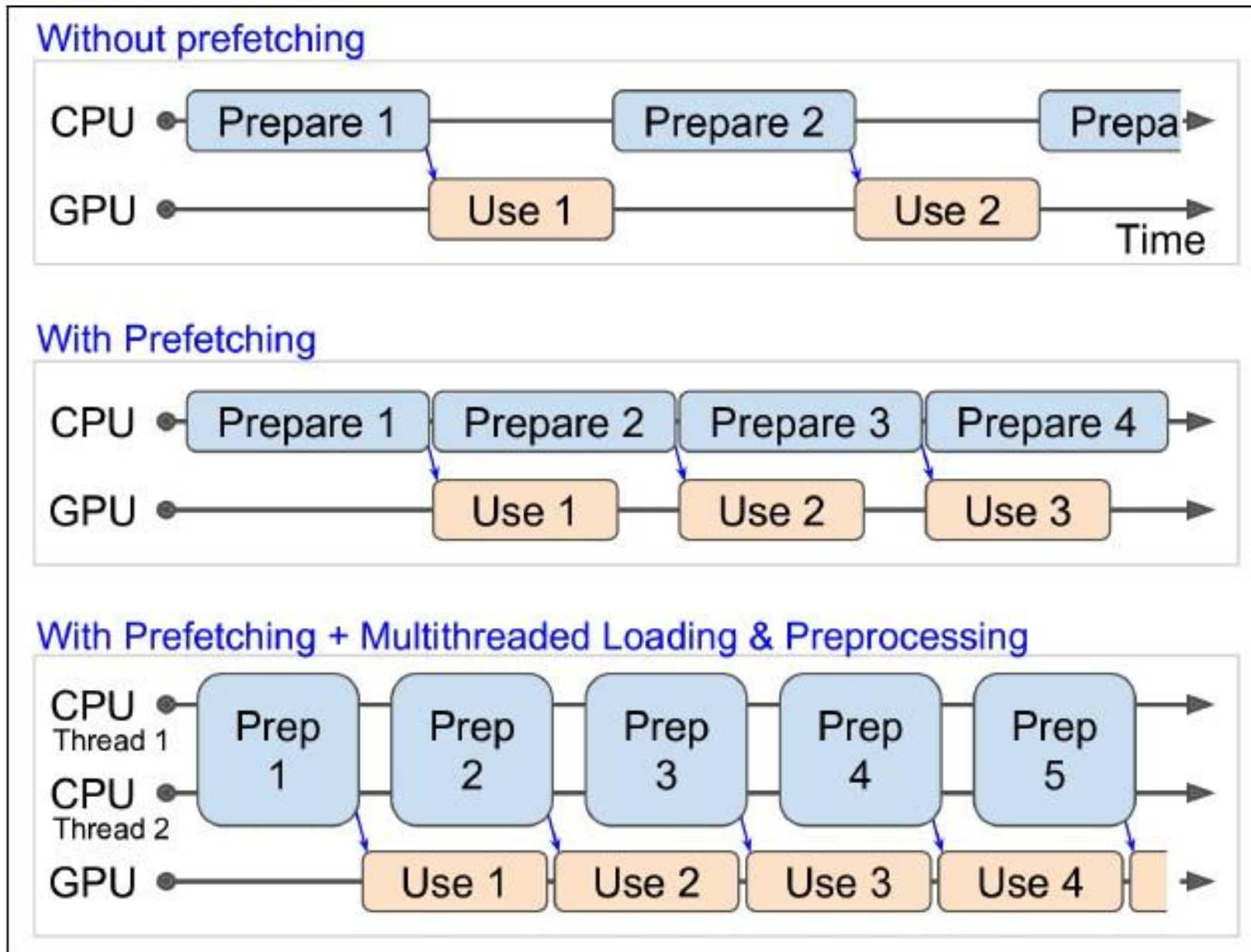


Figure 13-3. Speedup Training Thanks to Prefetching and Multithreading

TensorFlow's Data API – Pulling Lines from Text Files

```
X_mean, X_std = [...] # mean and scale of each feature in the training set
n_inputs = 8
```

```
def preprocess(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    x = tf.stack(fields[:-1])
    y = tf.stack(fields[-1:])
    return (x - X_mean) / X_std, y

def csv_reader_dataset(filepaths, repeat=None, n_readers=5,
                       n_read_threads=None, shuffle_buffer_size=10000,
                       n_parse_threads=5, batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths).repeat(repeat)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.shuffle(shuffle_buffer_size)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.batch(batch_size)
    return dataset.prefetch(1)
```

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

TensorFlow Data API – TFRecord file format

- For storing and reading data efficiently with tensorflow, often programmers elect to use the TFRecord file format.
- create TFRecord file with `tf.io.TFRecordWriter` class:

```
with tf.io.TFRecordWriter("my_data.tfrecord") as f:  
    f.write(b"This is the first record")  
    f.write(b"And this is the second record")
```

- load a collection of TFrecords files with `TFRecordDataset()`

```
filepaths = ["my_data.tfrecord"]  
dataset = tf.data.TFRecordDataset(filepaths)  
for item in dataset:  
    print(item)
```

- `num_parallel_reads` argument can be used to read several files at once
- can alternatively use the `list_files()` and `interleave()` technique used in last lecture, just followed by `TFRecordDataset` rather than `TextLineDataset`

TensorFlow Data API – TFRecord file format

- Alternative Approach Using `list_files()` and `interleave()`: You can use the `tf.data.Dataset.list_files()` method to list multiple TFRecord files and `interleave()` to read them in parallel.

```
filepaths = tf.data.Dataset.list_files("path/to/tfrecords/*.tfrecord")
dataset = filepaths.interleave(
    lambda filepath: tf.data.TFRecordDataset(filepath),
    cycle_length=4, num_parallel_calls=tf.data.experimental.AUTOTUNE
)
```

- if size on disk is concern, when writing TFRecords can use compression

```
options = tf.io.TFRecordOptions(compression_type="GZIP")
with tf.io.TFRecordWriter("my_compressed.tfrecord", options) as f:
    [...]
```

- but if you do so, you need to specify the compression you used to make it when you load the file again

```
dataset = tf.data.TFRecordDataset(["my_compressed.tfrecord"],
                                   compression_type="GZIP")
```


TensorFlow Data API – TFRecord file format

- TFrecords can store any binary format you want, but usually they are used to contain “serialized protocol buffers” == *protobuf*s a Google file format.
- `protoc` command takes a protobuf definition (special language) and turns it into class definitions in whatever programming language you want (for us – python)
- TensorFlow already contains the necessary classes for the following, most common, format – the `Example` class

```
syntax = "proto3";
message ByteList { repeated bytes value = 1; }
message FloatList { repeated float value = 1 [packed = true]; }
message Int64List { repeated int64 value = 1 [packed = true]; }

message Feature {
  oneof kind {
    ByteList bytes_list = 1;
    FloatList float_list = 2;
    Int64List int64_list = 3;
  }
};

message Features { map<string, Feature> feature = 1; };
message Example { Features features = 1; };
```

Example contains a **Features** dictionary mapping strings (i.e. title of the feature) to **Feature** objects.

A **Feature** object contains either a list of bytes, a list of floats, or a list of integers. The floats and integers lists can optionally be “packed” when stored.

TensorFlow Data API – TFRecord file format

Here's how to use the python classes generated from this protobuf:

```
from tensorflow.train import ByteList, FloatList, Int64List
from tensorflow.train import Feature, Features, Example

person_example = Example(
    features=Features(
        feature={
            "name": Feature(bytes_list=ByteList(value=[b"Alice"])),
            "id": Feature(int64_list=Int64List(value=[123])),
            "emails": Feature(bytes_list=ByteList(value=[b"a@b.com",
                                                         b"c@d.com"])))
        })
    )
```

then, we can save the example to the tfrecords file

```
with tf.io.TFRecordWriter("my_contacts.tfrecord") as f:
    f.write(person_example.SerializeToString())
```

Typical use: one Example per instance, with several examples written to the same tfrecords file (just call serialize and write several times).

TensorFlow Data API – TFRecord file format

Later, we can read the tfrecord file in with

```
feature_description = {
    "name": tf.io.FixedLenFeature([], tf.string, default_value=""),
    "id": tf.io.FixedLenFeature([], tf.int64, default_value=0),
    "emails": tf.io.VarLenFeature(tf.string),
}

for serialized_example in tf.data.TFRecordDataset(["my_contacts.tfrecord"]):
    parsed_example = tf.io.parse_single_example(serialized_example,
                                                feature_description)
```

Now, the
tensor.

```
{
    "name": <tf.Tensor: shape=(), dtype=string>,
    "id": <tf.Tensor: shape=(), dtype=int64>,
    "emails": <tf.SparseTensor: dtype=string>
}
```

ir associated

- fixed
- varia

- can use `tf.sparse.to_dense()` to convert sparse to dense

```
>>> tf.sparse.to_dense(parsed_example["emails"], default_value=b"")
<tf.Tensor: [...] dtype=string, numpy=array([b'a@b.com', b'c@d.com'], [...])>
>>> parsed_example["emails"].values
<tf.Tensor: [...] dtype=string, numpy=array([b'a@b.com', b'c@d.com'], [...])>
```

TensorFlow Data API – TFRecord file format - Images

To store an image, e.g.,

1. `tf.io.encode_jpeg()` to encode it into JPEG format, then
2. put the binary data into a `ByteList` feature in an `Example`

Then later, to load the image

3. parse the example, pull out the image's feature `ByteList`
4. `tf.decode_image()`

Practice Converting Dataset into TFRecords using Example Protobufs

let's show how the previous ideas could be applied to a simple Pandas dataframe
"California Housing" dataset we can download with the code.

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()
housing.info()
```

Practice Converting Dataset into TFRecords using Example Protobufs

```
import tensorflow as tf
from tensorflow.train import ByteList, FloatList, Int64List
from tensorflow.train import Feature, Features, Example

# see that all but last column are floats, let's only save those first
num_cols=housing.columns[:-1]
# method a - bigger file size, each instance has all dictionary labels
with tf.io.TFRecordWriter('test-housing.tfrecord') as f:
    for row in housing.iterrows():
        myDict=dict(zip(num_cols,map(lambda x:
            Feature(float_list=FloatList(value=[x])),row[1][num_cols])))
        myExamp=Example(features=Features(feature=myDict))
        f.write(myExamp.SerializeToString())

# method b - smaller file size, have to remember feature names
with tf.io.TFRecordWriter('test-housing-smaller.tfrecord') as f:
    for row in housing.iterrows():
        myDict={'features':Feature(float_list=FloatList(value=row[1][num_cols].values))}
        myExamp=Example(features=Features(feature=myDict))
        f.write(myExamp.SerializeToString())
```

Practice Converting Dataset into TFRecords using Example Protobufs -comparison

Observe that the original csv is 1.4MB, while the tfrecord saving the different float features individually as a dict repeating each of their names is 5.3 MB (bigger) owing to repeating the dictionary labels in each instance. Using a single feature name gives a tfrecord comparable to original data size. Using the **gzip** option would shrink further

Practice Converting Dataset into TFRecords using Example Protobufs - how to reload the data?

```
# larger tfrecord method
feature_description=dict(zip(num_cols, [
    tf.io.FixedLenFeature([], tf.float32, default_value=0.0)]*len(num_cols)))
def parse_examples(serialized_examples):
    examples=tf.io.parse_example(serialized_examples, feature_description)
    targets=examples.pop('median_house_value')
    return examples, targets
datasetB=tf.data.TFRecordDataset(['test-housing.tfrecord']).batch(32).map(
    parse_examples).cache()

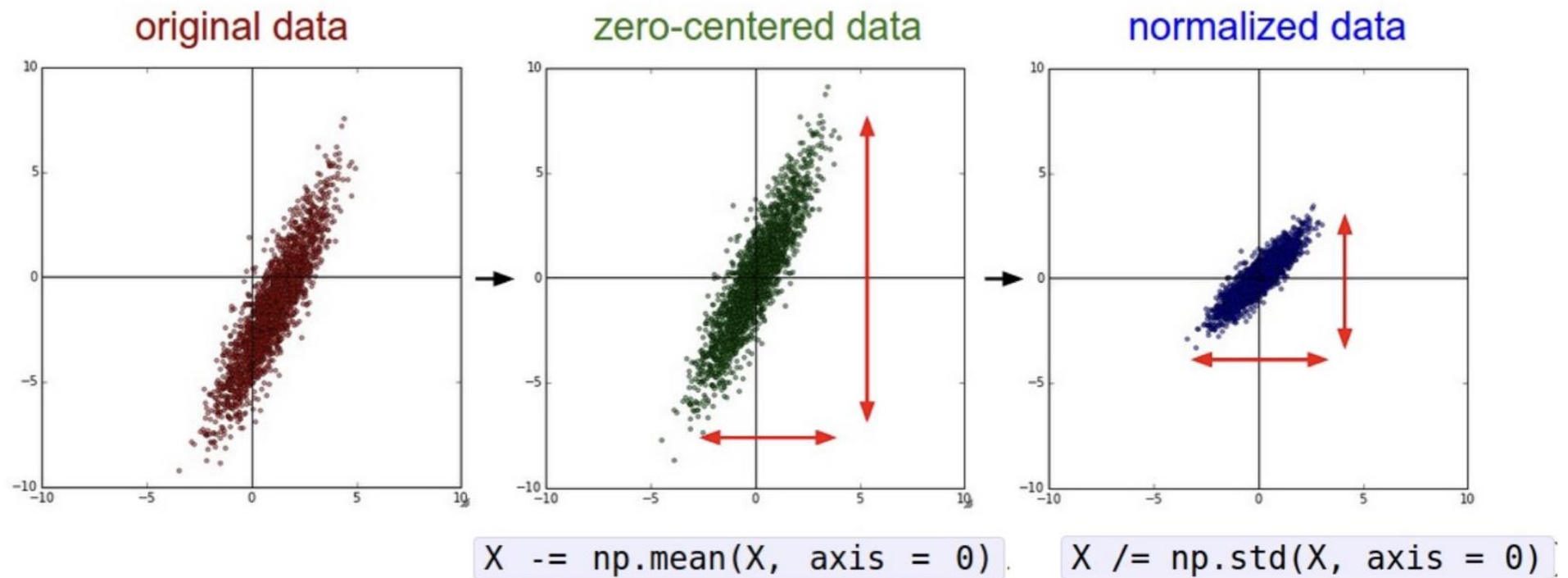
# smaller tfrecord method
feature_description2={'features':tf.io.FixedLenFeature([len(num_cols)], tf.float32,
    default_value=np.zeros(len(num_cols)))}
def parse_examples2(serialized_examples):
    examples=tf.io.parse_example(serialized_examples, feature_description2)
    targets=examples['features'][:, -1]
    return examples['features'][:, :-1], targets
dataset=tf.data.TFRecordDataset(['test-housing-smaller.tfrecord']).batch(32).map(
    parse_examples2).cache()
```

Inspecting the result, the first keeps the features as a dictionary, the second as a tensor.

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

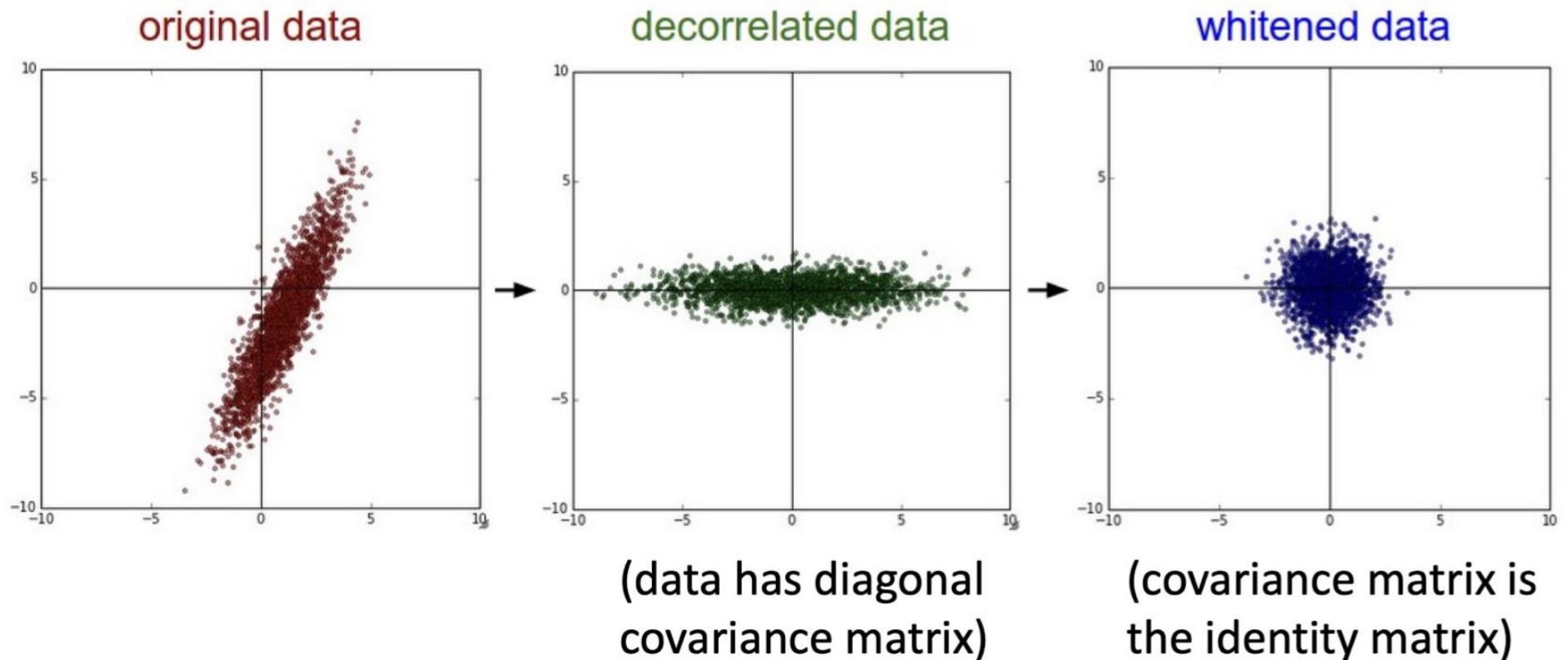
Data Preprocessing



(Assume X [NxD] is data matrix,
each example in a row)

Data Preprocessing

In practice, you may also see **PCA** and **Whitening** of the data



Data Preprocessing for Images

e.g. consider CIFAR-10 example with [32,32,3] images

- Subtract the mean image (e.g. AlexNet)
(mean image = [32,32,3] array)
- Subtract per-channel mean (e.g. VGGNet)
(mean along each channel = 3 numbers)
- Subtract per-channel mean and
Divide by per-channel std (e.g. ResNet)
(mean along each channel = 3 numbers)

Not common to
do PCA or
whitening

Using Keras Preprocessing Layers to Handle Simple Data Processing

https://www.tensorflow.org/guide/keras/preprocessing_layers

—Normalization Layer:

```
# continue with previous example using a standard scaling layer
normalizer=tf.keras.layers.Normalization(axis=-1)
# the adapt method learns the weights - targets need to be removed for it
normalizer.adapt(dataset.map(lambda x,y: x))
# inspecting the means and variances learned
normalizer.weights
```

Once a preprocessing layer is adapted, we can either simply apply it via a map, e.g.

```
proced=dataset.map(lambda x,y: normalizer(x))
```

feeding the result into model we make, OR can include it within the keras model itself:

```
# dimension of feature vector is one less because of popping out the target
ins=tf.keras.Input((len(num_cols)-1),dtype=tf.float32)
proc=normalizer(ins)
rest=tf.keras.Sequential([tf.keras.layers.Dense(128,activation='relu'),
                           tf.keras.layers.Dense(1)])
outs=rest(proc)
whole_model=tf.keras.Model(inputs=ins,outputs=outs)
```

Including in model enables preprocessor to be saved in model.save(...).

Using Keras Preprocessing Layers to Handle Simple Data Processing

Other preprocessing layers of interest

- Make categorical from Numerical Feature?
 - `tf.keras.layers.Discretization` - quantizes
- Categorical Feature?
 - `tf.keras.layers.CategoryEncoding` to take integers into one-hot, multi-hot, or count encoding
 - `tf.keras.layers.StringLookup` - turns string categorical values an encoded representation that can be read by an Embedding layer or Dense layer (int, multi-hot, count, tf-idf).
- For text have a look at
 - `tf.keras.layers.TextVectorization`: turns raw strings into an encoded representation that can be read by an Embedding layer or Dense layer.
- there are also layers for images, see the docs

Will use some of these in next heterogenous data example.

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

How to Work with Heterogeneous Data

- Very often we will want to build models that take different types of data as input
 - e.g. some raw data features are categorical, some are numerical
 - e.g. both an image and a caption of the image
- Tensors can not store different data types (e.g. float and int, float and string, int and string) within same tensor
- In this instance, we make the instances in the dataset dictionary valued. The dictionary labels index different tensors.
- Data API works seamlessly with sources that are dictionary valued.
- In TF1, we use `feature_columns` to preprocess these, but now use Keras preprocessing layers in TF2.
- Keras Inputs can specify different named data sources. Models then provided dictionary with these names as labels.

Define input layers for different data types

```
numerical_input = Input(shape=(3,), name='numerical')  
categorical_input = Input(shape=(1,), name='categorical')
```


Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

How to Work with Heterogeneous Data - Saving & Reloading

Let's rework prev. example and keep categorical attribute

```
with tf.io.TFRecordWriter('test-housing-wCat.tfrecord') as f:
    for row in housing.iterrows():
        myDict={'num_feats':Feature(float_list=FloatList(value=row[1][num_cols].values))
                'cat_feat':Feature(bytes_list=BytesList(value=[
                    row[1]['ocean_proximity'].encode())) )
        myExamp=Example(features=Features(feature=myDict))
        f.write(myExamp.SerializeToString())

feature_description3={'num_feats':tf.io.FixedLenFeature([len(num_cols)],
    tf.float32,default_value=np.zeros(len(num_cols))),
    'cat_feat':tf.io.FixedLenFeature((),tf.string,default_value="")}

def parse_examples3(serialized_examples):
    examples=tf.io.parse_example(serialized_examples,feature_description3)
    targets=examples['num_feats'][:, -1]
    examples['num_feats']=examples['num_feats'][:, :-1]
    return examples, targets

dataset=tf.data.TFRecordDataset(['test-housing-wCat.tfrecord']).batch(128).map(
    parse_examples3).cache()
```

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

How to Work with Heterogeneous Data - Handling Preprocessing

We now have a dictionary valued dataset w/ categorical and numerical features separated. Define preprocessing layers for categorical separately from numerical as follows.

```
# preprocessor for categorical feature
nTokens=5 # already know this from dataset
catEncoder=tf.keras.layers.StringLookup(max_tokens=nTokens,num_oov_indices=0)
catEncoder.adapt(dataset.map(lambda x,y:x['cat_feat']))
# demonstrate it working
for el in dataset.map(lambda x,y:x['cat_feat']).take(1):
    print(el)
    print(catEncoder(el))
# preprocessor for numerical features
normalizer=tf.keras.layers.Normalization(axis=-1)
normalizer.adapt(dataset.map(lambda x,y: x['num_feats']))
```

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

How to Work with Heterogeneous Data - Building Model

Model uses dictionary of inputs as follows

```
# constructing keras model
```

```
inputDict={'num_feats':tf.keras.Input(len(num_cols)-1,dtype=tf.float32),  
          'cat_feat':tf.keras.Input((),dtype=tf.string)}
```

```
# categorical feature should also be embedded
```

```
catInts=catEncoder(inputDict['cat_feat'])
```

```
embDim=2
```

```
catEmbs=tf.keras.layers.Embedding(nTokens,embDim)(catInts)
```

```
# now that embedded, same tf.float32 type as normalized, so can concat
```

```
preproced=tf.concat([normalizer(inputDict['num_feats']),catEmbs],axis=-1)
```

```
# now preproced can be fed in to dense layers
```

```
restMod=tf.keras.Sequential([tf.keras.layers.Dense(128,activation='relu'),  
                             tf.keras.layers.Dense(1)])
```

```
decs=restMod(preproced)
```

```
whole_model=tf.keras.Model(inputs=inputDict,outputs=decs)
```

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. A Quick Peek Under the Hood in TensorFlow Computations
2. Other Parts of TensorFlow you may use
3. Convenient Dataset Commands
 - (a) batch, filter, take, map, shuffle, cache, prefetch
4. Converting your data to easily load into Data API w/ tfrecords
5. Using Keras Preprocessing Layers to Handle Simple Data Processing
6. How to Work with Heterogeneous Data
 - (a) Datasets of Dictionaries
 - i. Saving Them
 - ii. Reloading them & Preprocessing
 - iii. Making a Keras Model For them
7. Customizing Keras
 - (a) Custom Loss Functions
 - (b) Custom Layer

Customizing Keras on TensorFlow

- Can customize, e.g.
 - Loss Function (also extend call)
 - layer (also extend build, call, compute_output_shape)
- **Best Method** is to subclass the associated class
 - put necessary code into call method
 - by including the `get_config` method, you can ensure that hyperparameters you define and set are saved along with the object

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

i. Custom Losses

ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Linear Regression

- Classification: Credit approval (yes / no)
- Regression: Credit line (dollar amount)

- Same input, $\mathbf{x} =$

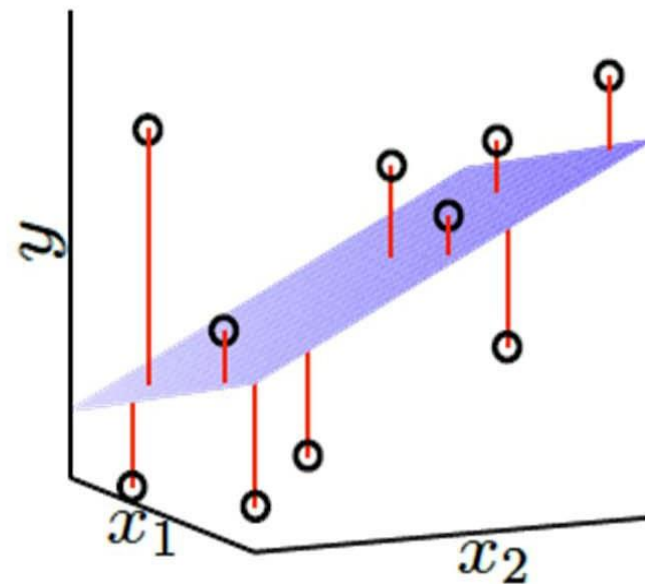
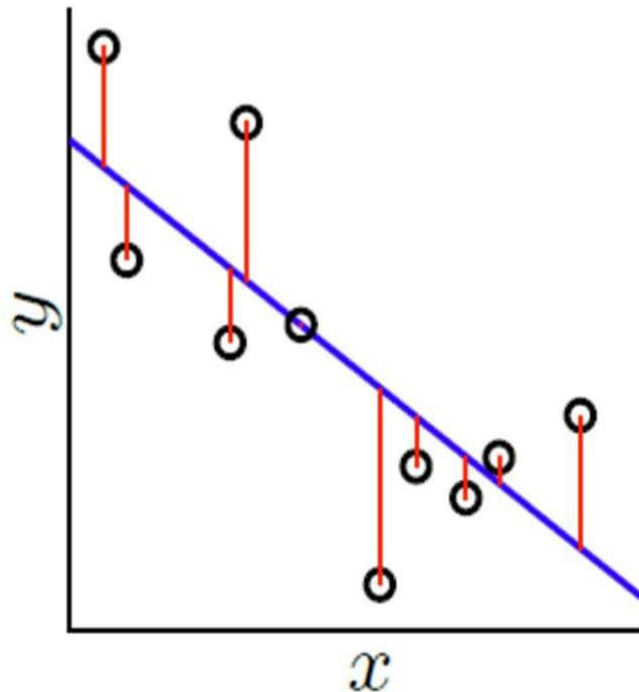
age	23 years
annual salary	\$30,000
years in residence	1 year
years in job	1 year
current debt	\$15,000
...	...

- Linear regression output:

$$h(\mathbf{x}) = \sum_{i=0}^d w_i x_i = \mathbf{w}^T \mathbf{x}$$

Linear Regression Visually

- Circles are **training examples**
- Line/Plane represent the **hypothesis**
- Red lines represent **in-sample error**



Mean squared error (MSE)

Mean squared error (MSE) can be computed by taking the actual value and predicted value as the inputs and returning the error via the below equation (mean squared error equation).

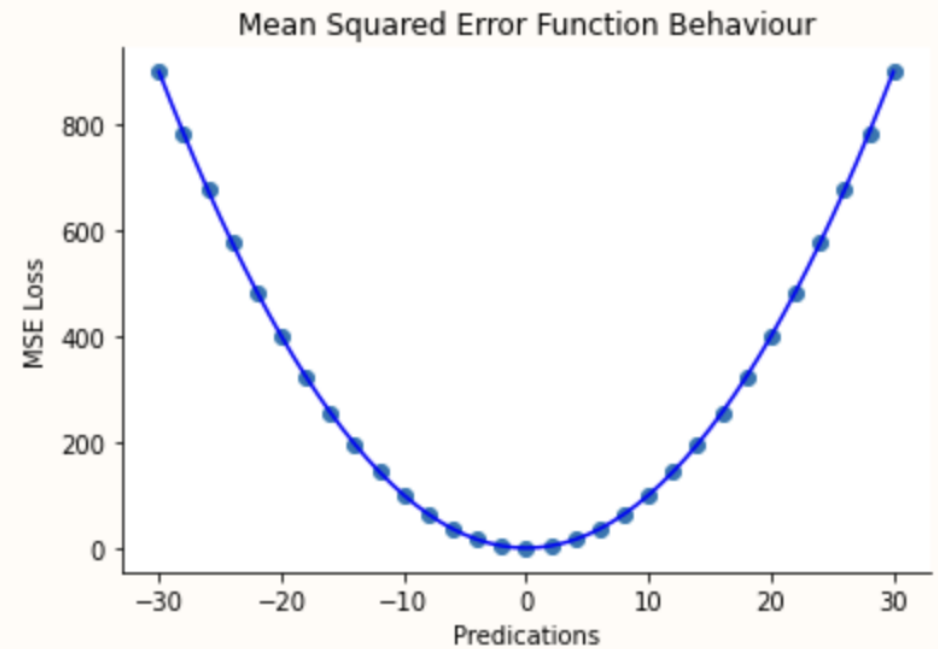
$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Advantages

- Simple Equation, Can be optimized easily compared to other equations.
- Has proven its ability over the years, this is majorly used in fields like signal processing

Disadvantages

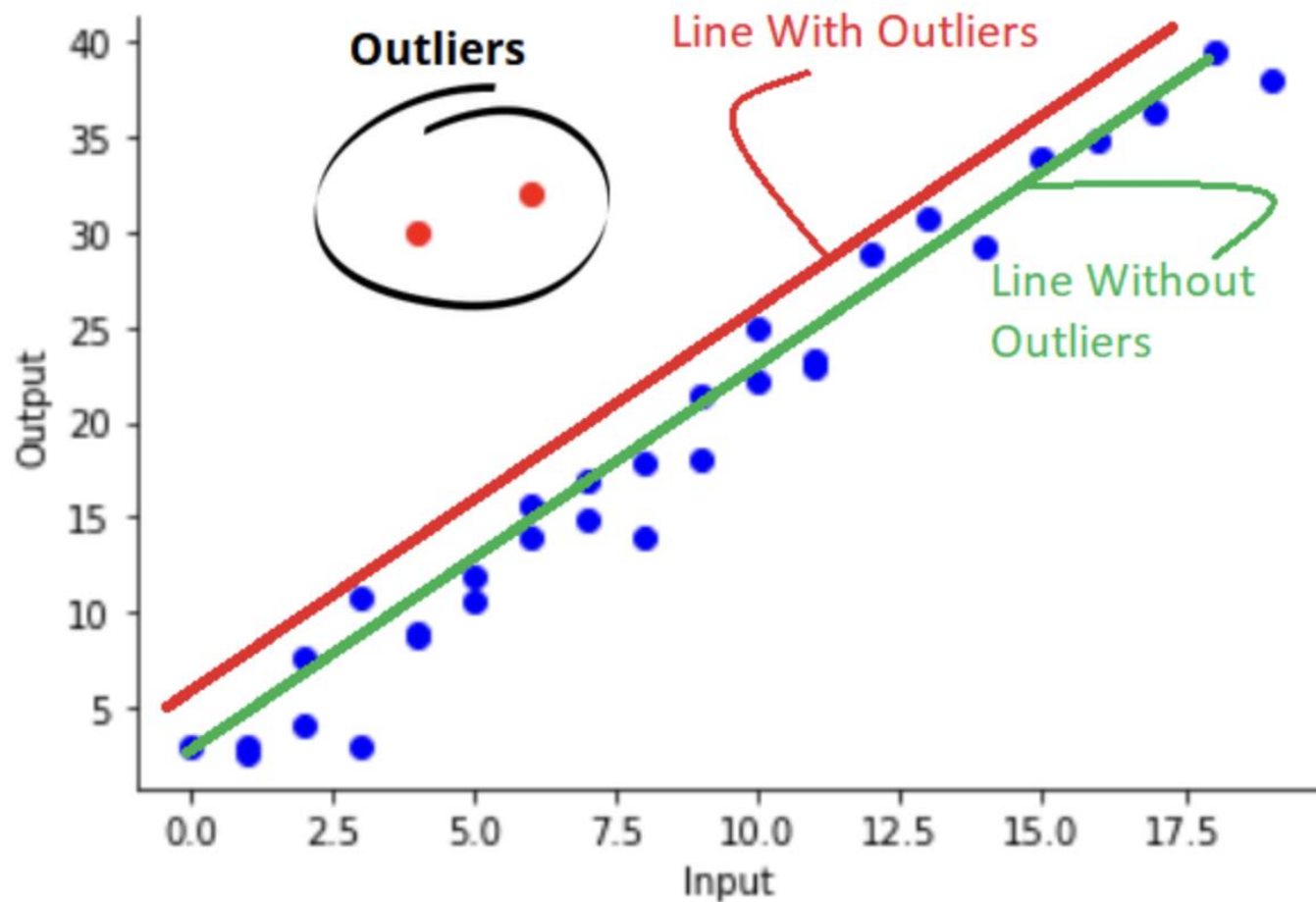
- **It is highly affected by Outliers.** So if the data contains outliers, better not to use it.



MSE loss vs prediction error

Mean squared error (MSE)

The linear regression line with MSE loss will tend to align with those outliers.



How do outliers affect the solution?

Mean Absolute Error

Like MSE, Mean Absolute Error (MAE) gives you how far/close are you from the prediction value, but the twist is here instead of squaring you will be performing the mod operation. Mathematically it can be written as

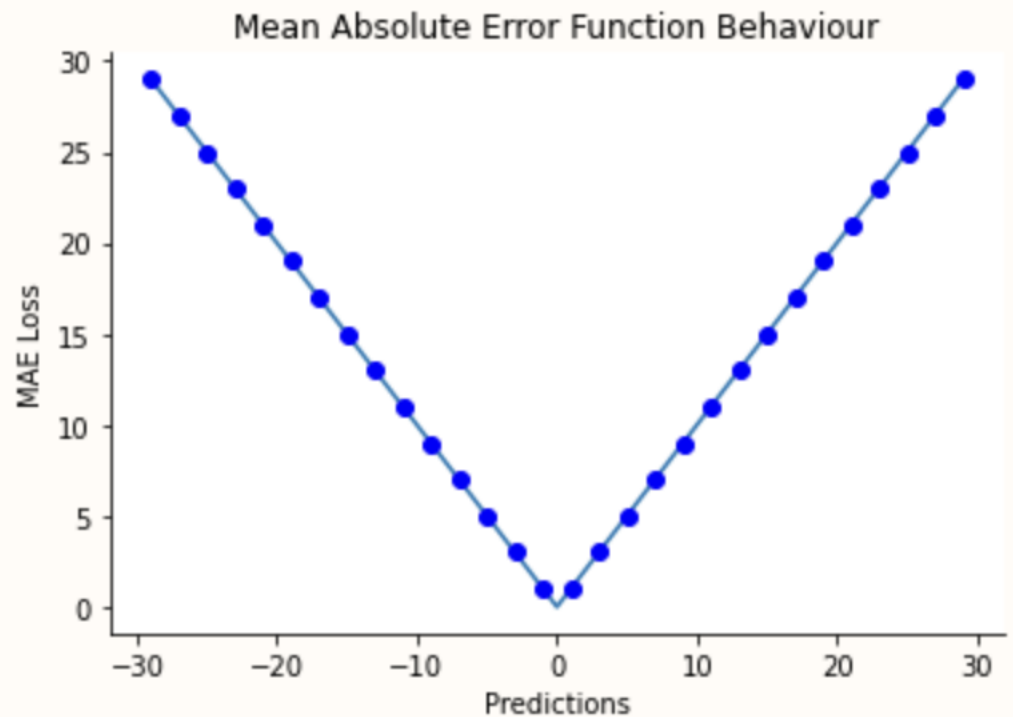
$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Advantages

- It is not much affected by outliers. So if the data contains outliers, better to use MAE over MSE as the loss function.

Disadvantages

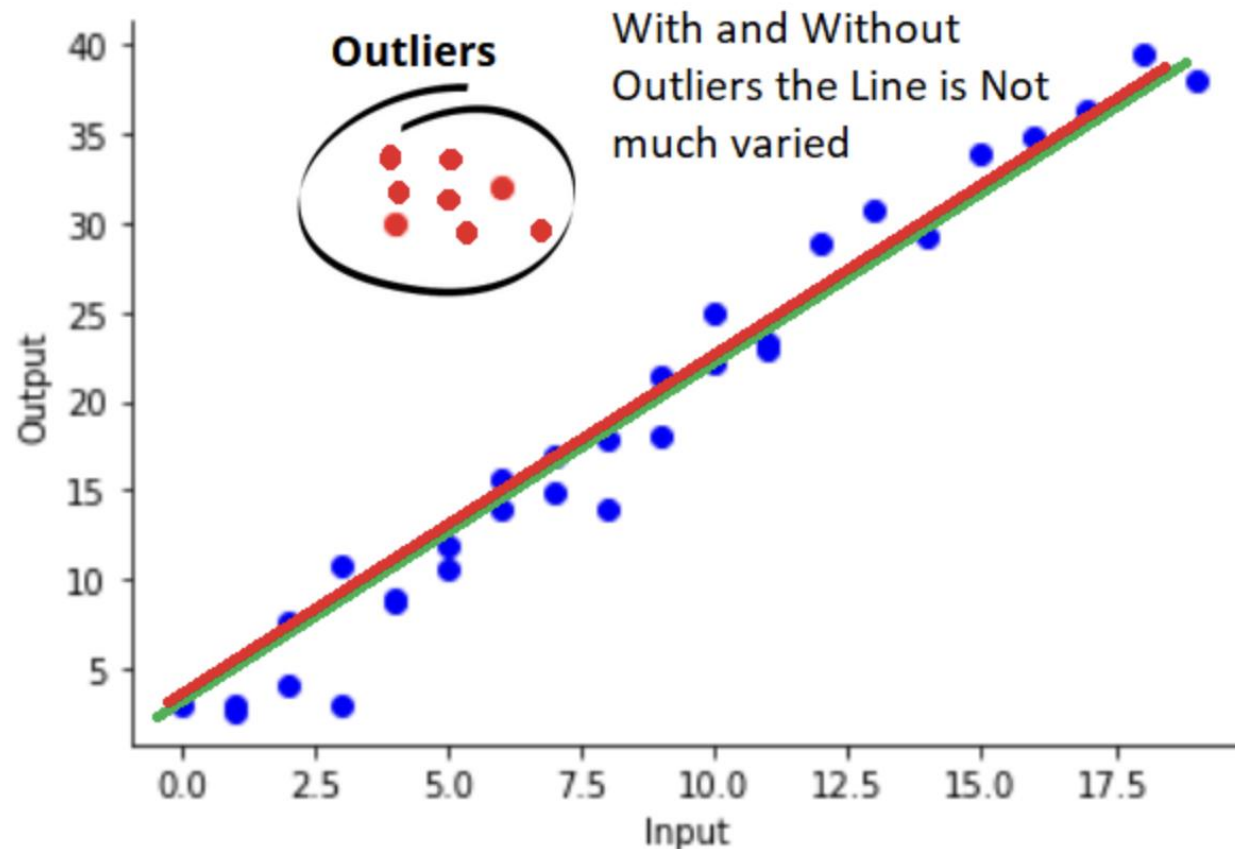
- The Optimization is a little bit complex compared to MSE



MAE loss function vs prediction error

Mean Absolute Error

The drawback is if there are more data with irregular patterns (which may not be an outlier and might be useful data), the MAE will ignore those as well. The below figure shows this:



Linear Regression Line for MAE Loss Function with a more irregular pattern

Huber Loss

Huber Loss can be interpreted as a combination of the Mean squared loss function and Mean Absolute Error. The equation is:

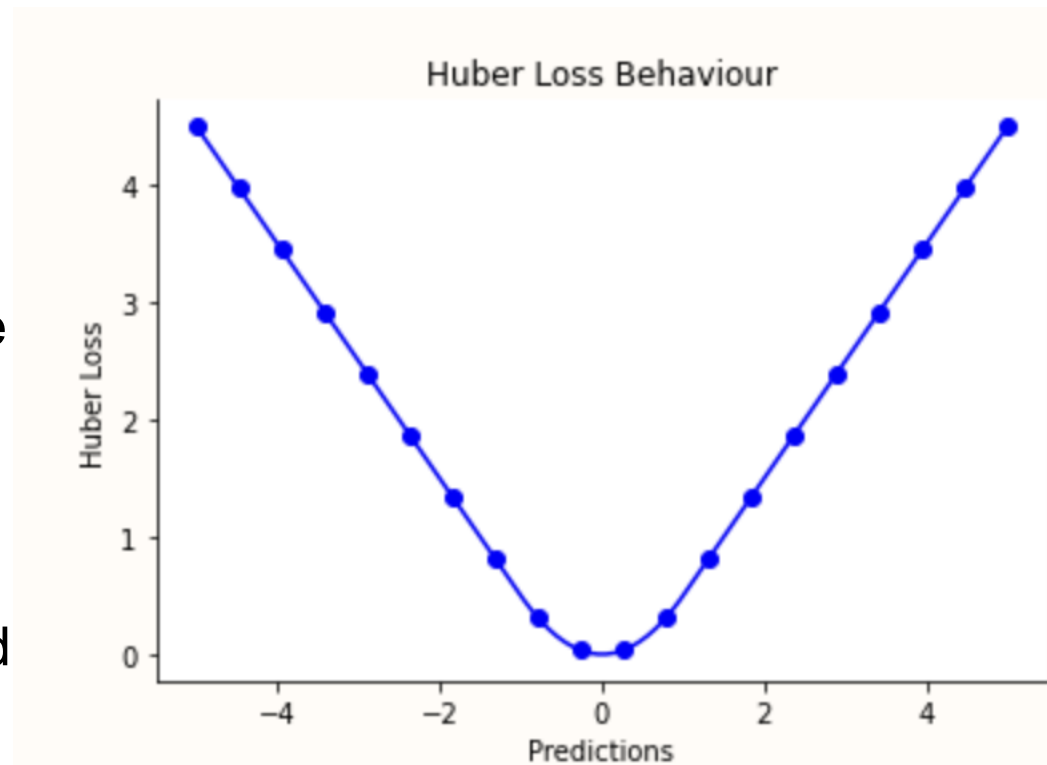
$$L_{\delta}(a) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta \cdot (|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

Advantages

- The **Huber loss is effective when there are Outliers in data**
- The optimization is easy, as there are no non-differentiable points

Disadvantages

- The equation is a bit complex and we also need to **adjust the δ based on our requirement**

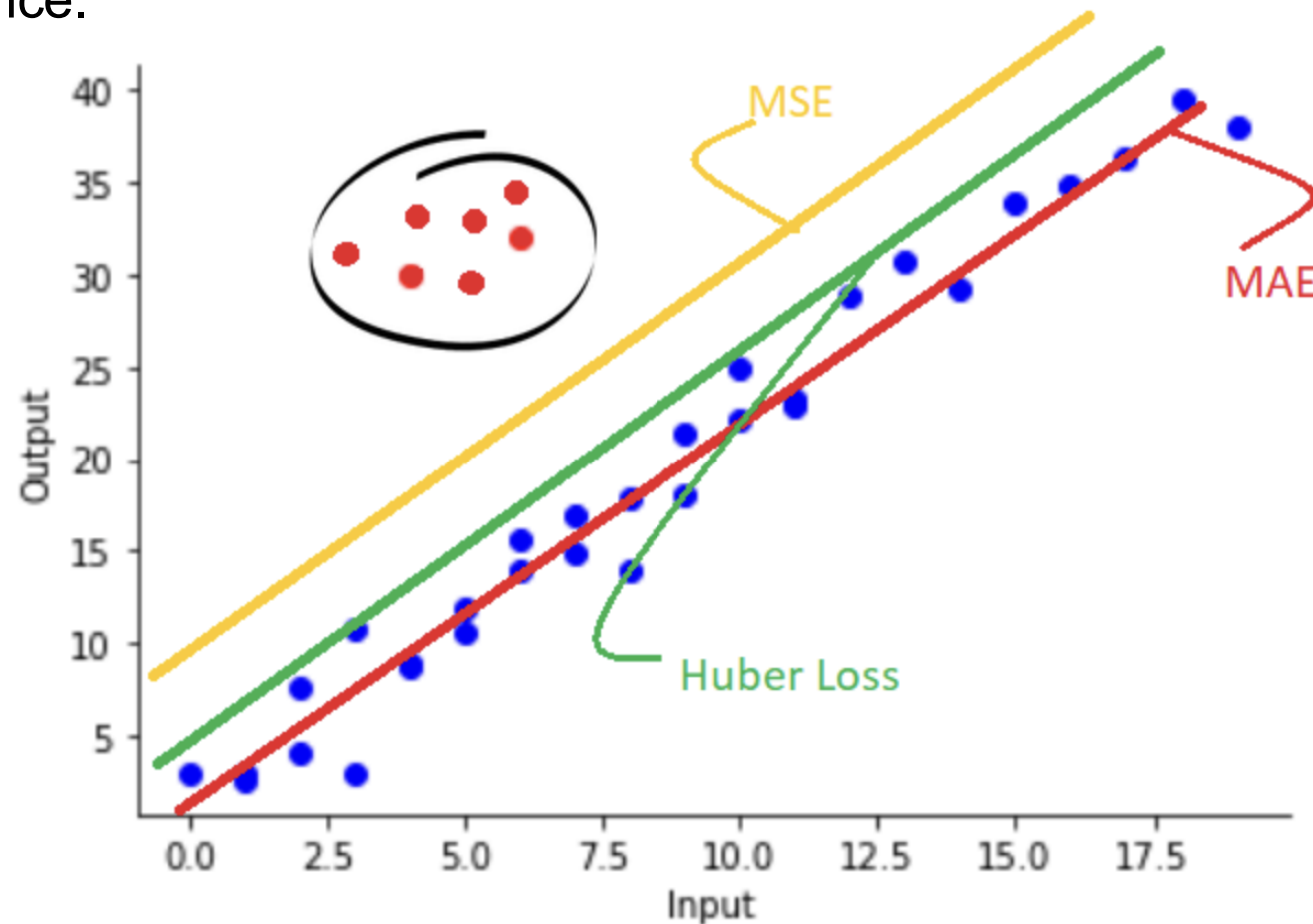


Huber loss vs prediction error

Huber Loss

Huber loss can be useful when we need the balance of Mean squared error and mean absolute error.

The MAE will completely ignore the outliers (even if it contains 20–30% of data), but Huber loss can prevent the outliers to some extent, but if the outliers are large it will make a balance.



Huber loss and outliers

Customizing Keras on TensorFlow

– Your Own Loss

```
class HuberLoss(keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)
    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold**2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Customizing Keras on TensorFlow

– Your Own Loss

- The `get_config()` method returns a dictionary mapping each hyperparameter name to its value. It first calls the parent class's `get_config()` method, then adds the new hyperparameters to this dictionary (note that the convenient `**x` syntax was added in Python 3.5).

You can then use any instance of this class when you compile the model:

```
model.compile(loss=HuberLoss(2.), optimizer="nadam")
```

When you save the model, the threshold will be saved along with it, and when you load the model you just need to map the class name to the class itself:

```
model = keras.models.load_model("my_model_with_a_custom_loss_class.h5",  
                                custom_objects={"HuberLoss": HuberLoss})
```

When you save a model, Keras calls the loss instance's `get_config()` method and saves the config as JSON in the HDF5 file. When you load the model, it calls the `from_config()` class method on the `HuberLoss` class: this method is implemented by the base class (`Loss`) and just creates an instance of the class, passing `**config` to the constructor.

Customizing Keras on TensorFlow

– Your Own Loss

TensorFlow saves the configuration and parameters of the custom classes with the model, but not the class definitions themselves. Therefore, the class definitions must be present to recreate the model correctly upon loading.

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

i. Custom Losses

ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Customizing Keras on TensorFlow

– Your Own Layer

```
class MyDense(keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias", shape=[self.units], initializer="zeros")
        super().build(batch_input_shape) # must be at the end

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def compute_output_shape(self, batch_input_shape):
        return tf.TensorShape(batch_input_shape.as_list()[:-1] + [self.units])

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "units": self.units,
            "activation": keras.activations.serialize(self.activation)}
```

Customizing Keras on TensorFlow

– Layers – Train/Test Dependent Behavior

```
class MyGaussianNoise(keras.layers.Layer):
    def __init__(self, stddev, **kwargs):
        super().__init__(**kwargs)
        self.stddev = stddev

    def call(self, X, training=None):
        if training:
            noise = tf.random.normal(tf.shape(X), stddev=self.stddev)
            return X + noise
        else:
            return X

    def compute_output_shape(self, batch_input_shape):
        return batch_input_shape
```

Example: Custom Preprocessing Layer

```
class myImputer(tf.keras.layers.experimental.preprocessing.PreprocessingLayer):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
    def build(self, batch_input_shape):
        self.imps=self.add_weight(name='imps', shape=(batch_input_shape[-1]),
                                   initializer="zeros", trainable=False)
        super().build(batch_input_shape)

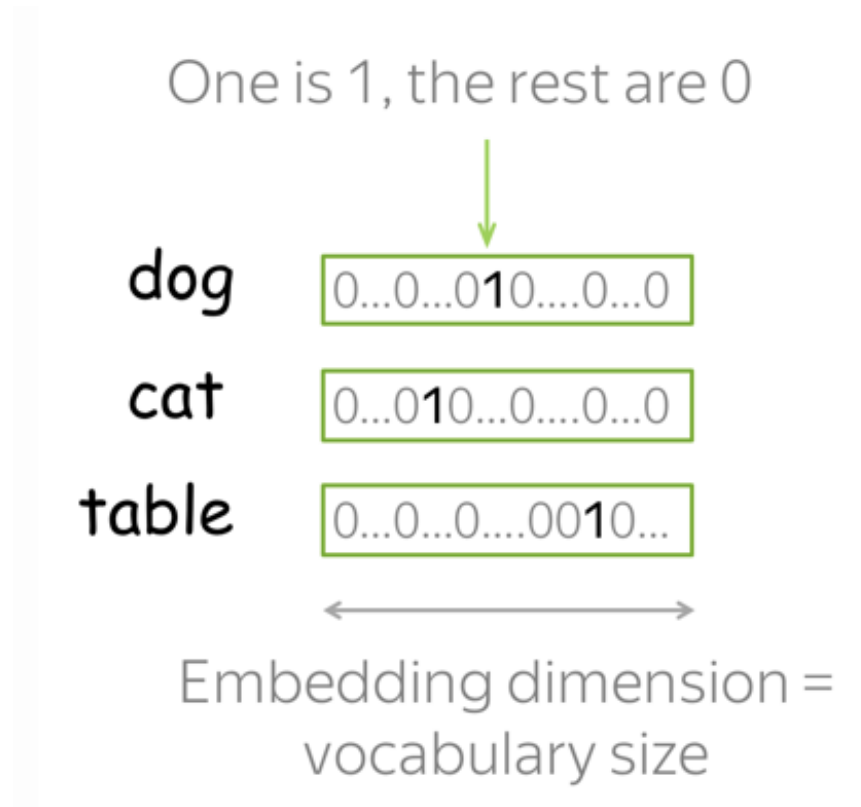
    def call(self, X):
        return tf.where(tf.math.is_nan(X), self.imps, X)

    def adapt(self, dataset):
        self.build(dataset.element_spec.shape)
        sumOfNonNaNs=dataset.map(lambda z: tf.where(tf.math.is_nan(z),
                                                    tf.zeros_like(z), z)).reduce(
                                                    tf.zeros_like(self.imps),
                                                    lambda x,y: x+tf.reduce_sum(y,axis=0))
        numberOfNonNaNs=dataset.map(lambda z: tf.where(tf.math.is_nan(z),
                                                    tf.zeros_like(z), tf.ones_like(z))).reduce(
                                                    tf.zeros_like(self.imps),
                                                    lambda x,y: x+tf.reduce_sum(y,axis=0))
        self.imps.assign(tf.math.divide(sumOfNonNaNs, numberOfNonNaNs))
    def computer_output_shape(self, batch_input_shape):
        return batch_input_shape
```


Embedding Layers

This layer maps input information from a high-dimensional to a lower-dimensional space, allowing the network to learn more about the relationship between inputs and to process the data more efficiently.

Often use one-hot-vector



Embedding Layers

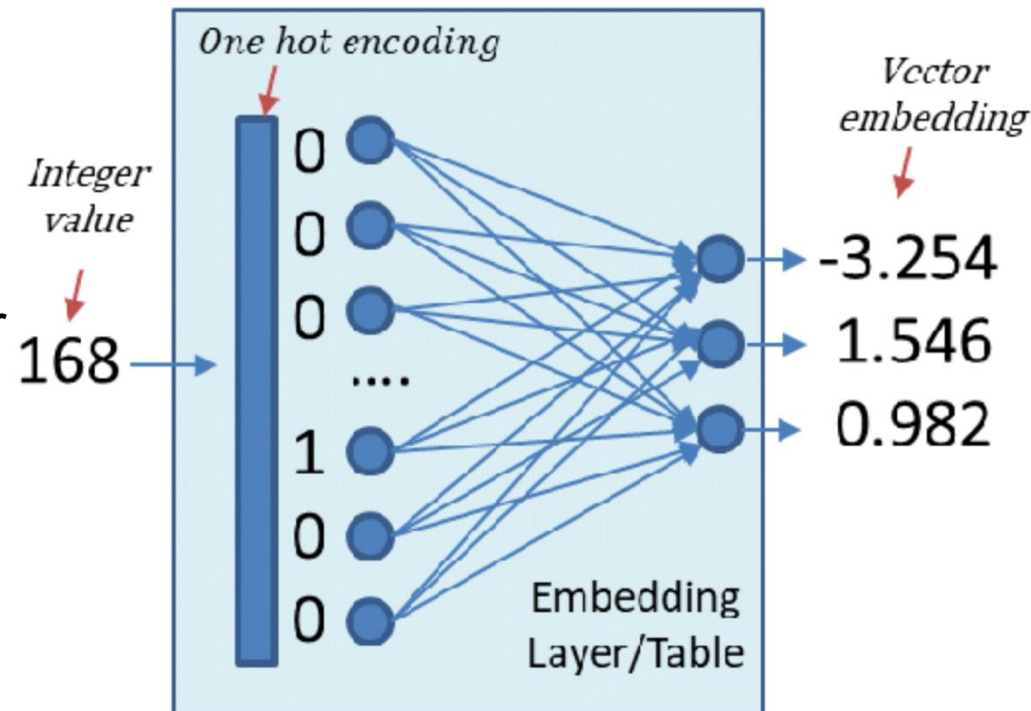
What is an Embedding layer ?

- **Definition:** A trainable dense vector representing a category.
- **Initialization:** Embeddings are initialized randomly.

Example:

- **"NEAR BAY":** Initially [0.131, 0.890]
- **"NEAR OCEAN":** Initially [0.631, 0.791]
- **Number of dimensions:** A hyperparameter to tweak (e.g., 2D here).
- **Training and Representation Learning:**
 - **Gradual Improvement:** Embeddings improve during training.
 - **Similar Categories:** Move closer together (e.g., "NEAR BAY" and "NEAR OCEAN").
 - **Different Categories:** Move apart (e.g., "NEAR BAY" and "INLAND").
 - **Goal:** Better representations lead to easier and more accurate neural network predictions.

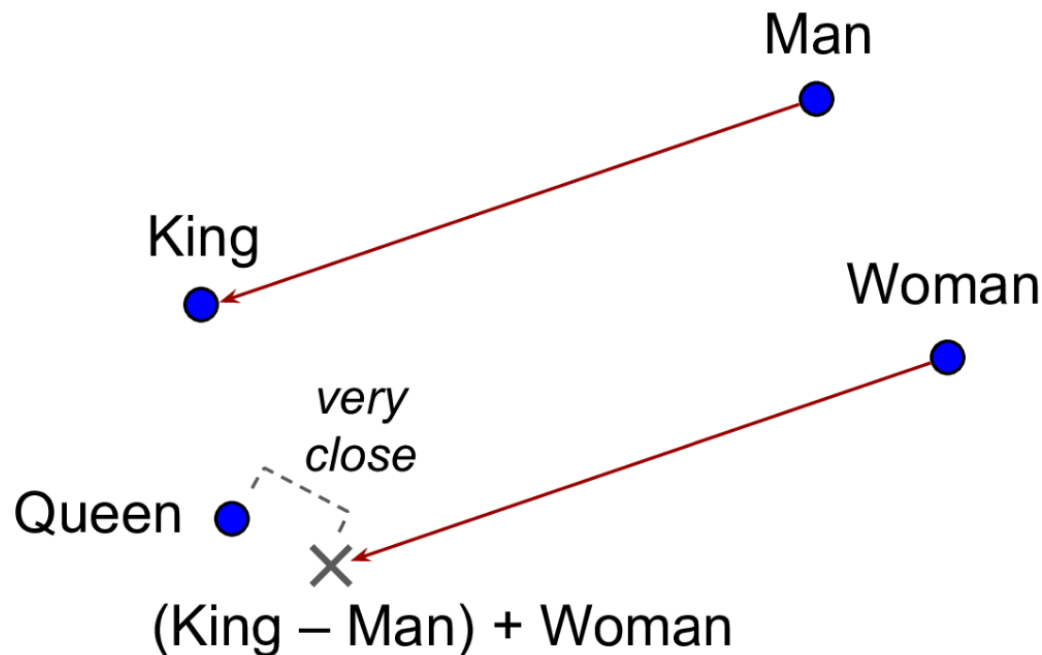
```
ocean_proximity_embed =  
tf.feature_column.embedding_column  
(ocean_proximity, dimension=2)
```



Reusing Embeddings in Natural Language Processing

Pretrained embeddings are particularly beneficial for NLP tasks, as demonstrated by the success of word embeddings in encoding meaningful semantic relationships.

Embedding Space



- **Gender Concept:**
- Calculation:
 $King - Man + Woman \approx Queen$
- Shows that embeddings encode gender.

Figure 13-5. Word Embeddings

Example: Custom Preprocessing Layer - Whole Model

chain imputer and normalizer, and adapt on numerical features

Chain Imputer and Normalizer for Numerical Features

```
myImp=myImputer()  
myImp.adapt(dataset.map(lambda x,y: x['num_feats'])))  
normalizer=tf.keras.layers.Normalization(axis=-1)  
normalizer.adapt(dataset.map(lambda x,y: myImp(x['num_feats'])))
```

Define Input Dictionary

```
# aggregate into dictionary input fed model  
inputDict={'num_feats':tf.keras.Input(len(num_cols)-1,dtype=tf.float32),  
           'cat_feat':tf.keras.Input((),dtype=tf.string)}
```

Initialize and Adapt Categorical Feature Encoder

```
nTokens=5 # inspected the number of values categorical feature 'ocean_proximty'  
catEncoder=tf.keras.layers.StringLookup(max_tokens=nTokens,num_oov_indices=0)  
catEncoder.adapt(dataset.map(lambda x,y:x['cat_feat']))
```

Encode and Embed Categorical Feature

```
catInts=catEncoder(inputDict['cat_feat'])  
embDim=2  
catEmbs=tf.keras.layers.Embedding(nTokens,embDim)(catInts)
```

Example: Custom Preprocessing Layer - Whole Model

now that embedded, same tf.float32 type as normalized, so can concat

Concatenate Preprocessed Features

```
preproced=tf.concat([normalizer(myImp(inputDict['num_feats'])),catEmbs],axis=-1)
```

Define and Compile the Model

```
restMod=tf.keras.Sequential([tf.keras.layers.Dense(128,activation='relu'),  
                             tf.keras.layers.Dense(1)])
```

Create the Complete Model

```
decs=restMod(preproced)  
whole_model=tf.keras.Model(inputs=inputDict,outputs=decs)
```

Compile the Model

```
rmse_metric=tf.keras.metrics.RootMeanSquaredError()  
whole_model.compile(loss='mse',optimizer='adam',metrics=[rmse_metric])
```

Example:Final Model - Tuning Batch Size

```
secsReqd=[]
for batchSize in [32,64,128,512,1024,2048,4096,8*1024]:
    dataset=tf.data.TFRecordDataset(['test-housing-
wCat.tfrecord']).batch(batchSize
                           ).map(parse_examples3,num_parallel_calls=tf.data.AUTOTUNE).c
    ache()
    start_time=time.time()
    modWimp.fit(dataset,epochs=5,verbose
=0)
    secsReqd.append((time.time()-
start_time))
```

After running the code, largest batch sizes (around a quarter to half of the dataset size) resulted in the best performance (fastest training times).

Example:Final Model - Splitting Data and Fitting

```
raw_dataset=tf.data.TFRecordDataset(['test-housing-Cat.tfrecord'])
datLen=raw_dataset.reduce(0,lambda x,y: x+1)
n_valid=int(datLen.numpy()*0.1)
n_test=int(datLen.numpy()*0.1)
n_train=datLen.numpy()-n_valid-n_test
dataset_tr=raw_dataset.take(n_train).batch(2048).map(
    parse_examples3,num_parallel_calls=8).cache()
dataset_ts=raw_dataset.skip(n_train).take(n_test).batch(2048).map(
    ( parse_examples3,num_parallel_calls=8).cache()
dataset_vd=raw_dataset.skip(n_train+n_test).take(n_valid
    ).batch(2048).map(parse_examples3,num_parallel_calls=8).cache()

myOpt=tf.keras.optimizers.Adam(learning_rate=5)
modWimp.compile(loss='mse',optimizer=myOpt,metrics=[
    tf.keras.metrics.RootMeanSquaredError()])
modWimp.fit(dataset_tr,epochs=500,verbose=2,validation_data=dataset_vd,callbacks
=[
    tf.keras.callbacks.EarlyStopping(patience=10,restore_best_weights=True)])

modWimp.evaluate(dataset_ts)
```

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Datasets: MNIST



10 classes: Digits 0 to 9
28x28 grayscale images
50k training images
10k test images

Datasets: CIFAR10

airplane

automobile

bird

cat

deer

dog

frog

horse

ship

truck



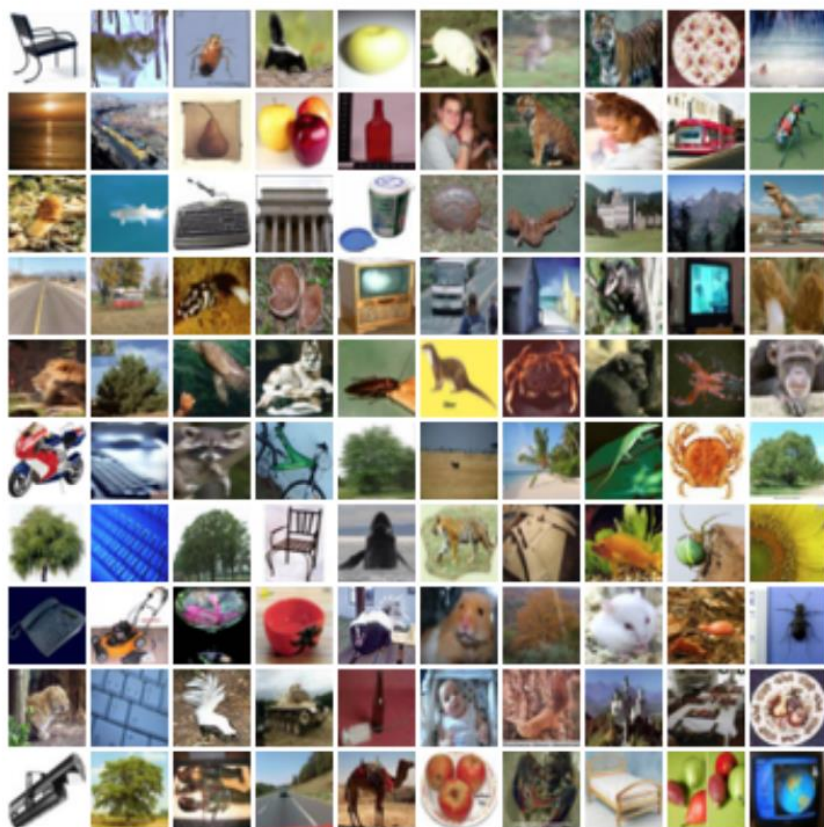
10 classes

50k training images (5k per class)

10k testing images (1k per class)

32x32 RGB images

Datasets: CIFAR100



100 classes

50k training images (500 per class)

10k testing images (100 per class)

32x32 RGB images

20 superclasses with 5 classes each:

Aquatic mammals: beaver, dolphin, otter, seal, whale

Trees: Maple, oak, palm, pine, willow

Datasets: ImageNet

1000 classes

~1.3M training images (~1.3K per class)

50K validation images (50 per class)

100K test images (100 per class)

test labels are secret!



flamingo

cock

ruffed grouse

quail

partridge

...



Egyptian cat

Persian cat

Siamese cat

tabby

lynx

...



dalmatian

keeshond

miniature schnauzer

standard schnauzer

giant schnauzer

Images have variable size, but often
resized to **256x256** for training

There is also a 22k category version of
ImageNet, but less commonly used

Datasets: ImageNet

1000 classes

~1.3M training images (~1.3K per class)

50K validation images (50 per class)

100K test images (100 per class)

Performance metric: **Top 5 accuracy**
Algorithm predicts 5 labels for each image; one of them needs to be right



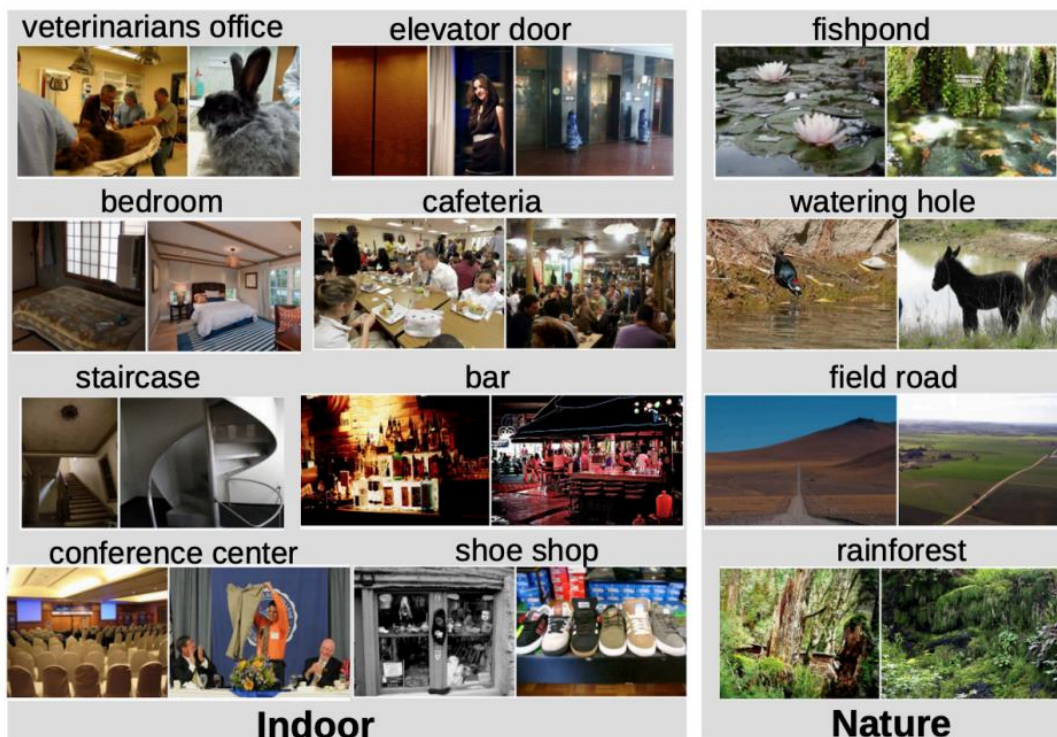
ImageNet - A Particularly Successful Dataset & Competition

- **ImageNet Large Scale Visual Recognition Challenges (ILSVRC)?**

[homepage](#), [summary article](#) A series of annual competitions from 2010-2017 consisting of different tasks including

- Image classification: each image is associated with one of [1000 object categories \(synsets\)](#) for its contents. 1000 Categories same since 2012. Rank likelihood of each of these categories.
 - Single Object localization: also give bounding box for locations of those objects
 - Object detection: localize multiple object categories (200) in the image
 - Object detection from video (2017): 30 categories tracked in video
- Training data subsamples ImageNet and is augmented for validation and testing
 - **“trained on ImageNet”?** - typically a model designed to solve the image classification ILSVRC (outputs prob.s for [1000](#) classes).

Datasets: MIT Places



365 classes of different scene types

~8M training images

18.25K val images (50 per class)

328.5K test images (900 per class)

Images have variable size, often
resize to **256x256** for training

TensorFlow Datasets

separate python package `tensorflow_datasets` contains some well-known common datasets already converted into a TensorFlow Dataset.

load the package

```
import tensorflow_datasets as tfds
```

The titles of the presently available datasets

```
print(tfds.list_builders())
```

For instance, MNIST can be loaded with

```
datasets = tfds.load(name="mnist")  
mnist_train, mnist_test = datasets["train"], datasets["test"]
```


TensorFlow Datasets

Beauty of using this module is that it can be fed right into `tf.keras`, e.g.

```
datasets = tfds.load(name="mnist", batch_size=32,
as_supervised=True)

mnist_train = datasets["train"].repeat().prefetch(1)
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28, 1]),
    keras.layers.Lambda(lambda images: tf.cast(images,
tf.float32)),
    keras.layers.Dense(10, activation="softmax")])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=keras.optimizers.SGD(lr=1e-3),
              metrics=["accuracy"])
model.fit(mnist_train, steps_per_epoch=60000 // 32, epochs=5)
```

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

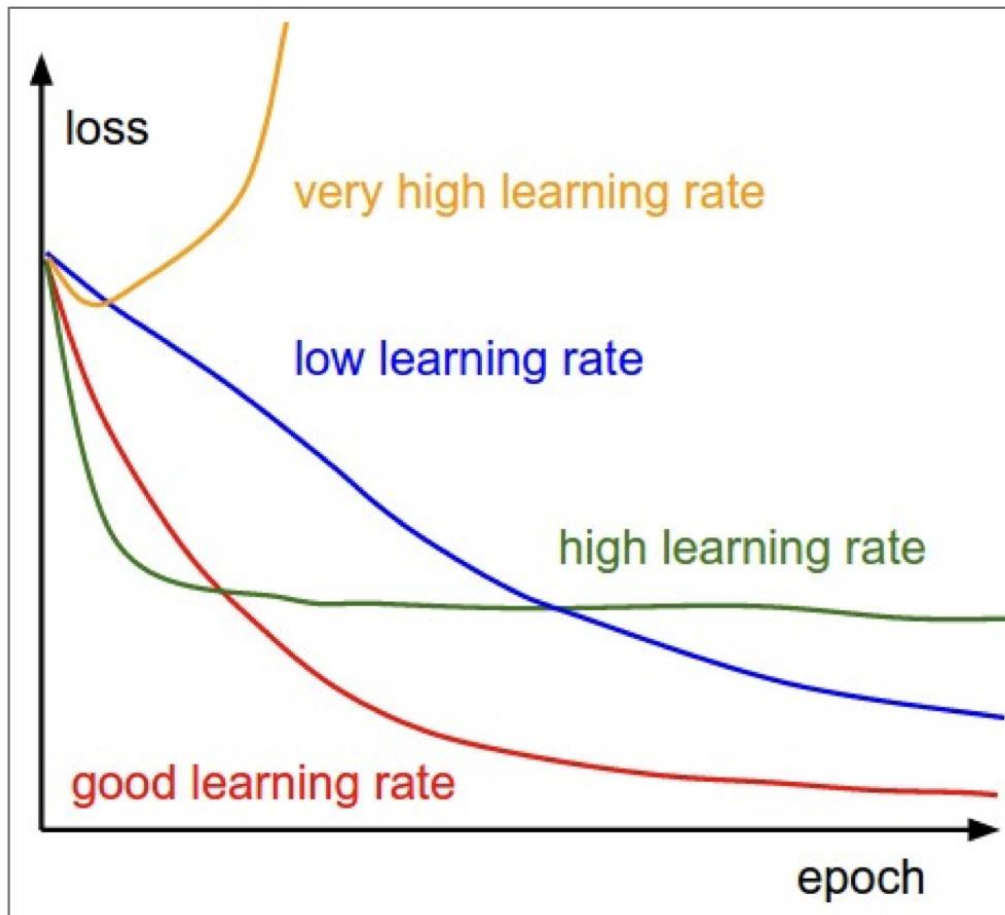
(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Learning Rate Schedules

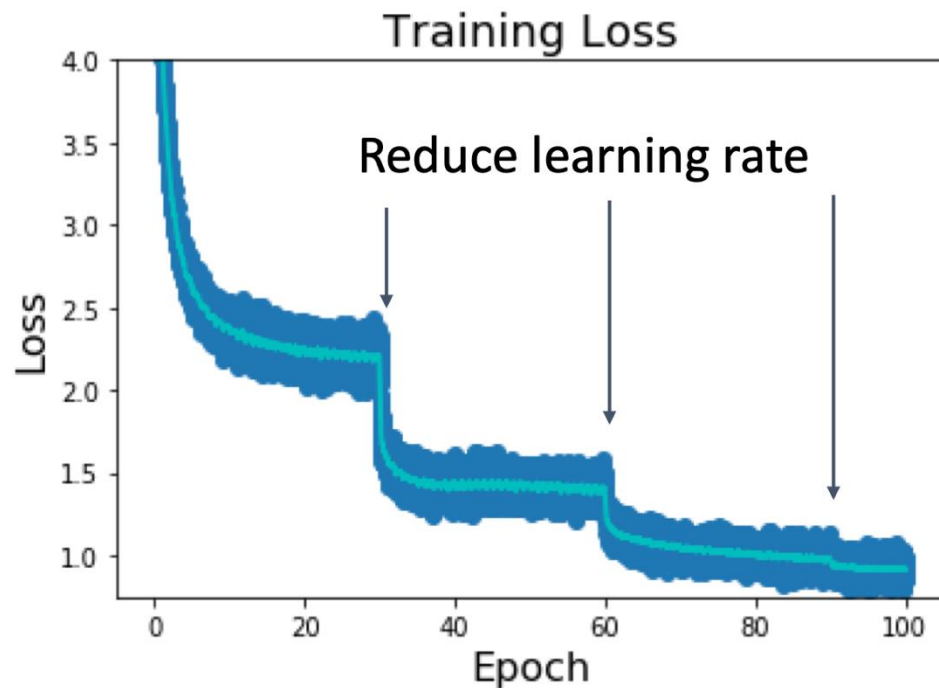
SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



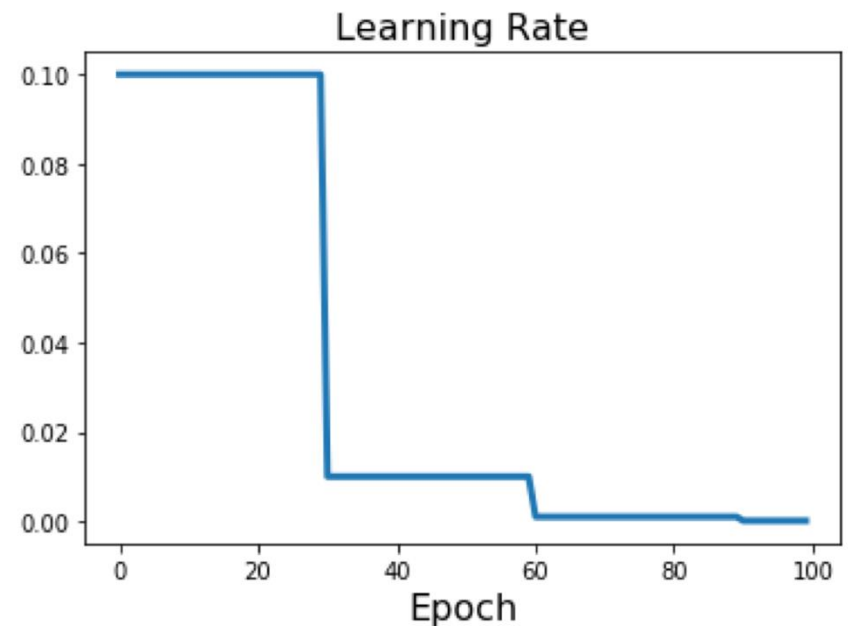
Q: Which one of these learning rates is best to use?

A: All of them! Start with large learning rate and decay over time

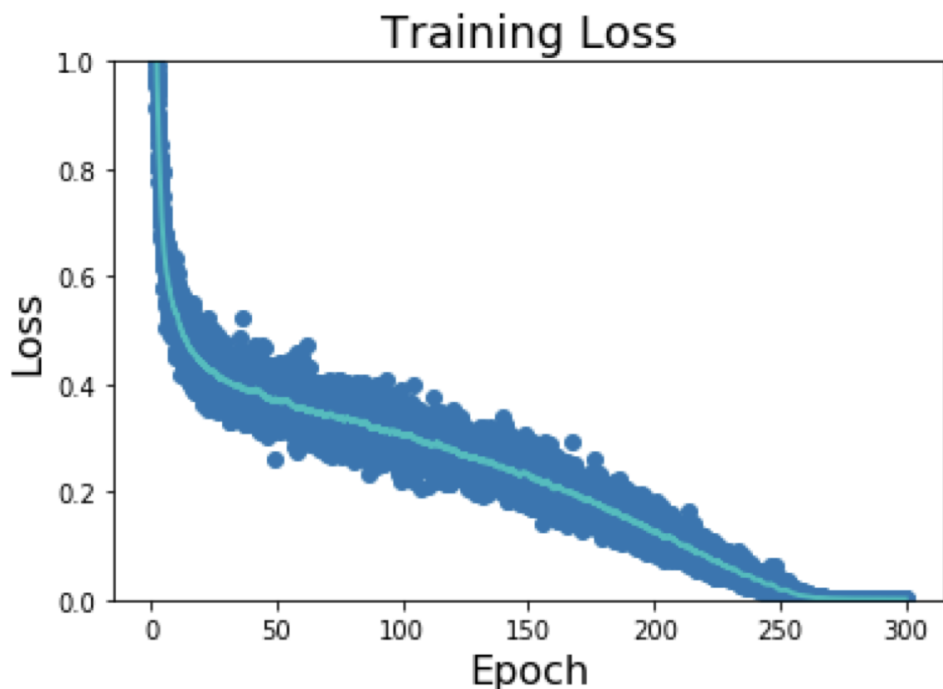
Learning Rate Decay: Step



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

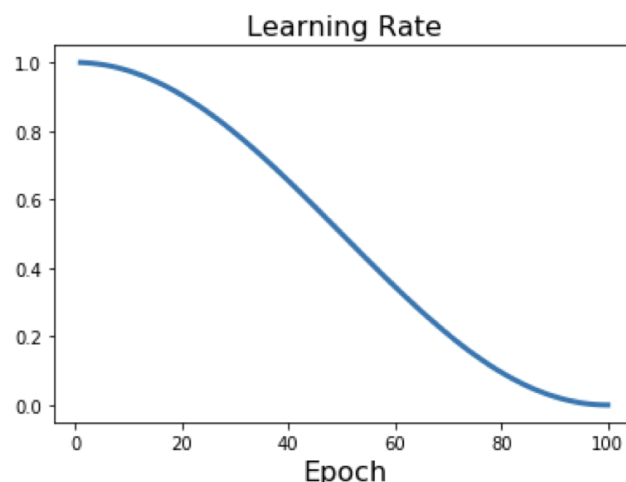


Learning Rate Decay: Cosine



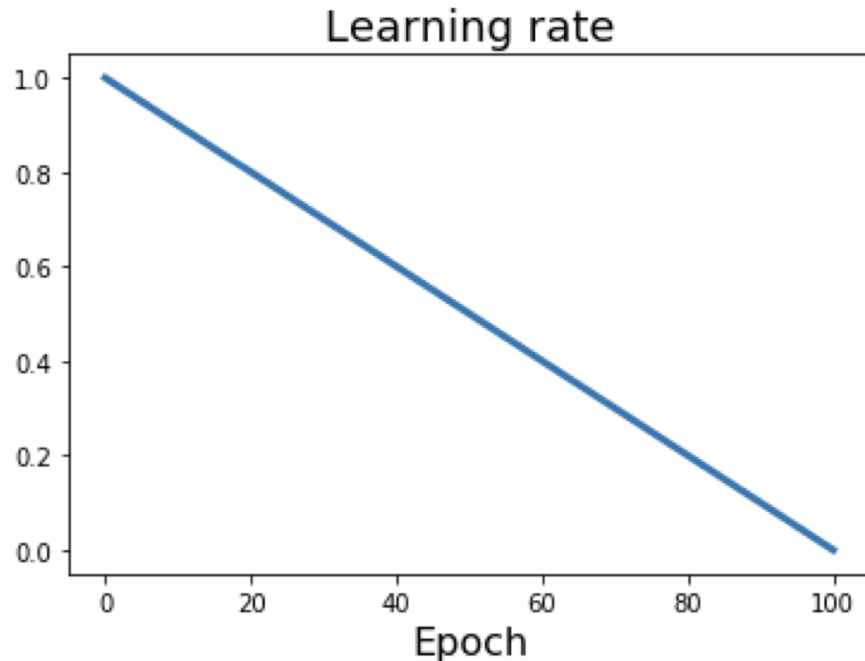
Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(t\pi/T))$$



Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", ICCV 2019
Radosavovic et al, "On Network Design Spaces for Visual Recognition", ICCV 2019
Child et al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Learning Rate Decay: Linear



Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

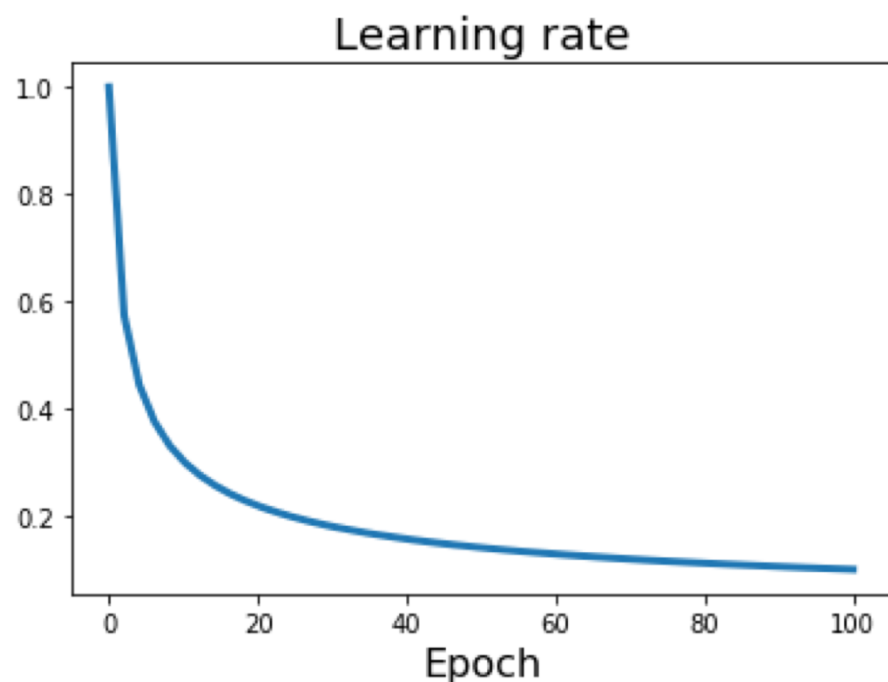
Linear: $\alpha_t = \alpha_0(1 - t/T)$

Devlin et al, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL 2018

Liu et al, "RoBERTa: A Robustly Optimized BERT Pretraining Approach", 2019

Yang et al, "XLNet: Generalized Autoregressive Pretraining for Language Understanding", NeurIPS 2019

Learning Rate Decay: Inverse Sqrt



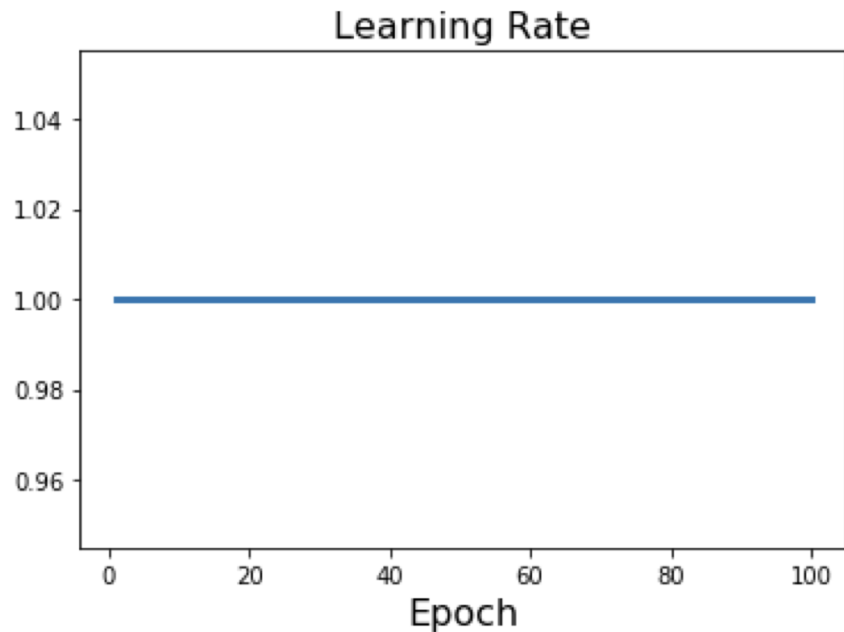
Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

Learning Rate Decay: Constant!



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

Constant: $\alpha_t = \alpha_0$

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

Define the learning rate schedule function

```
def lr_schedule(epoch, lr):
    if epoch == 10:
        return lr * 0.1
    elif epoch == 20:
        return lr * 0.1
    return lr
```

Learning Rate Schedule Function (lr_schedule): This function defines the logic for adjusting the learning rate. In this example, the learning rate is reduced by a factor of 10 at epoch 10 and epoch 20.

Create the LearningRateScheduler callback

```
lr_scheduler = tf.keras.callbacks.LearningRateScheduler(lr_schedule)
```

Create a simple model

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax') ])
```

LearningRateScheduler Callback: This callback applies the learning rate schedule function at the beginning of each epoch.

Compile the model

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Load and normalize the data

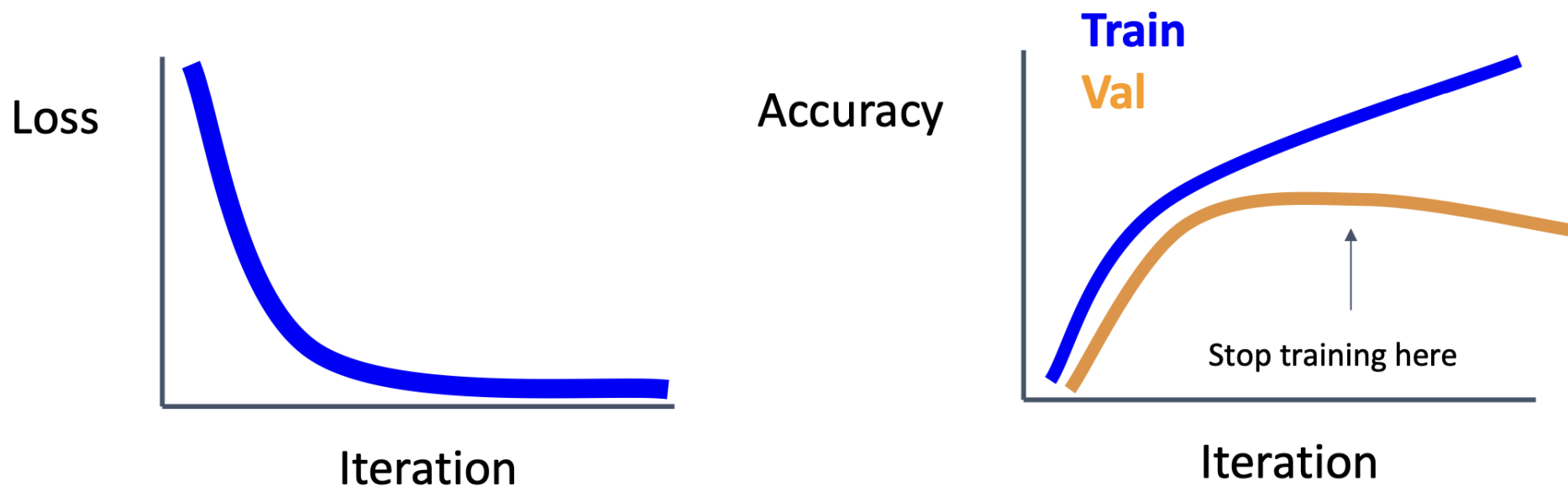
```
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()  
X_train, X_test = X_train / 255.0, X_test / 255.0
```

Train the model with the LearningRateScheduler callback

```
model.fit(X_train, y_train, epochs=30, validation_data=(X_test, y_test),  
callbacks=[lr_scheduler])
```

Model Training: The fit method trains the model and applies the learning rate adjustments at the specified epochs.

How long to train? Early Stopping



Stop training the model when accuracy on the validation set decreases
Or train for a long time, but always keep track of the model snapshot that worked best on val. **Always a good idea to do this!**

Choosing Hyperparameters

Step 1: Check initial loss

Turn off weight decay, sanity check loss at initialization
e.g. $\log(C)$ for softmax with C classes

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Try to train to 100% training accuracy on a small sample of training data (~5-10 minibatches); fiddle with architecture, learning rate, weight initialization. Turn off regularization.

Loss not going down? LR too low, bad initialization

Loss explodes to Inf or NaN? LR too high, bad initialization

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~ 100 iterations

Good learning rates to try: $1e-1$, $1e-2$, $1e-3$, $1e-4$

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for ~1-5 epochs

Choose a few values of learning rate and weight decay around what worked from Step 3, train a few models for ~1-5 epochs.

Good weight decay to try: $1e-4$, $1e-5$, 0

```
model = Sequential([  
    Dense(128, activation='relu', kernel_regularizer=l2(1e-4), input_shape=(784,)),  
    Dense(64, activation='relu', kernel_regularizer=l2(1e-4)),  
    Dense(10, activation='softmax') ])
```


Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for ~1-5 epochs

Step 5: Refine grid, train longer

Pick best models from Step 4, train them for longer (~10-20 epochs) without learning rate decay

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

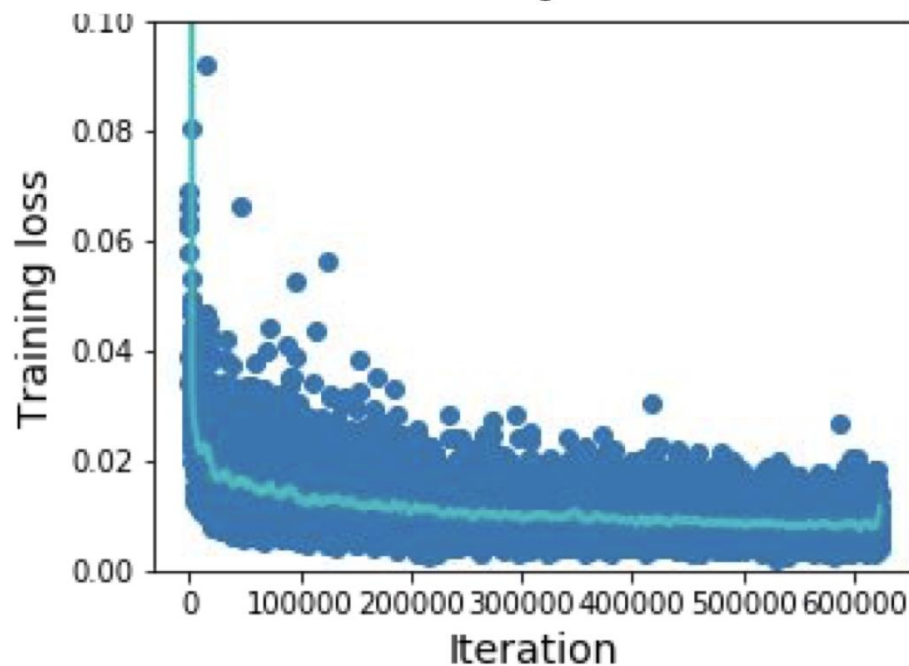
Step 4: Coarse grid, train for ~1-5 epochs

Step 5: Refine grid, train longer

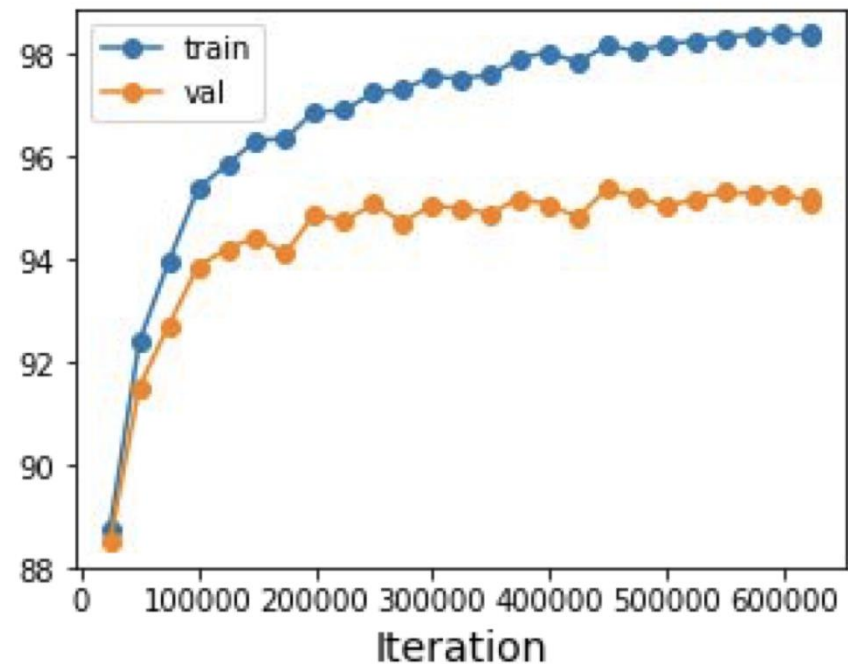
Step 6: Look at learning curves

Look at Learning Curves!

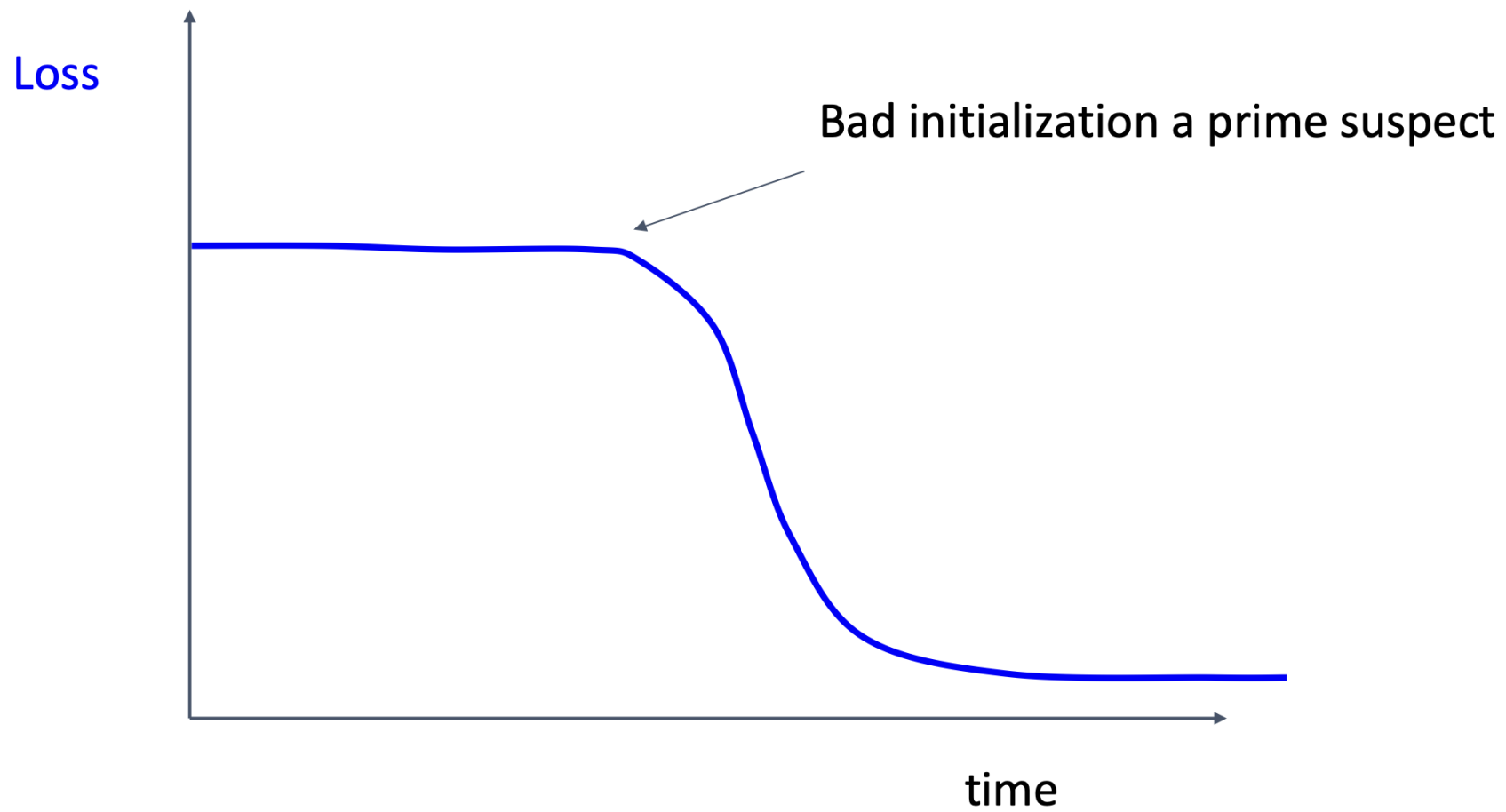
Training Loss

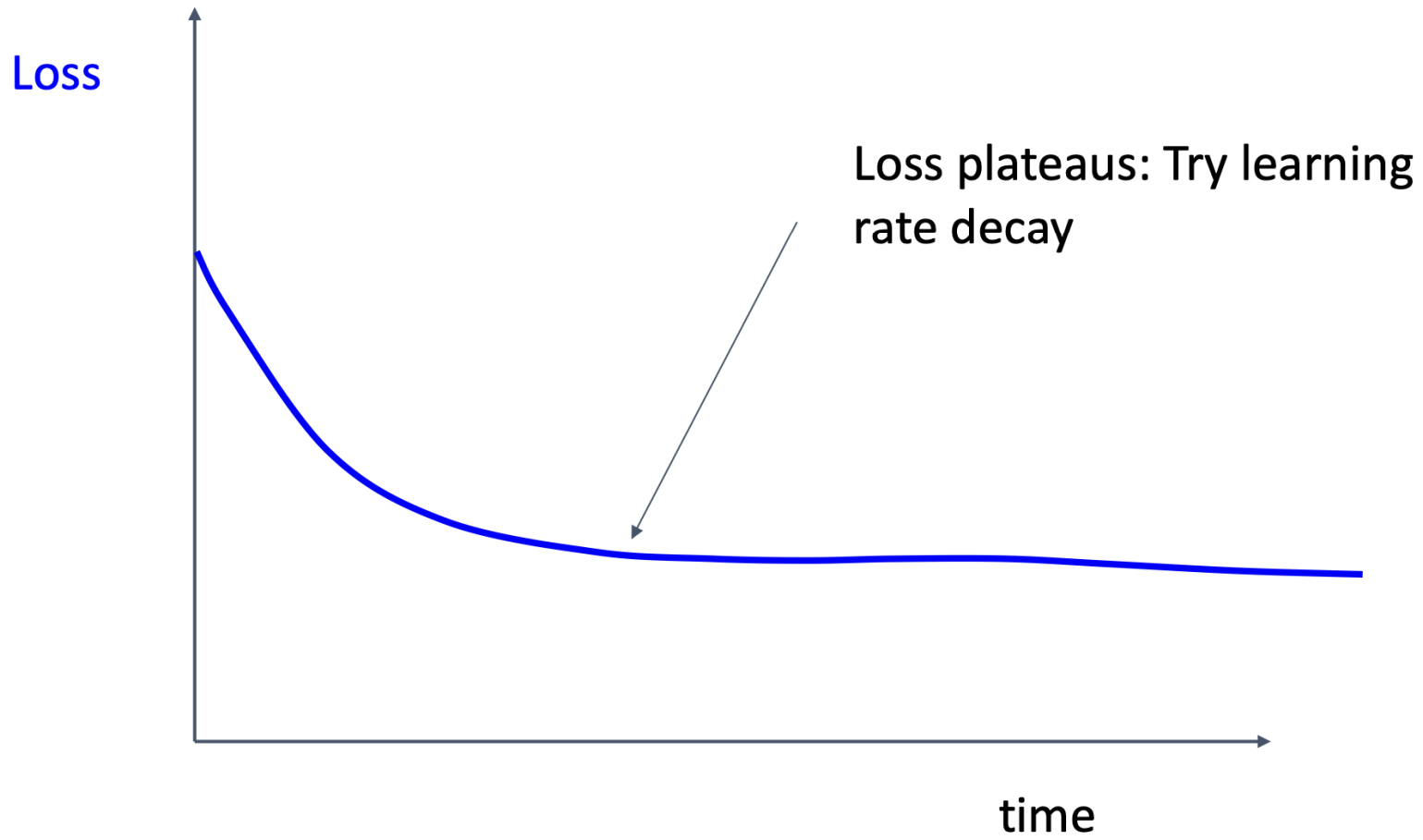


Train / Val Accuracy



Losses may be noisy, use a scatter plot and also plot moving average to see trends better

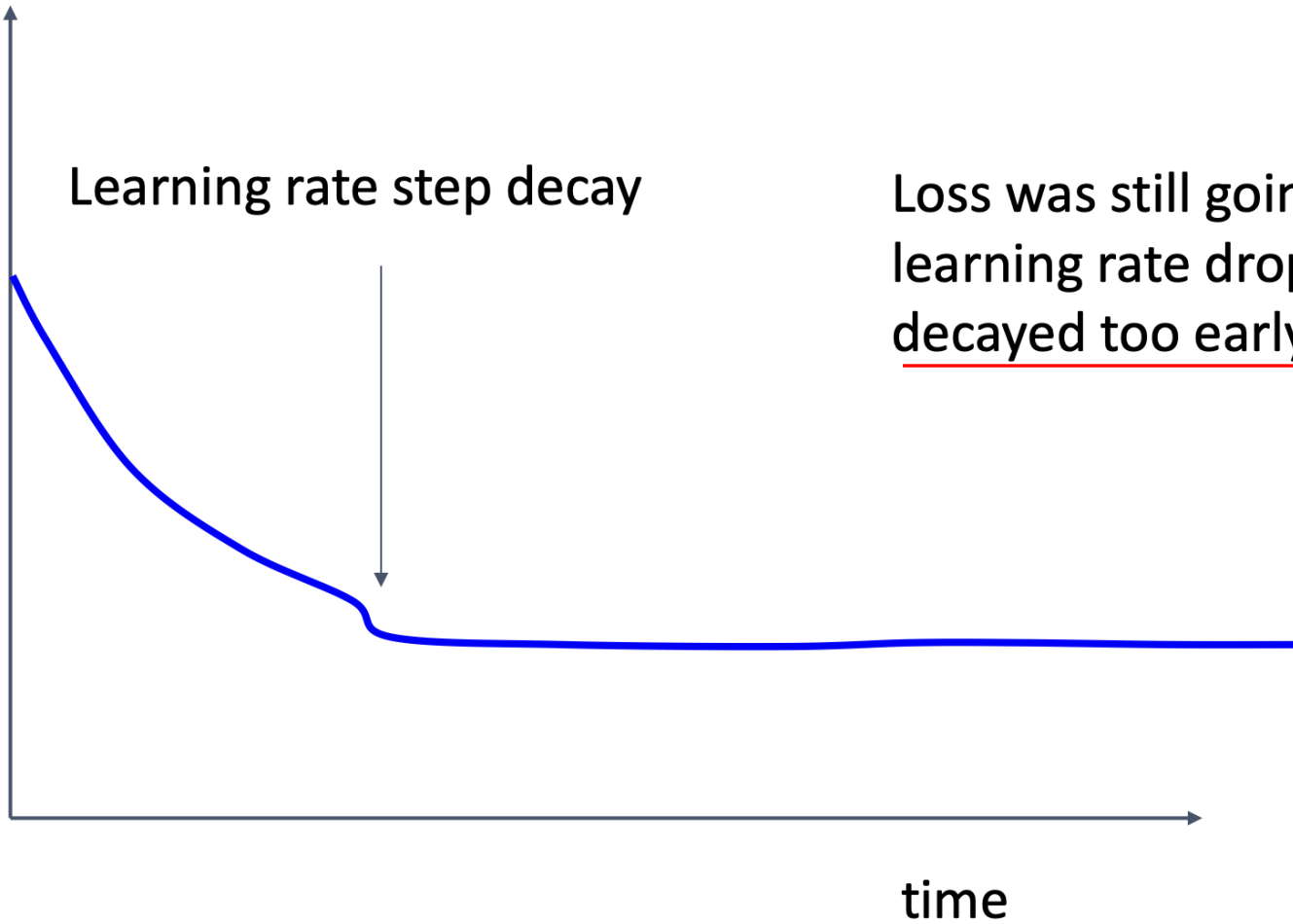


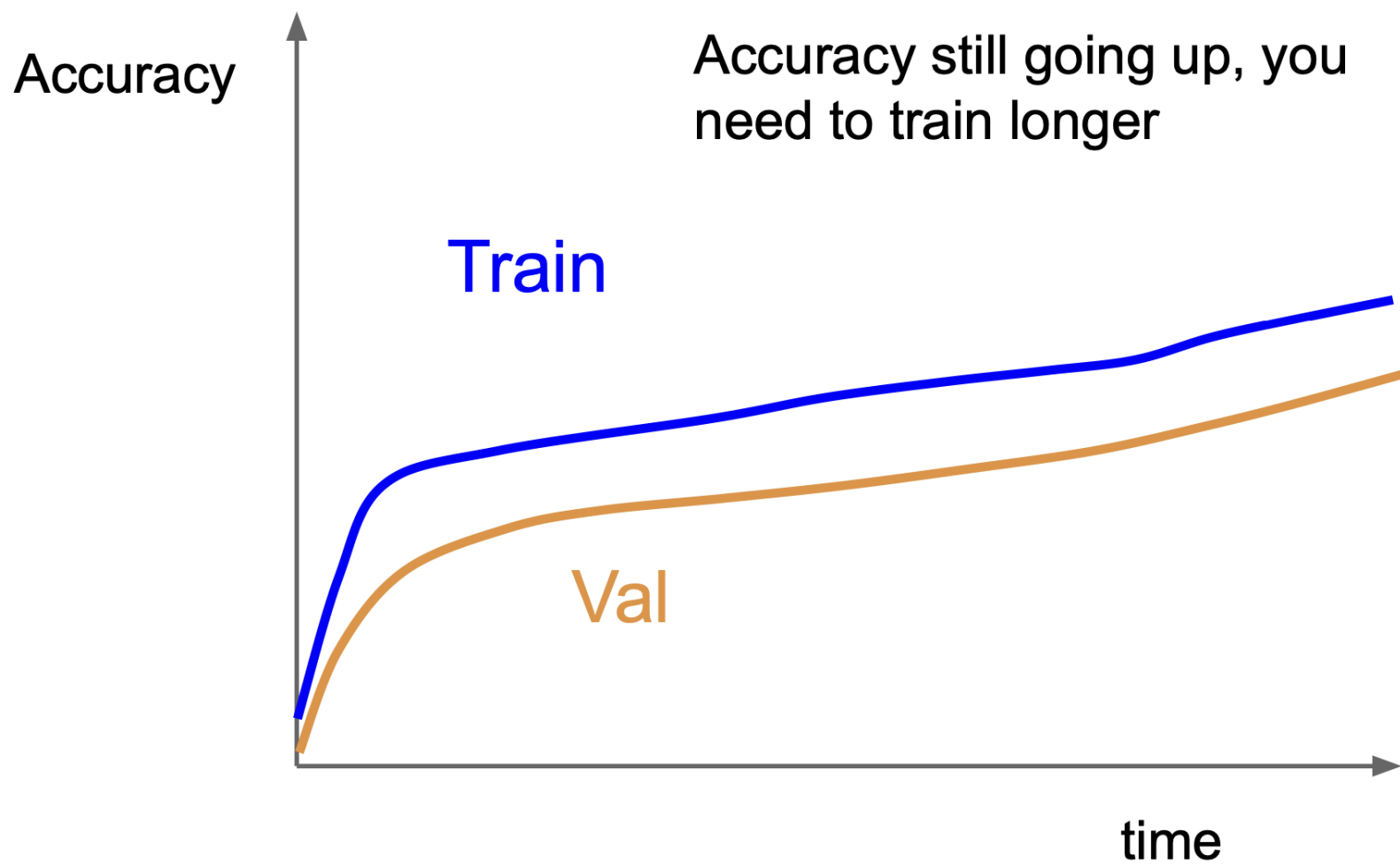


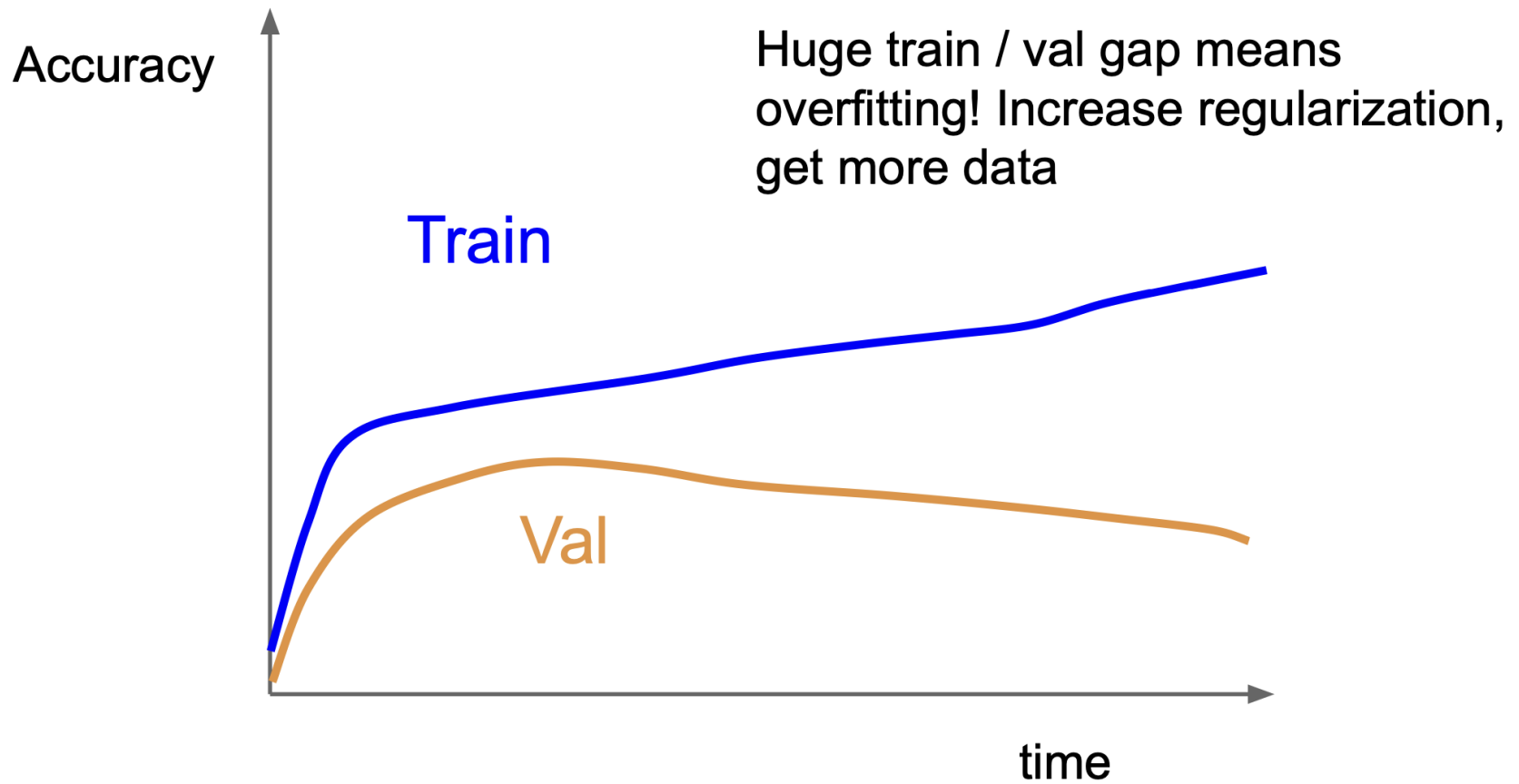
Loss

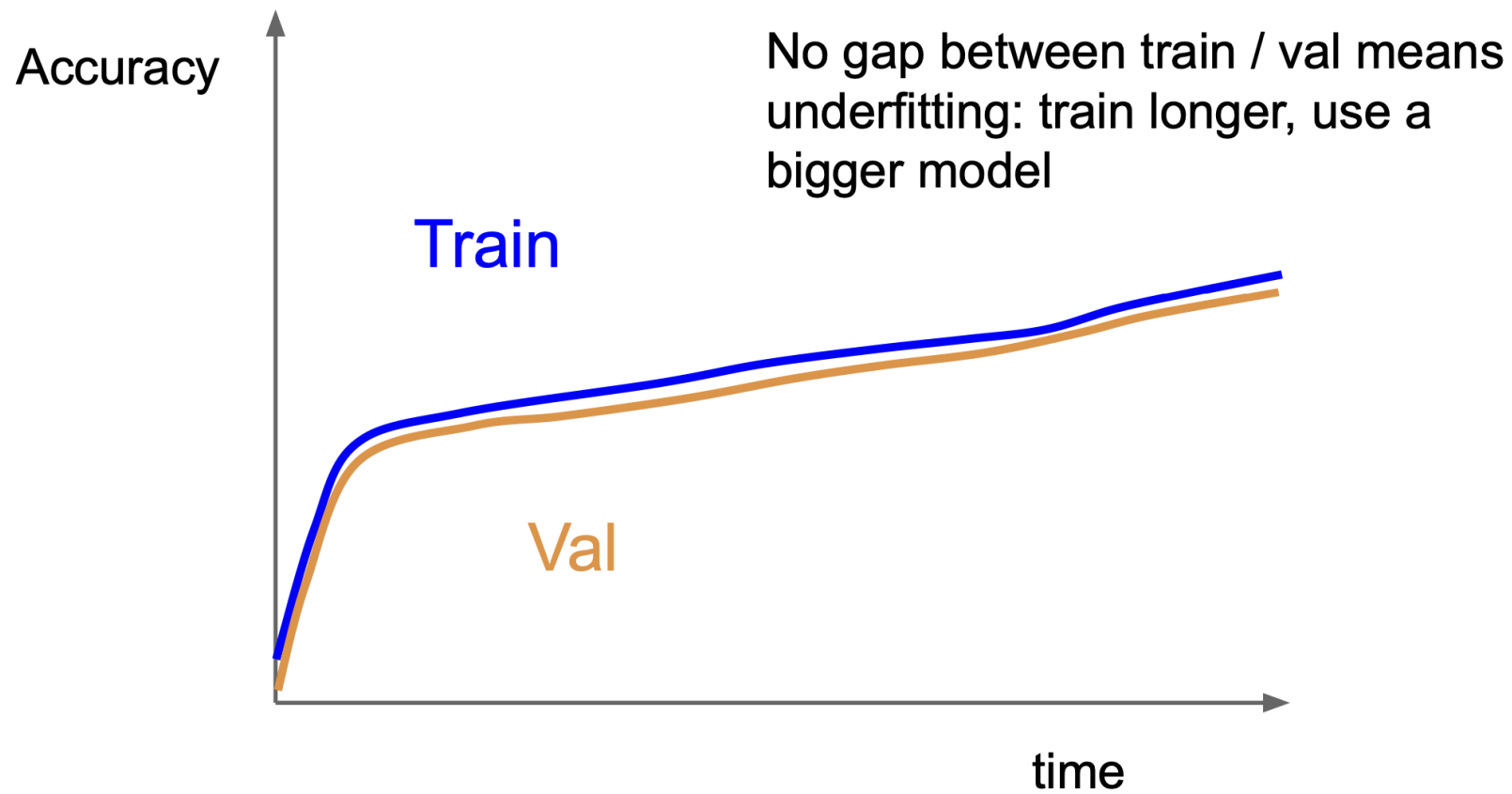
Learning rate step decay

Loss was still going down when
learning rate dropped, you
decayed too early!









Choosing Hyperparameters

- Step 1:** Check initial loss
- Step 2:** Overfit a small sample
- Step 3:** Find LR that makes loss go down
- Step 4:** Coarse grid, train for ~1-5 epochs
- Step 5:** Refine grid, train longer
- Step 6:** Look at loss curves
- Step 7:** GOTO step 5

Hyperparameters to play with:

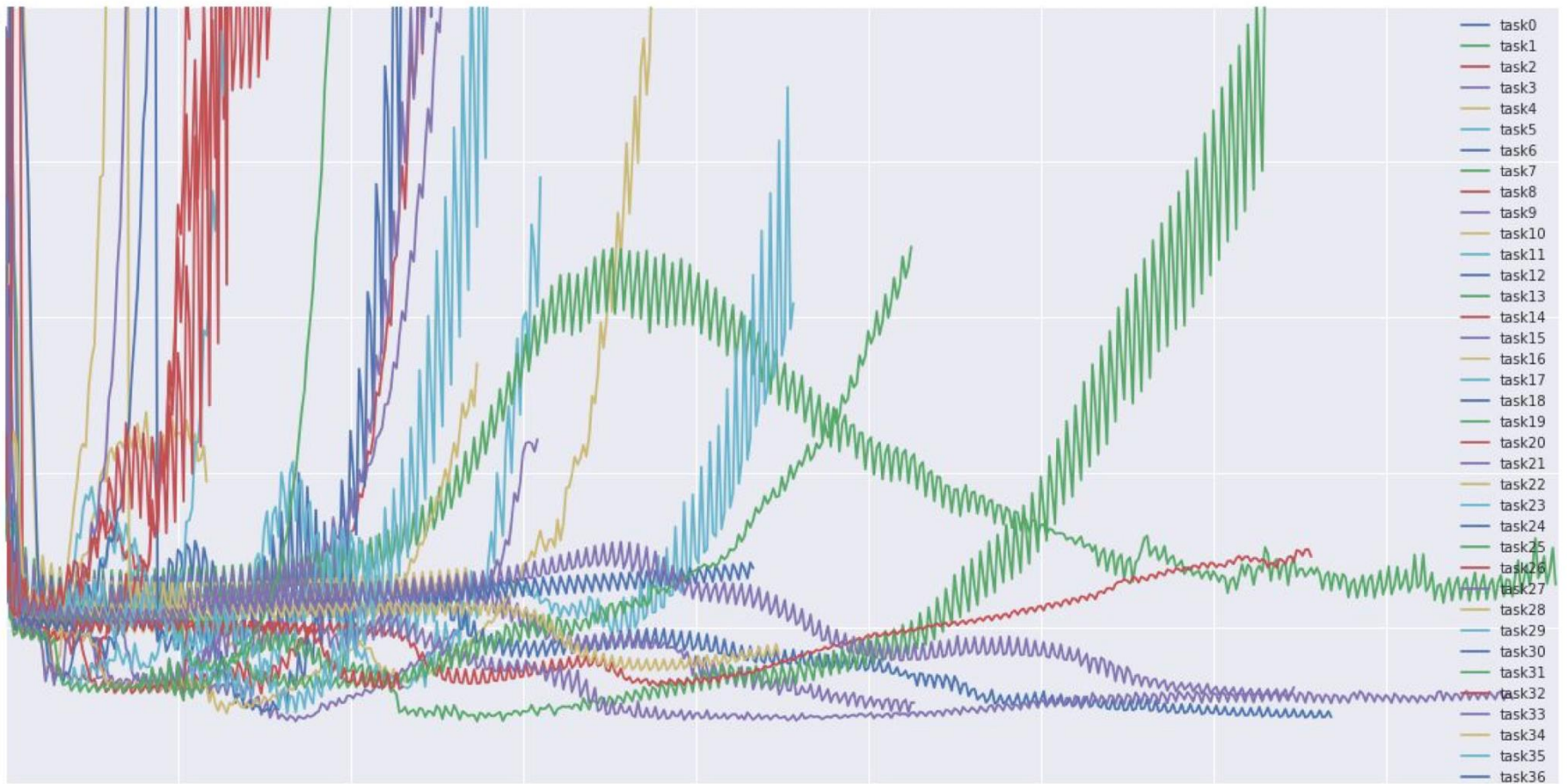
- network architecture
- learning rate, its decay schedule,
- regularization (L2/Dropout strength)

neural networks practitioner
music = loss function



[This image](#) by Paolo Guereta is licensed under [CC-BY 2.0](#)

You can plot all your loss curves for different hyperparameters on a single plot



Track ratio of weight update / weight magnitude

```
# assume parameter vector W and its gradient vector dW
param_scale = np.linalg.norm(W.ravel())
update = -learning_rate*dW # simple SGD update
update_scale = np.linalg.norm(update.ravel())
W += update # the actual update
print update_scale / param_scale # want ~1e-3
```

ratio between the updates and values: $\sim 0.0002 / 0.02 = 0.01$ (about okay)
want this to be somewhere around 0.001 or so

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Loading Pretrained Models with Keras

Using Pretrained Models From Keras

In general, you won't have to implement standard models like GoogLeNet or ResNet manually, since pretrained networks are readily available with a single line of code, in the `keras.applications` package. For example:

```
model = keras.applications.resnet50.ResNet50(weights="imagenet")
```


Loading Pretrained Models with Keras

Available models

Models for image classification with weights trained on ImageNet:

- Xception
- VGG16
- VGG19
- ResNet, ResNetV2
- InceptionV3
- InceptionResNetV2
- MobileNet
- MobileNetV2
- DenseNet
- NASNet

Loading Pretrained Models with Keras

Documentation for individual models

Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
Xception	88 MB	0.790	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.900	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.760	0.930	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.780	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.750	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-
NASNetLarge	343 MB	0.825	0.960	88,949,818	-

The top-1 and top-5 accuracy refers to the model's performance on the ImageNet validation dataset.

Using Pretrained Models with Keras

Now, *you* find some images (even take photos of objects in the listed 1000 classes yourself!) and check out how well the imageNet trained classifier you select does.

```
def loadPreprocAndStackImgs(fileNameList):
    resized=[]
    for fil in fileNameList:
        resized.append(tf.image.resize_with_pad(
            tf.keras.preprocessing.image.img_to_array(
                tf.keras.preprocessing.image.load_img(fil)),
            299,299))

    return tf.stack(resized)

filNames=['laptop.jpg','screwdriver.jpg','pen.jpg','gull.jpg','steakMPbroc.jpg']
imgs=loadPreprocAndStackImgs(filNames)
inputs=tf.keras.applications.xception.preprocess_input(imgs)
model=tf.keras.applications.xception.Xception(weights='imagenet')
Y_proba = model.predict(inputs)
# map the top classes back to labels
top_K = keras.applications.resnet50.decode_predictions(Y_proba, top=10)
for image_index in range(len(images)):
    print("Image #{}".format(image_index))
    for class_id, name, y_proba in top_K[image_index]:
        print(" {} - {:12s} {:.2f}%".format(class_id, name, y_proba * 100))
    print()
```

Using Pretrained Models from Keras!

That's all! This will create a ResNet-50 model and download weights pretrained on the ImageNet dataset. To use it, you first need to ensure that the images have the right size. A ResNet-50 model expects 224×224 images (other models may expect other sizes, such as 299×299), so let's use TensorFlow's `tf.image.resize()` function to resize the images we loaded earlier:

```
images_resized = tf.image.resize(images, [224, 224])
```



The `tf.image.resize()` will not preserve the aspect ratio. If this is a problem, you can try cropping the images to the appropriate aspect ratio before resizing. Both operations can be done in one shot with `tf.image.crop_and_resize()`.

The pretrained models assume that the images are preprocessed in a specific way. In some cases they may expect the inputs to be scaled from 0 to 1, or -1 to 1, and so on. Each model provides a `preprocess_input()` function that you can use to preprocess your images. These functions assume that the pixel values range from 0 to 255, so we must multiply them by 255 (since earlier we scaled them to the 0–1 range):

```
inputs = keras.applications.resnet50.preprocess_input(images_resized * 255)
```

Now we can use the pretrained model to make predictions:

```
y_proba = model.predict(inputs)
```

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

Convolutional Neural Networks – Typical Design

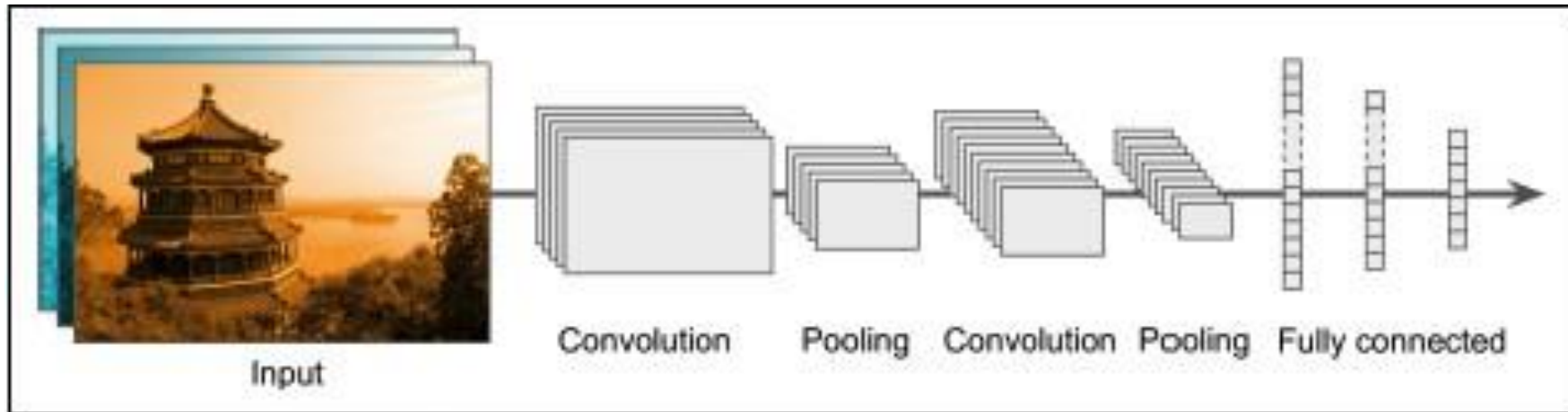


Figure 14-11. Typical CNN architecture

Convolutional Neural Networks – Keras Prototype

Here is how you can implement a simple CNN to tackle the fashion MNIST dataset (introduced in [Chapter 10](#)):

```
from functools import partial

DefaultConv2D = partial(keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding="SAME")

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

Convolutional Neural Networks

– Keras Prototype

- In this code, we start by using the `partial()` function to define a thin wrapper around the `Conv2D` class, called `DefaultConv2D`: it simply avoids having to repeat the same hyperparameter values over and over again.
- The first layer uses a large kernel size, but no stride because the input images are not very large. It also sets `input_shape=[28, 28, 1]`, which means the images are 28×28 pixels, with a single color channel (i.e., grayscale).
- Next, we have a max pooling layer, which divides each spatial dimension by a factor of two (since `pool_size=2`).
- Then we repeat the same structure twice: two convolutional layers followed by a max pooling layer. For larger images, we could repeat this structure several times (the number of repetitions is a hyperparameter you can tune).

Convolutional Neural Networks

– Keras Prototype

- Note that the number of filters grows as we climb up the CNN towards the output layer (it is initially 64, then 128, then 256): it makes sense for it to grow, since the number of low level features is often fairly low (e.g., small circles, horizontal lines, etc.), but there are many different ways to combine them into higher level features. It is a common practice to double the number of filters after each pooling layer: since a pooling layer divides each spatial dimension by a factor of 2, we can afford doubling the number of feature maps in the next layer, without fear of exploding the number of parameters, memory usage, or computational load.
- Next is the fully connected network, composed of 2 hidden dense layers and a dense output layer. Note that we must flatten its inputs, since a dense network expects a 1D array of features for each instance. We also add two dropout layers, with a dropout rate of 50% each, to reduce overfitting.

Convolutional Neural Networks

– Keras Prototype

This CNN reaches over 92% accuracy on the test set. It's not the state of the art, but it is pretty good, and clearly much better than what we achieved with dense networks in Chapter 10.

Overview

1. Data API wrap up:

(a) Subclassing Keras Components

- i. Custom Losses

- ii. Custom Layers

(b) Dataset Example

2. Training Neural Networks

(a) One time setup

Activation functions, data preprocessing, weight initialization, regularization

(b) Training dynamics

Learning rate schedules; hyperparameter optimization

3. Deep CNNs for Image Object Detection

(a) Accessible ImageNet Trained Models via `keras.applications`

(b) Basic CNN Architecture

(c) Some ImageNet Challenge Winner Architectures

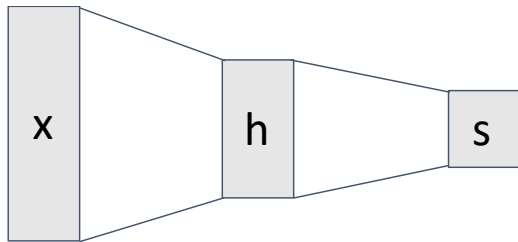
ConvNets – ImageNet History

Over the years, variants of this fundamental architecture have been developed, leading to amazing advances in the field. A good measure of this progress is the error rate in competitions such as the ILSVRC ImageNet challenge. In this competition the top-5 error rate for image classification fell from over 26% to less than 2.3% in just six years. The top-five error rate is the number of test images for which the system's top 5 predictions did not include the correct answer. The images are large (256 pixels high) and there are 1,000 classes, some of which are really subtle (try distinguishing 120 dog breeds). Looking at the evolution of the winning entries is a good way to understand how CNNs work.

We will first look at the classical LeNet-5 architecture (1998), then three of the winners of the ILSVRC challenge: AlexNet (2012), GoogLeNet (2014), and ResNet (2015).

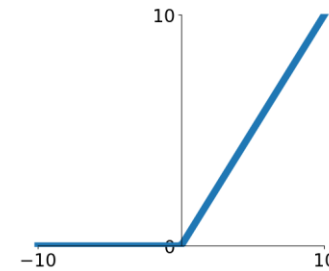
Convolutional Networks

Fully-Connected Layers



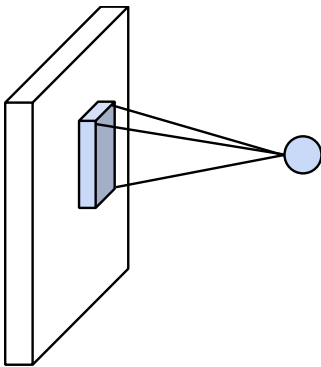
$$y = Wx + b$$

Activation Function

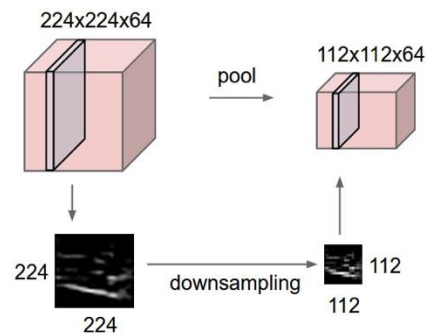


$$y = \max(0, x)$$

Convolution Layers

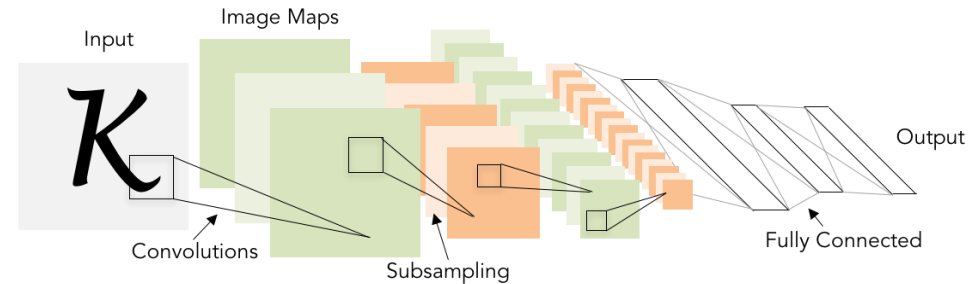


Pooling Layers



Example: Lenet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU	500	
Linear (500 -> 10)	10	500 x 10



As we go through the network:

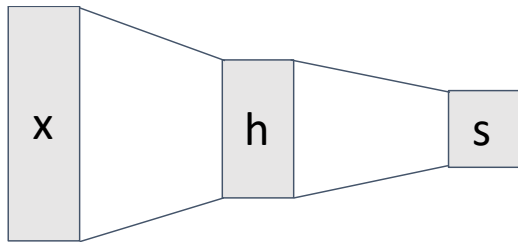
Spatial size **decreases**
(using pooling or strided conv)

Number of channels **increases**
(total “volume” is preserved!)

Problem: Deep Networks very hard to train!

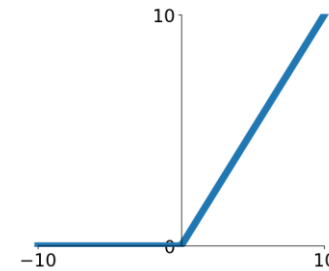
Convolutional Networks

Fully-Connected Layers



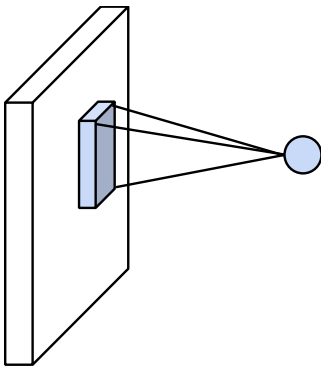
$$y = Wx + b$$

Activation Function

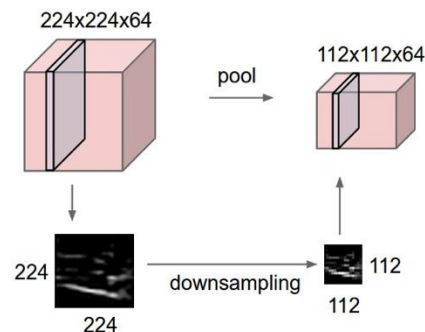


$$y = \max(0, x)$$

Convolution Layers



Pooling Layers



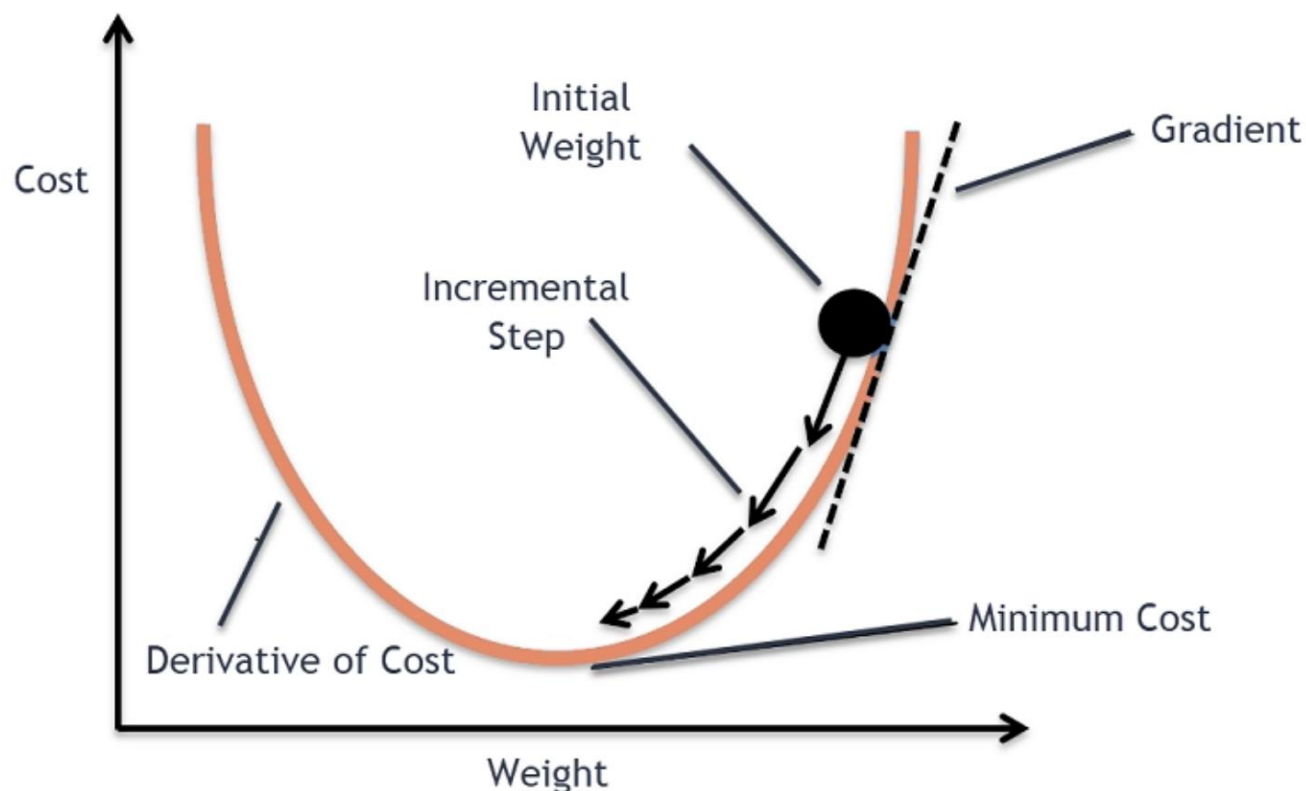
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

131

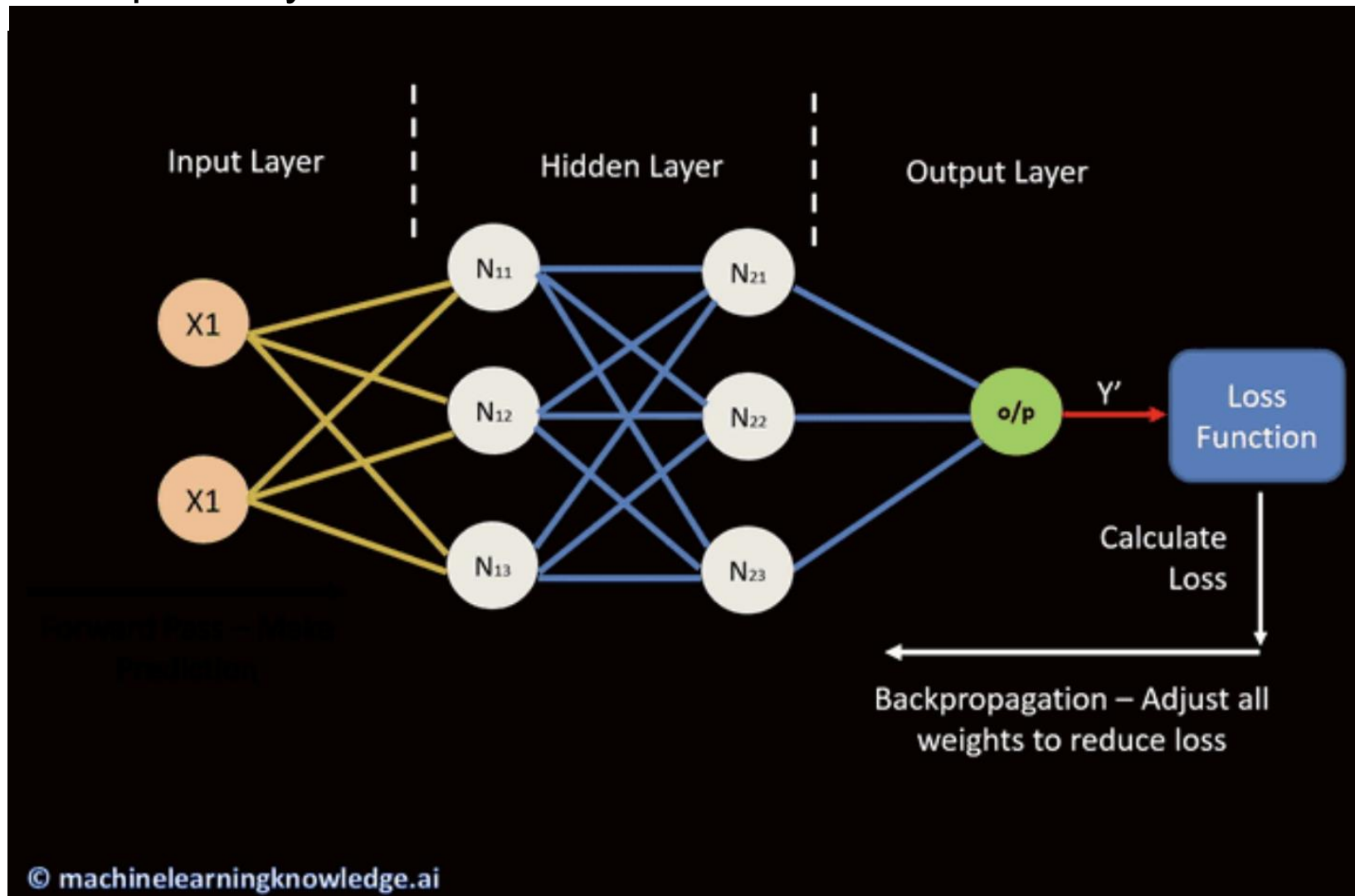
Vanishing and Exploding Gradients

Let us take a quick look at gradient-based learning. The main aim of gradient-based learning is to minimize the loss function. A gradient is simply the slope of a function. Gradient learning aims to travel down the slope of the loss function until we reach local minima.



Backpropagation

Backpropagation is a common method to calculate the derivatives of a neural network. It calculates the gradient of the loss function with respect to the neural network's weights. Backpropagation calculates the gradients and then feeds them back to the first layer of the network. This allows us to calculate the gradients of each layer more precisely.



Vanishing gradients, Exploding gradients

Vanishing gradients

Vanishing gradient arises when we repeatedly multiply the gradient with a number smaller than one. Due to this, the gradient becomes smaller and smaller in each step and becomes infinitesimally small. Due to the small gradient, the step size approaches zero, and as a result, we can never reach the optimum position of our loss function.

Exploding gradients

On the other hand, if we keep on multiplying the gradient with a number larger than one. The gradient keeps increasing and can become so large that it makes our network unstable. Sometimes, the parameters can become so large that they result in NaN values. NaN stands for not a number. It represents undefined values.

Batch Normalization

Idea: “Normalize” the outputs of each layer so they have zero mean and unit variance

Why? Helps reduce “internal covariate shift”, improves optimization

Batch Normalization

Idea: “Normalize” the outputs of each layer so they have zero mean and unit variance

Why? Helps reduce “internal covariate shift”, improves optimization

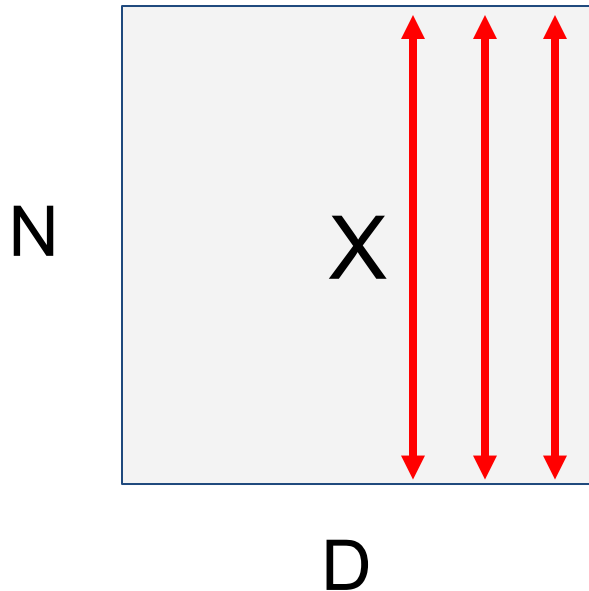
We can normalize a batch of activations like this:

$$\hat{x} = \frac{x - E[x]}{\sqrt{Var[x]}}$$

This is a **differentiable function**, so we can use it as an operator in our networks and backprop through it!

Batch Normalization

Input: $x \in \mathbb{R}^{N \times D}$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel
mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel
std, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x ,
Shape is $N \times D$

Batch Normalization: Test-Time

Input: $x \in \mathbb{R}^{N \times D}$

Learnable scale and shift parameters:

$$\gamma, \beta \in \mathbb{R}^D$$

Learning $\gamma = \sigma, \beta = \mu$
will recover the identity
function (in expectation)

$\mu_j =$ (Running) average of values seen during training
Per-channel mean, shape is D

$\sigma_j^2 =$ (Running) average of values seen during training
Per-channel std, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

Convolutional Networks

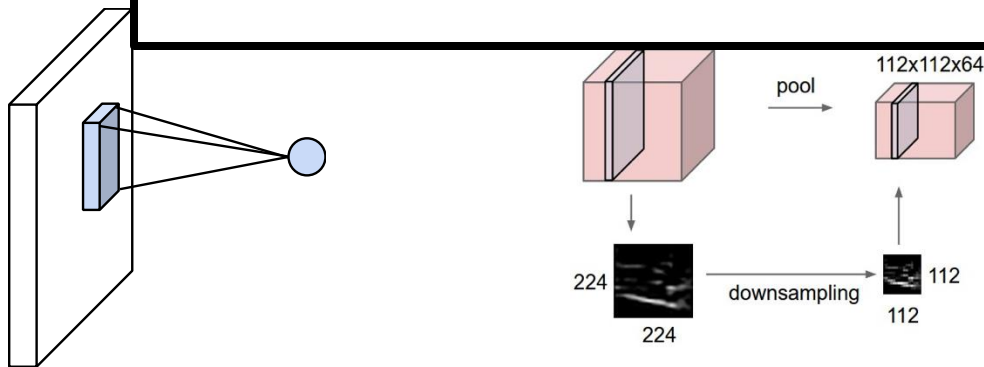
Fully-Connected Layers

Activation Function

How can we combine these components into full architectures?

Convolution

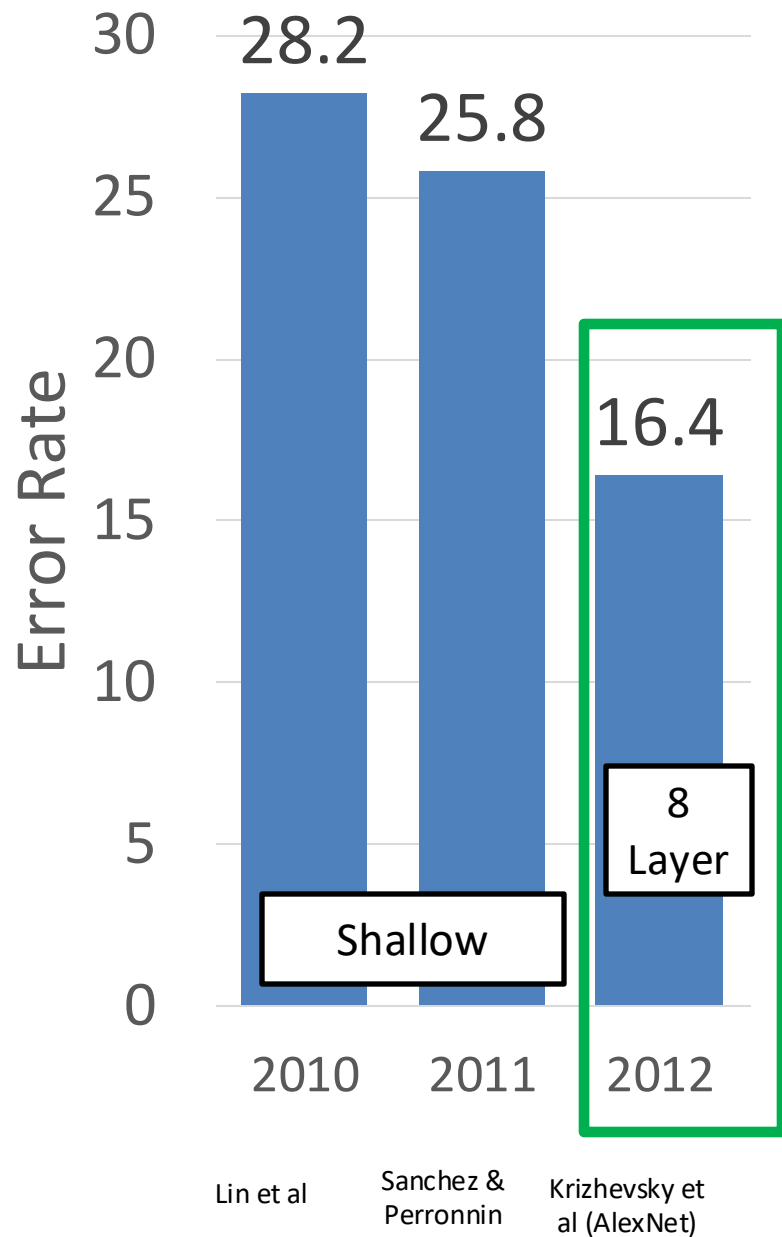
tion



$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

139

ImageNet Classification Challenge



AlexNet

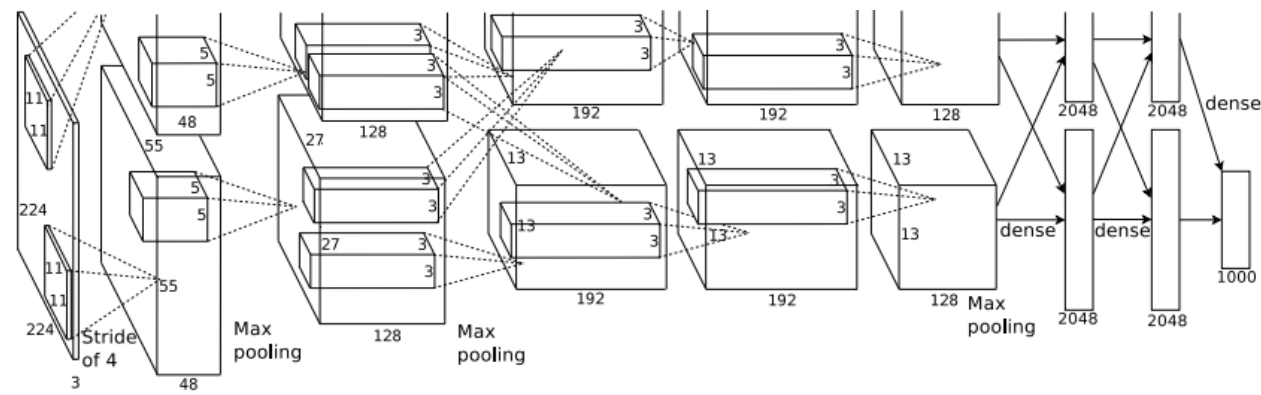
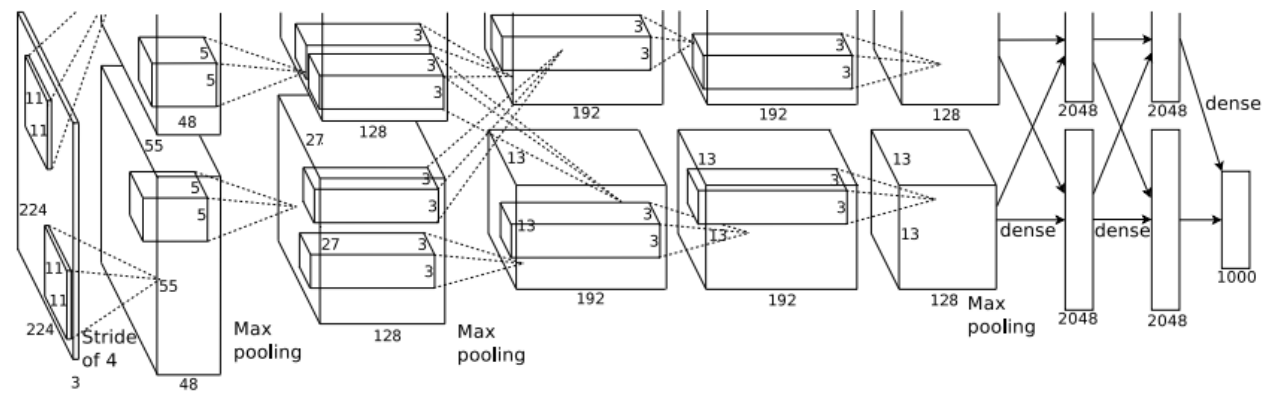


Table 14-2. AlexNet architecture

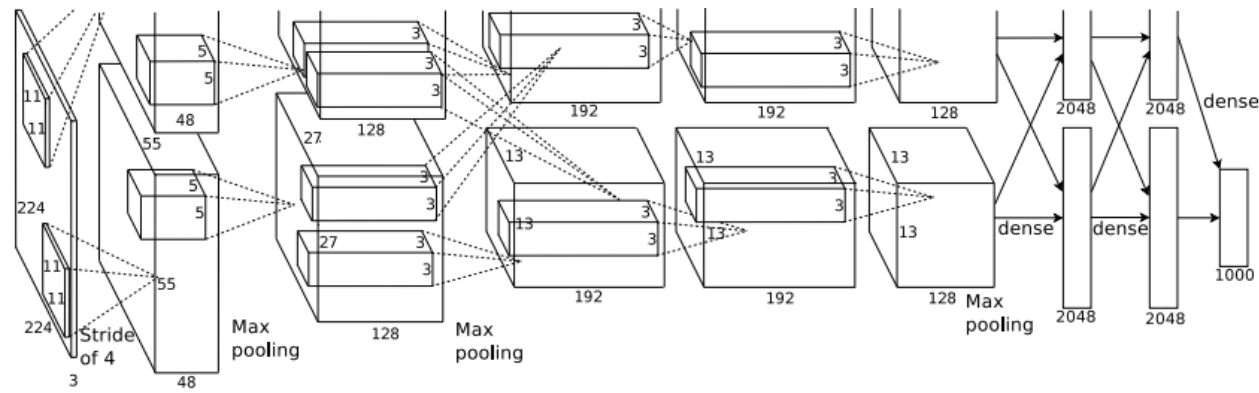
Layer	Type	Maps	Size	Kemel size	Stride	Padding	Activation
Out	Fully Connected	—	1,000	—	—	—	Softmax
F9	Fully Connected	—	4,096	—	—	—	ReLU
F8	Fully Connected	—	4,096	—	—	—	ReLU
C7	Convolution	256	13 × 13	3 × 3	1	SAME	ReLU
C6	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
C5	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
S4	Max Pooling	256	13 × 13	3 × 3	2	VALID	—
C3	Convolution	256	27 × 27	5 × 5	1	SAME	ReLU
S2	Max Pooling	96	27 × 27	3 × 3	2	VALID	—
C1	Convolution	96	55 × 55	11 × 11	4	VALID	ReLU
In	Input	3 (RGB)	227 × 227	—	—	—	—

AlexNet



227 x 227 inputs
5 Convolutional layers
Max pooling
3 fully-connected layers
ReLU nonlinearities

AlexNet



Used “Local response normalization”;
Not used anymore

227 x 227 inputs

5 Convolutional layers

Max pooling

3 fully-connected layers

ReLU nonlinearities

Trained on two GTX 580 GPUs – only
3GB of memory each! Model split
over two GPUs

reduce overfitting:

- 50% dropout used in training F8 and F9
- data augmentation involving shifting, horizontal flipping, lighting changes

AlexNet

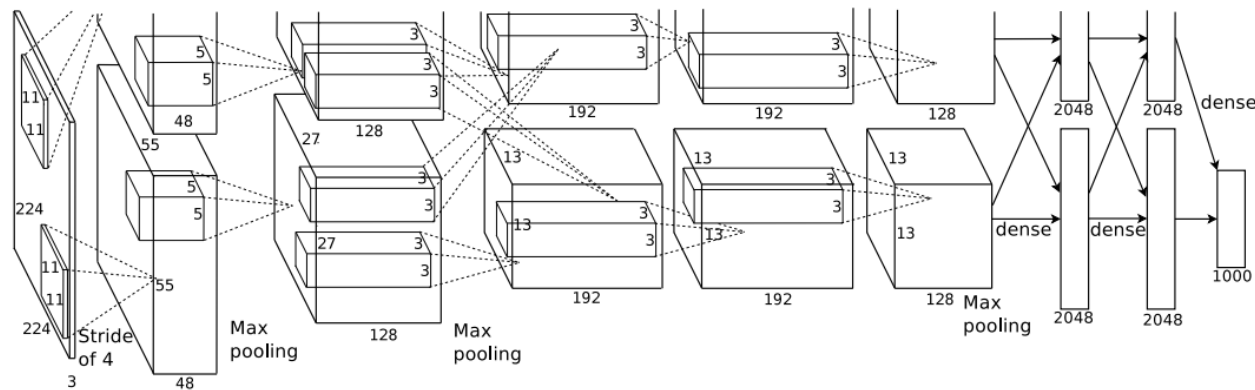
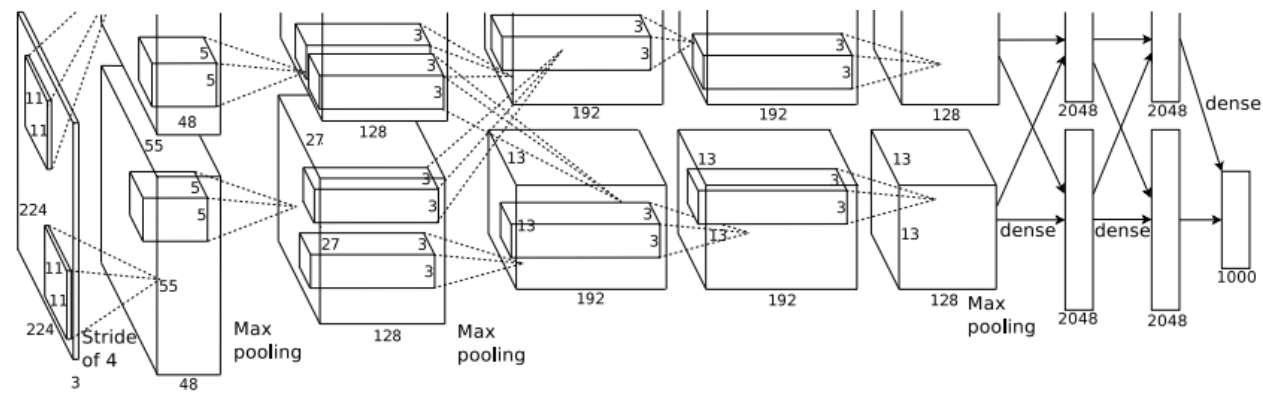
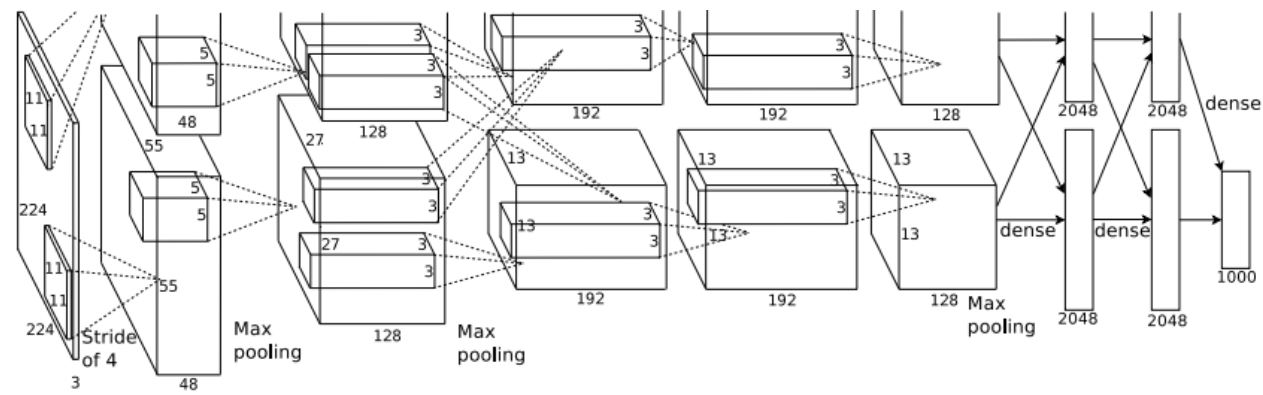
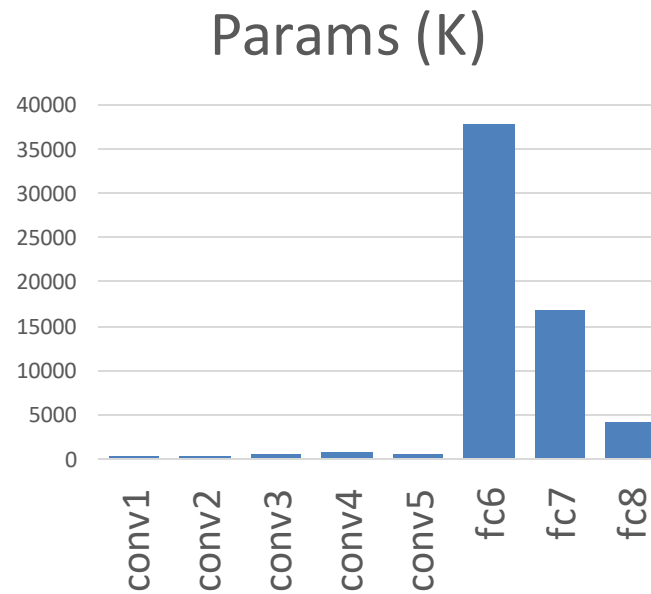


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

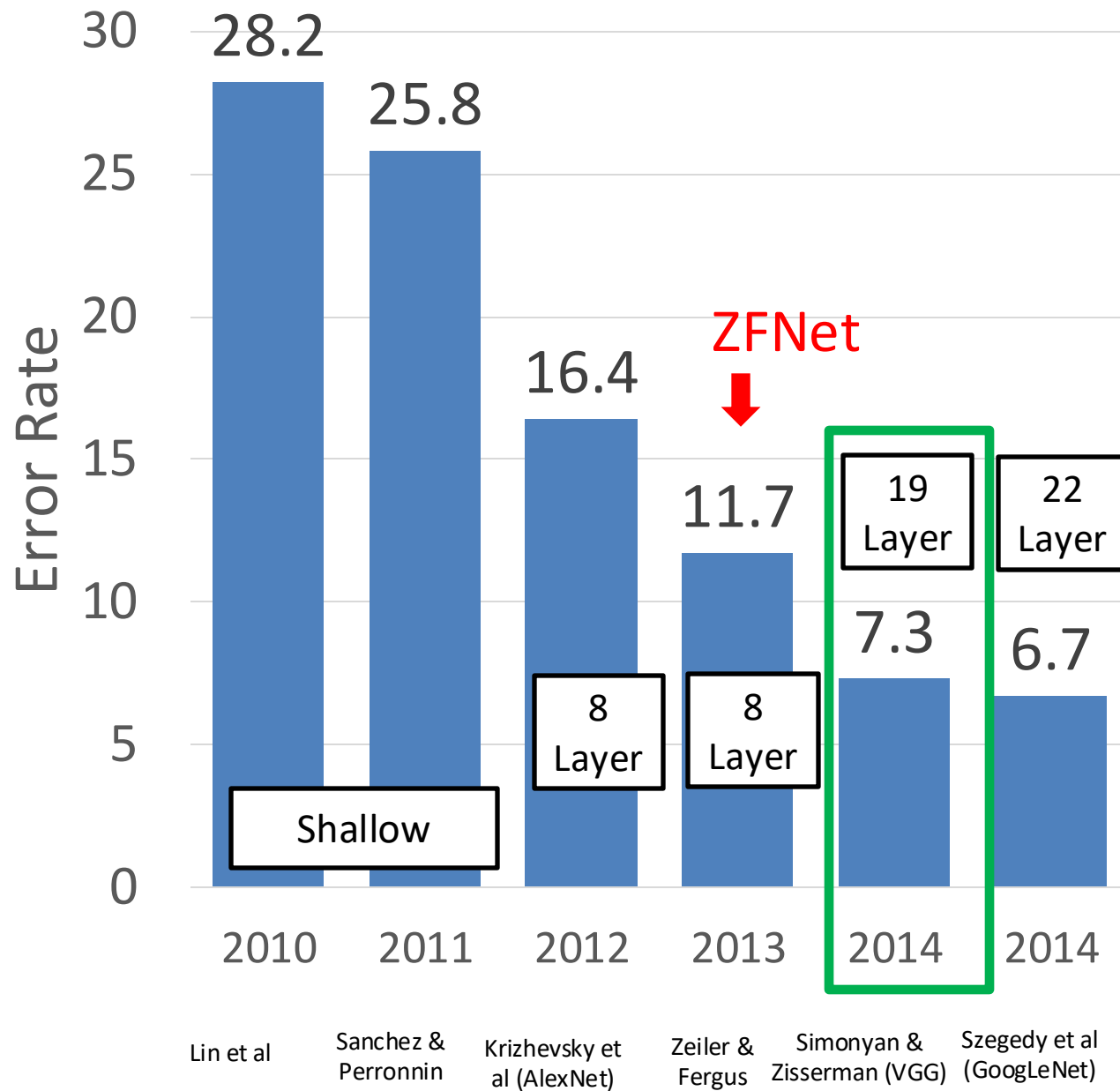
AlexNet



Nearly all **parameters** are in the fully-connected layers



ImageNet Classification Challenge



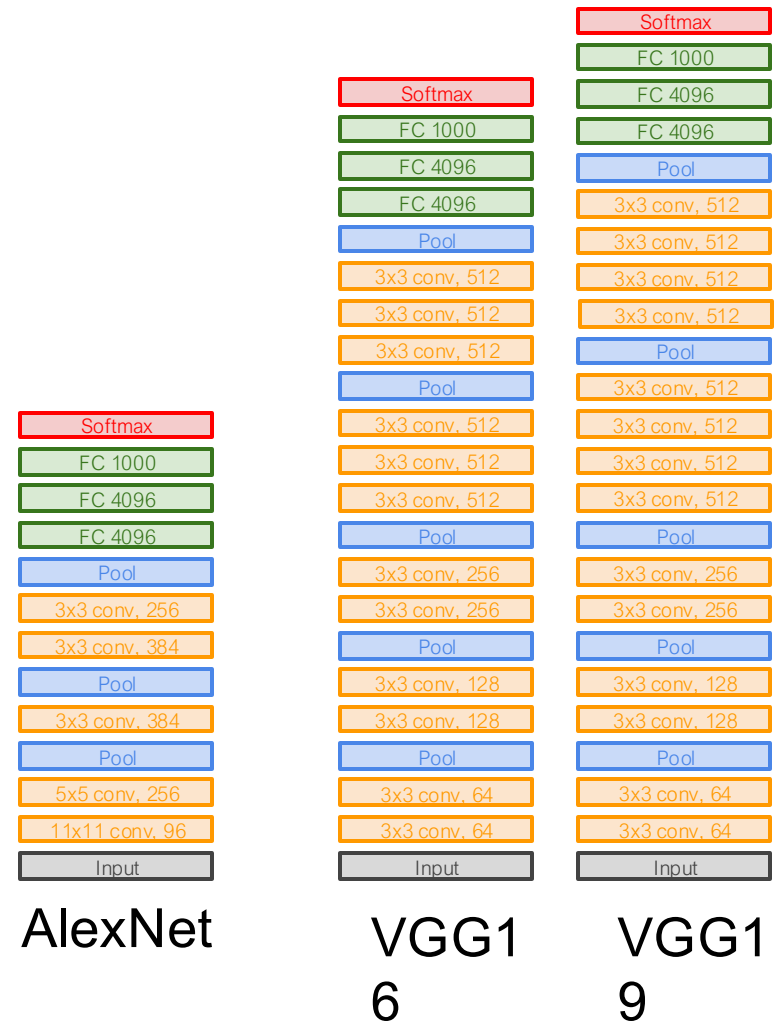
VGG: Deeper Networks, Regular Design

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels



VGG: Deeper Networks, Regular Design

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels

Network has 5 convolutional **stages**:

Stage 1: conv-conv-pool

Stage 2: conv-conv-pool

Stage 3: conv-conv-pool

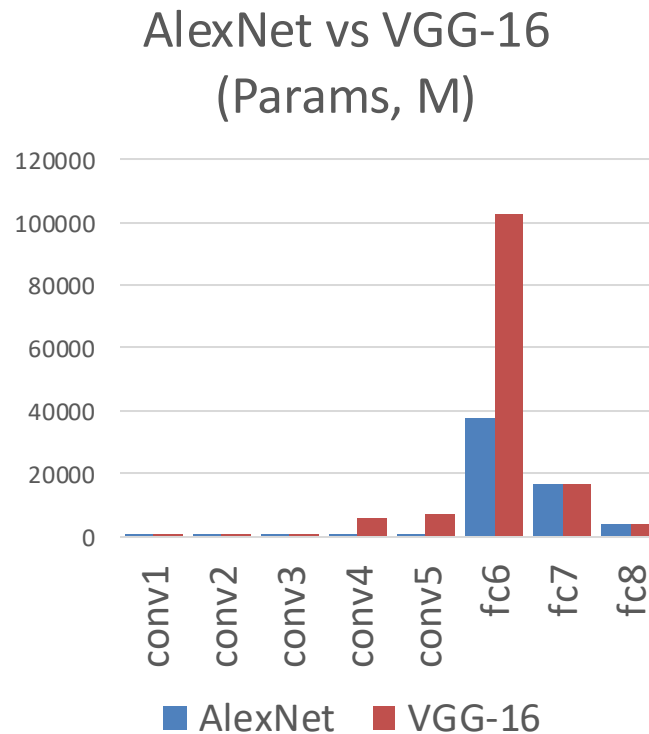
Stage 4: conv-conv-conv-[conv]-pool

Stage 5: conv-conv-conv-[conv]-pool

(VGG-19 has 4 conv in stages 4 and 5)



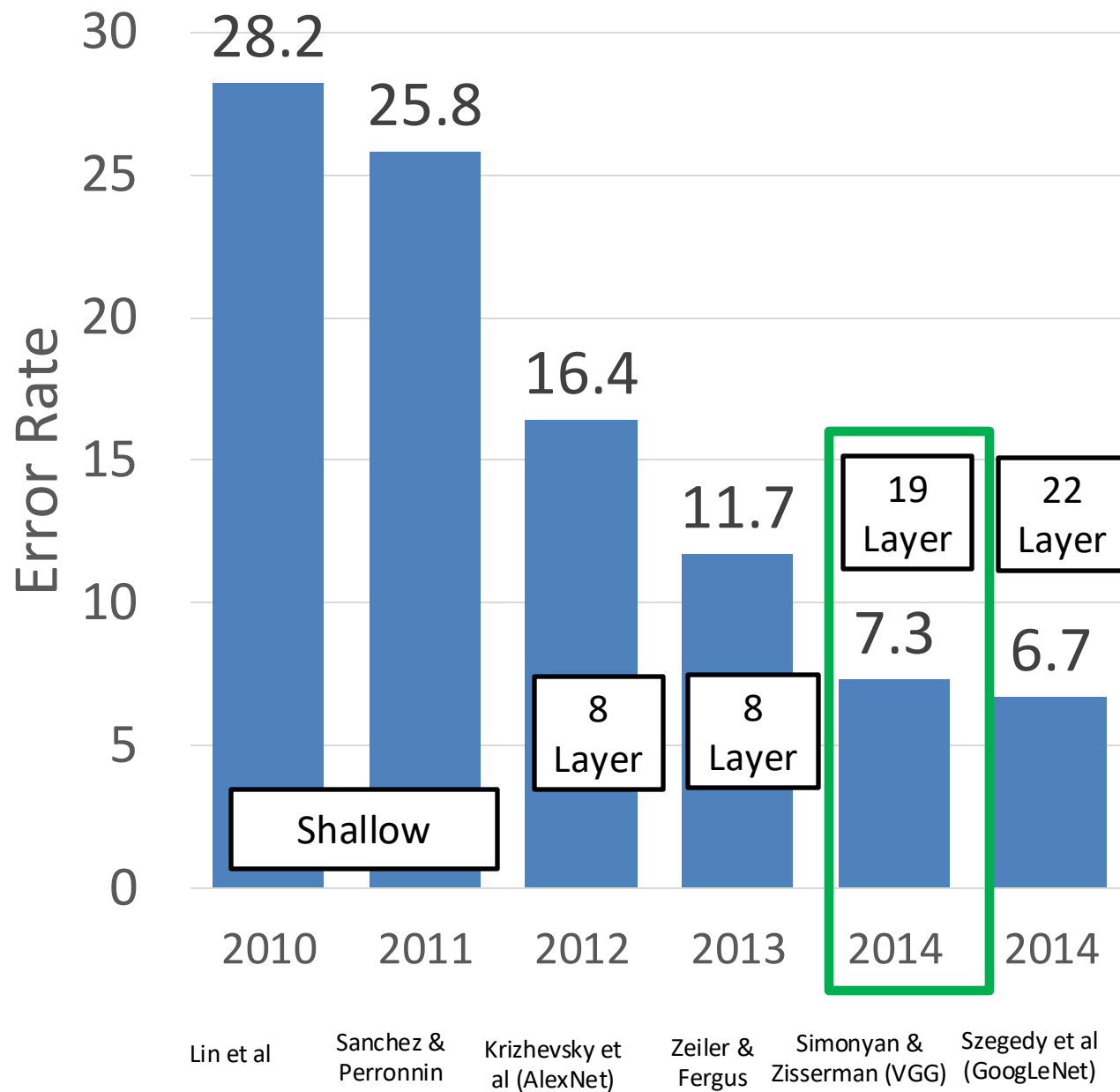
AlexNet vs VGG-16: Much Bigger!



AlexNet total: 61M

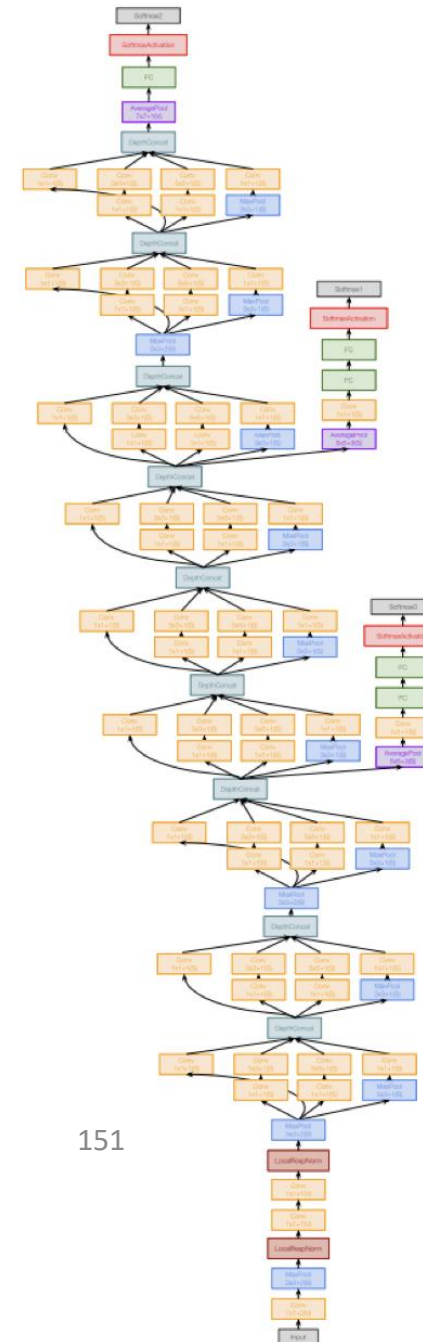
VGG-16 total: 138M (2.3x)

ImageNet Classification Challenge

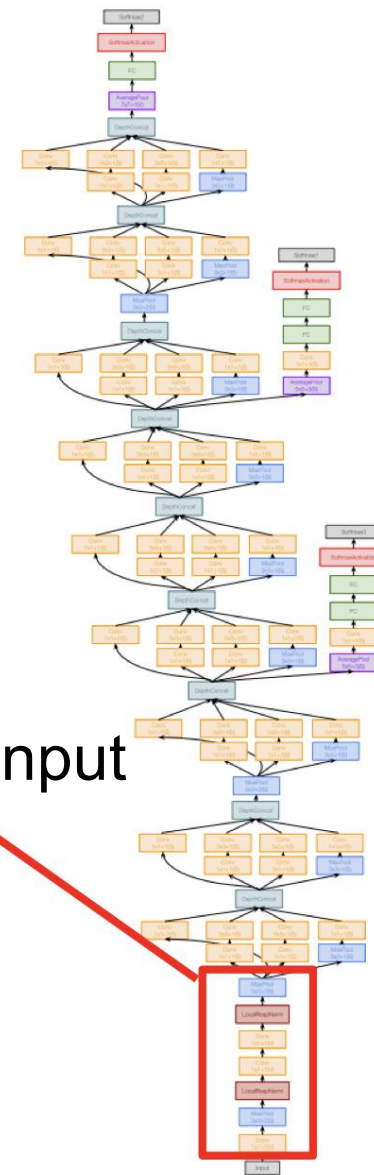


GoogLeNet: Focus on Efficiency

Many innovations for efficiency: reduce parameter count, memory usage, and computation



GoogLeNet: Aggressive Stem



Stem network at the start aggressively downsamples input

GoogLeNet: Inception Module

Inception module
Local unit with
parallel branches

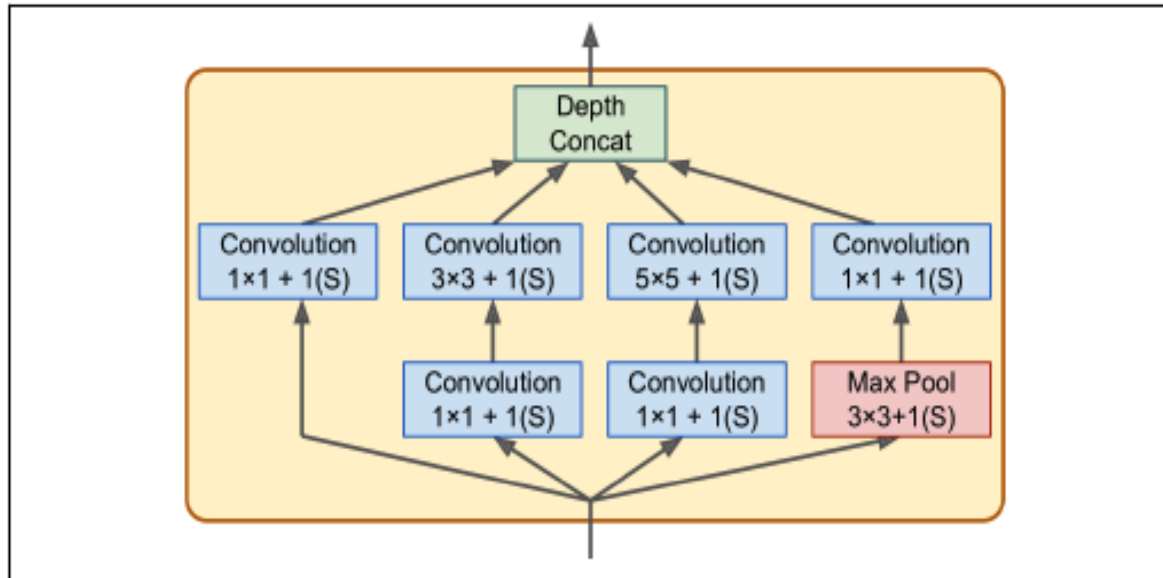
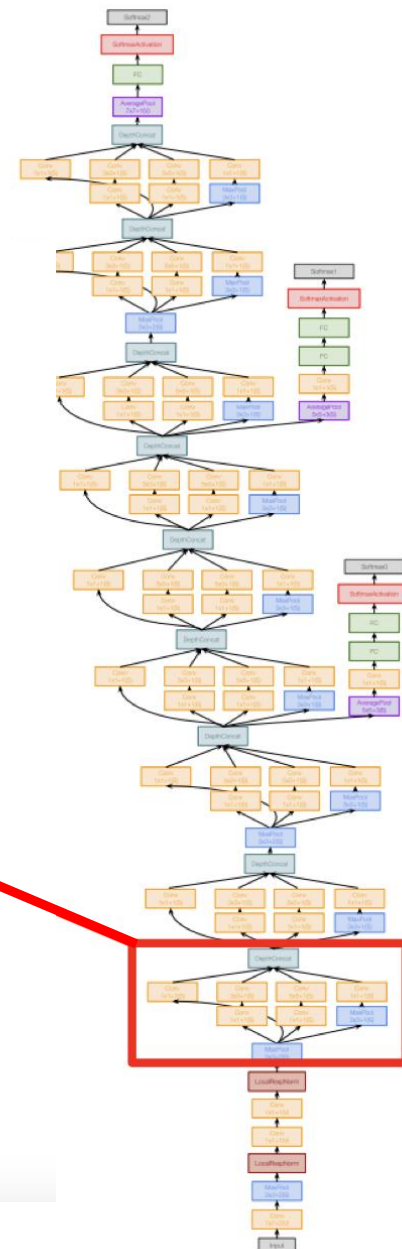


Figure 14-13. Inception module

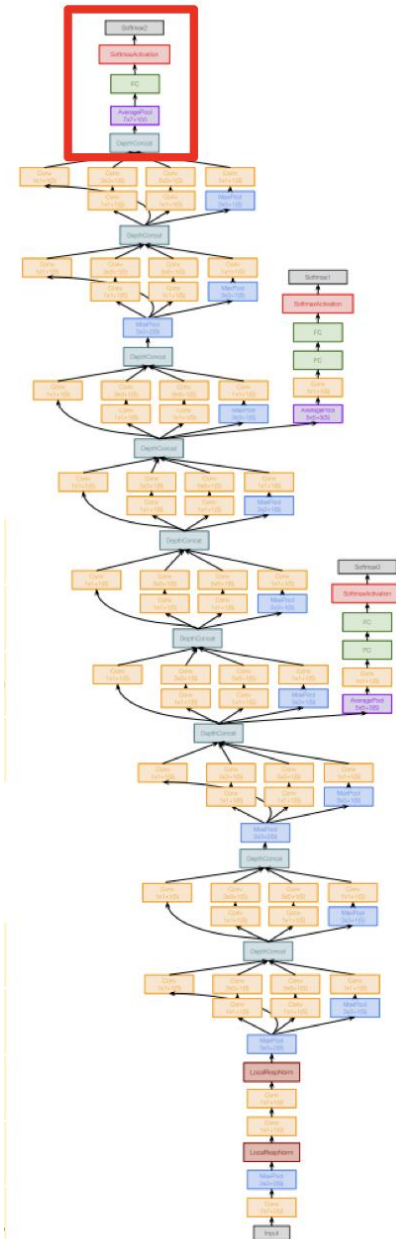
Local structure repeated
many times throughout the
network

Uses 1x1 “Bottleneck”
layers to reduce channel
dimension before
expensive conv (we will
revisit this with ResNet!)



GoogLeNet: Global Average Pooling

No large FC layers at the end! Instead uses “global average pooling” to collapse spatial dimensions, and one linear layer to produce class scores.

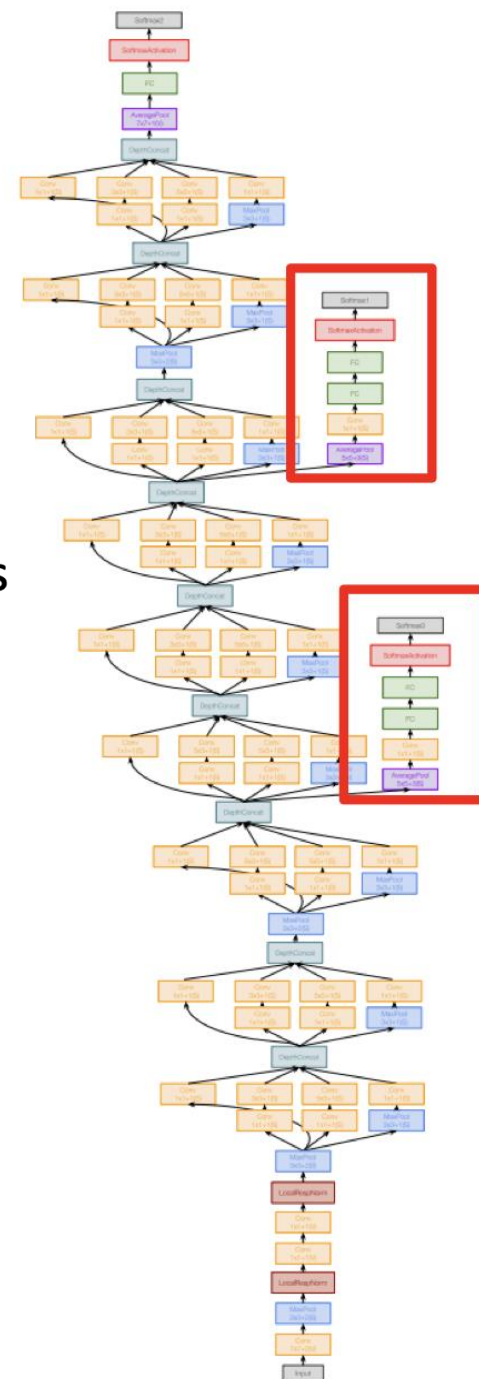


GoogLeNet: Auxiliary Classifiers

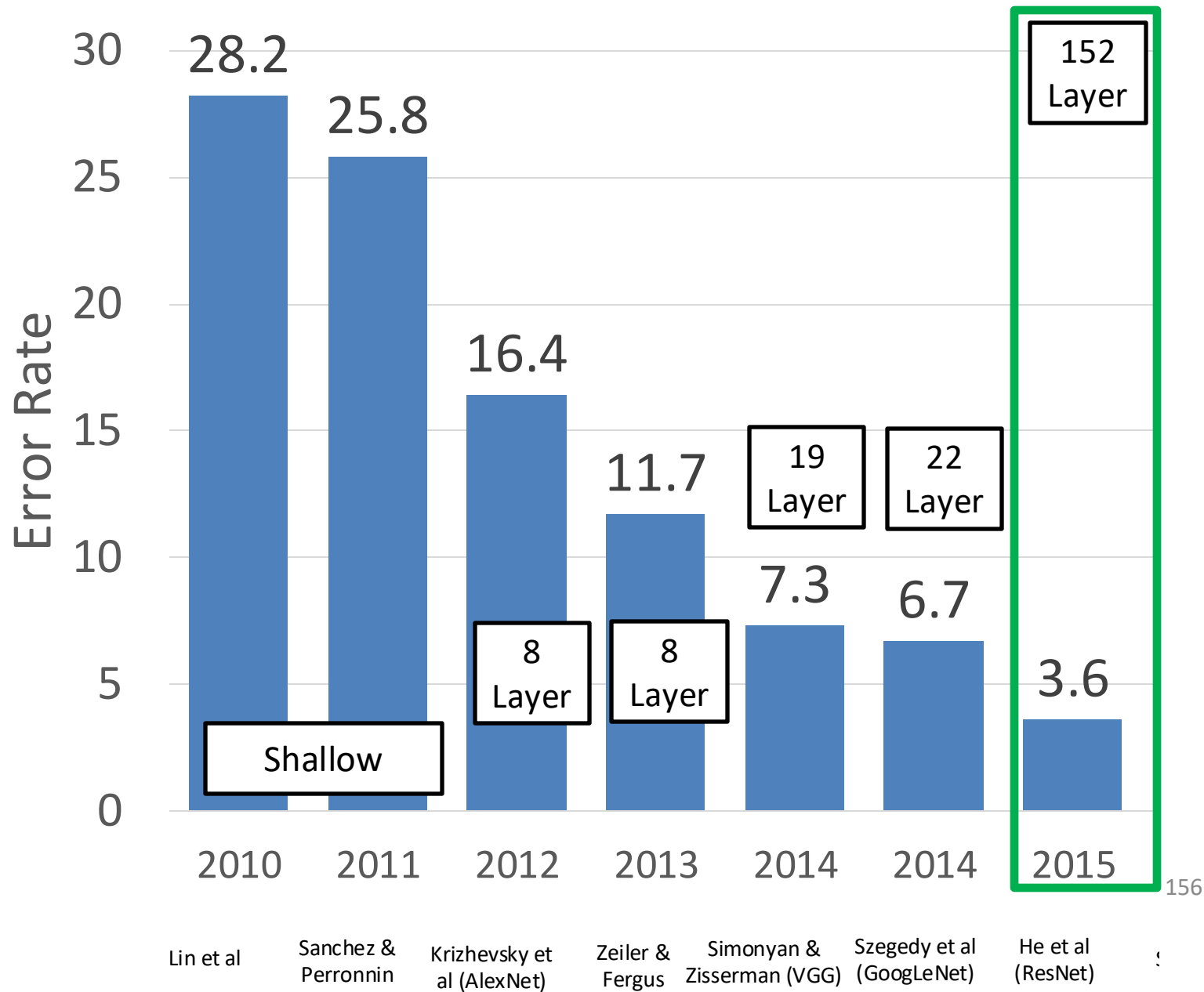
Training using loss at the end of the network didn't work well:
Network is too deep, gradients don't propagate cleanly

As a hack, attach “auxiliary classifiers” at several intermediate points in the network that also try to classify the image and receive loss

GoogLeNet was before batch normalization! With BatchNorm no longer need to use this trick



ImageNet Classification Challenge



Residual Networks

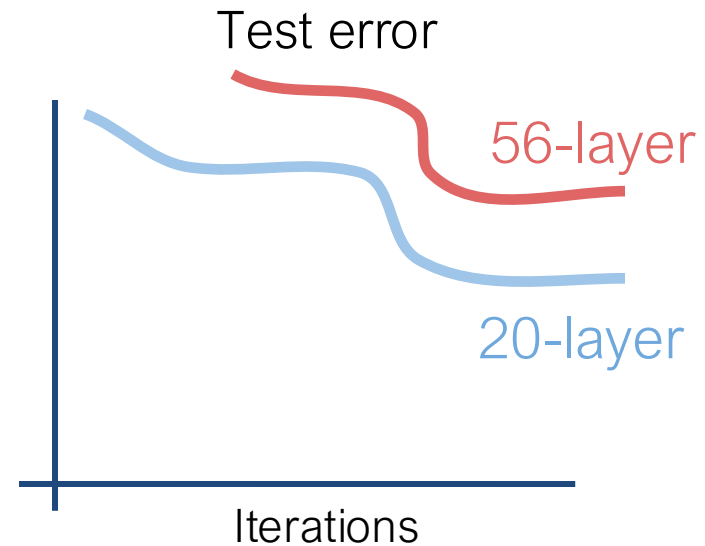
Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?

Residual Networks

Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?

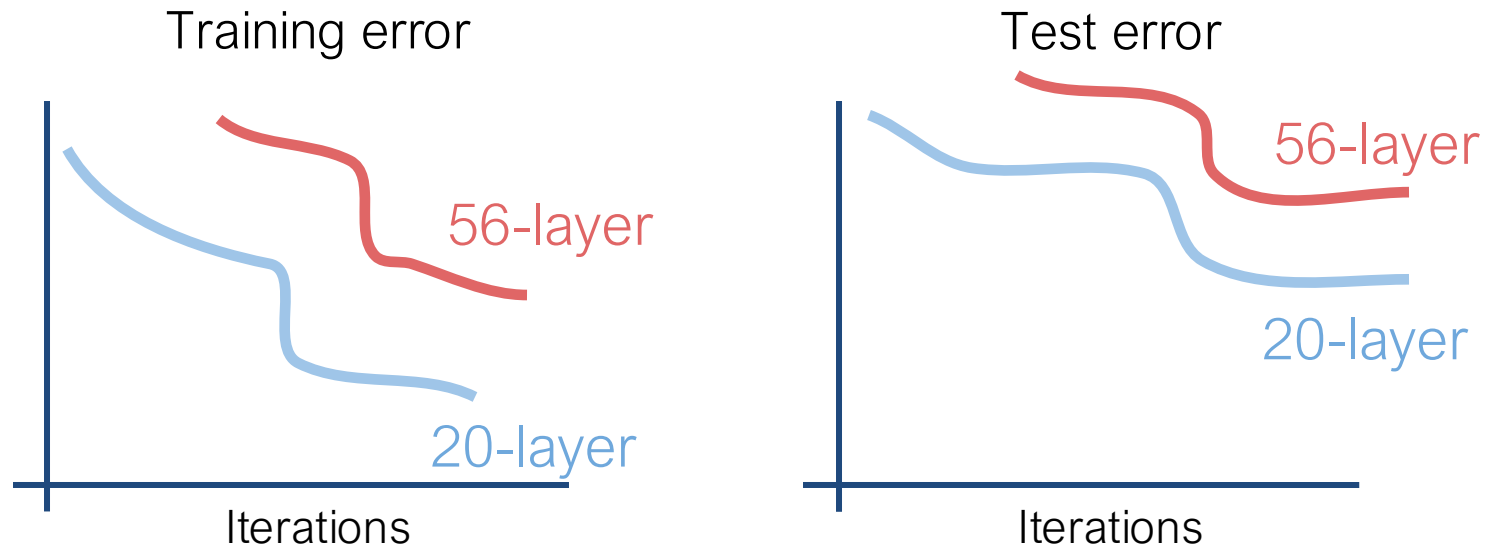
Deeper model does worse than shallow model!

Initial guess: Deep model is **overfitting** since it is much bigger than the other model



Residual Networks

Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?



In fact the deep model seems to be **underfitting** since it also performs worse than the shallow model on the training set! It is actually **underfitting**¹⁵⁹

Residual Networks

A deeper model can emulate a shallower model: copy layers from shallower model, set extra layers to identity

Thus deeper models should do at least as good as shallow models

Hypothesis: This is an optimization problem. Deeper models are harder to optimize, and in particular don't learn identity functions to emulate shallow models

Residual Networks

A deeper model can emulate a shallower model: copy layers from shallower model, set extra layers to identity

Thus deeper models should do at least as good as shallow models

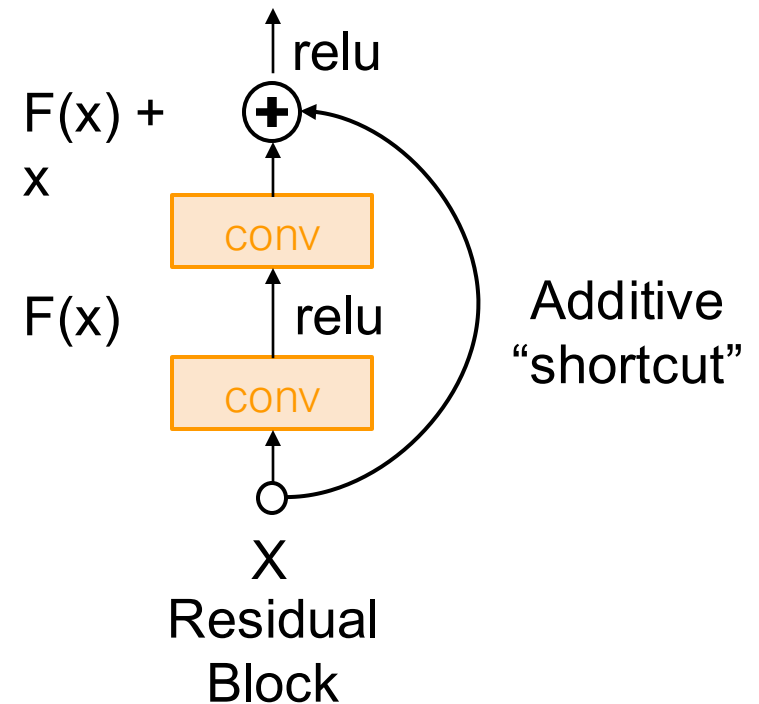
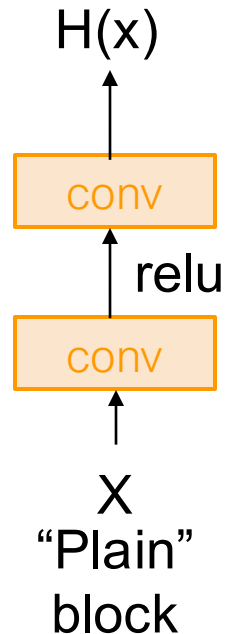
Hypothesis: This is an optimization problem. Deeper models are harder to optimize, and in particular don't learn identity functions to emulate shallow models

Solution: Change the network so learning identity functions with extra layers is easy!

Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!

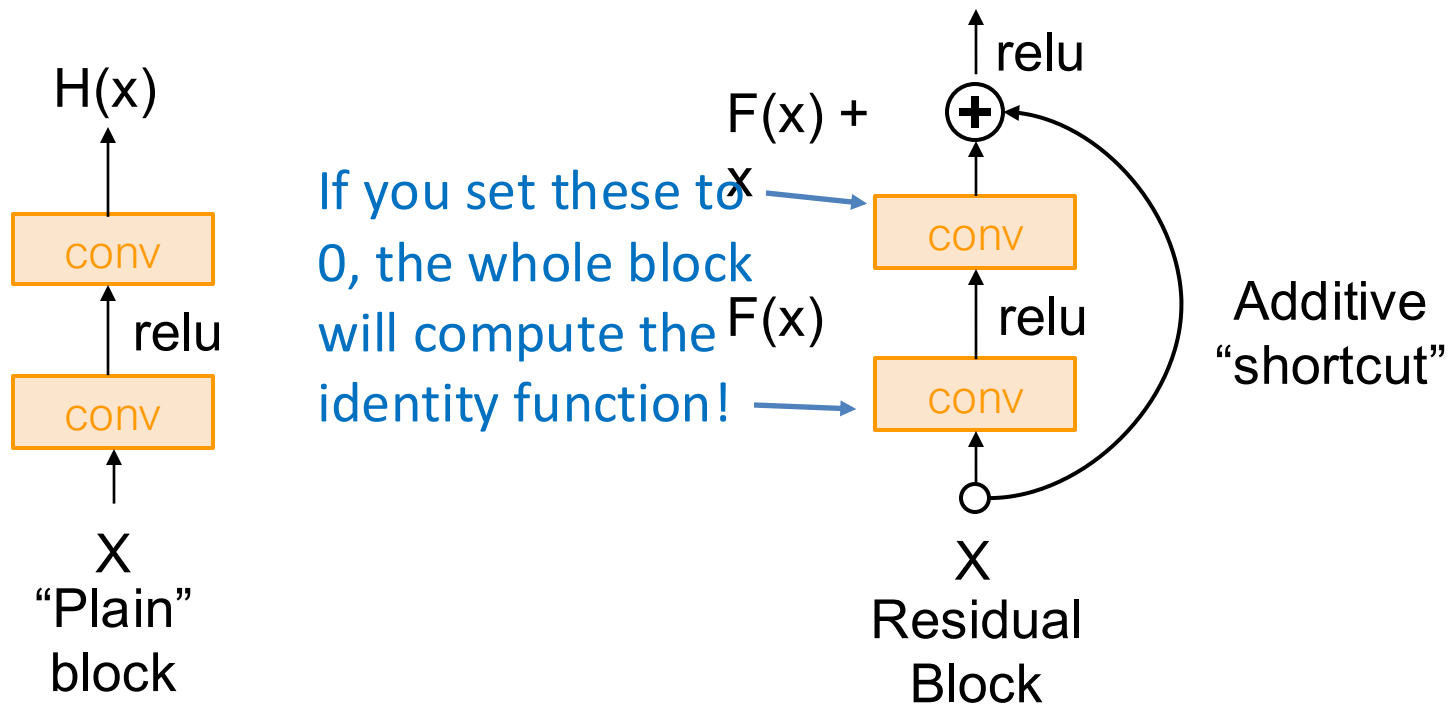
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016



When you initialize a regular neural network, its weights are close to zero, so the network just outputs values close to zero. If you add a skip connection, the resulting network just outputs a copy of its inputs; in other words, it initially models the identity function. If the target function is fairly close to the identity function (which is often the case), this will speed up training considerably.

Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!

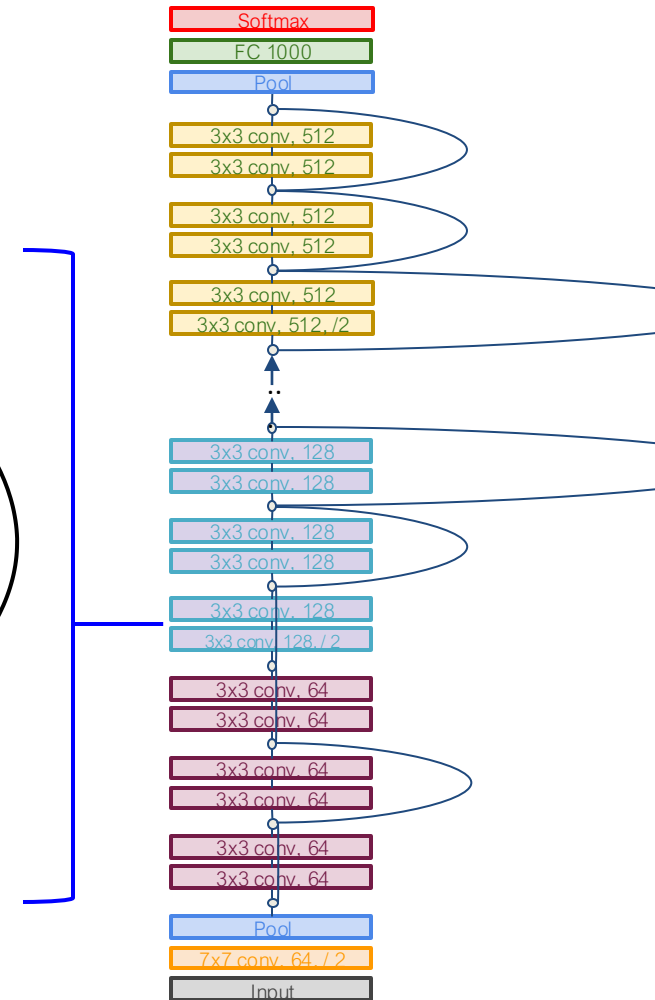
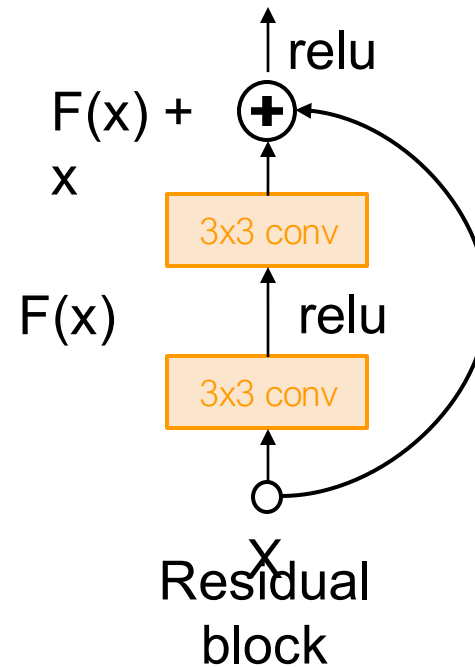


Residual Networks

A residual network is a stack of many residual blocks

Regular design, like VGG:
each residual block has two
3x3 conv

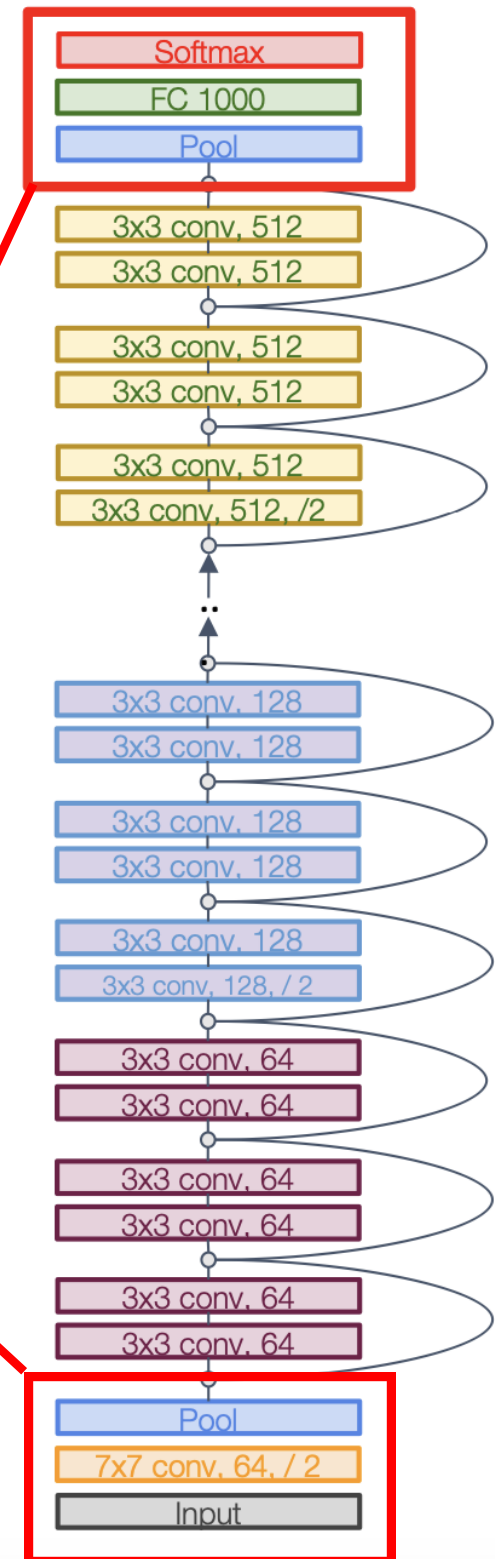
Network is divided into
stages: the first block of each
stage halves the resolution
(with stride-2 conv) and
doubles the number of
channels



Residual Networks

Like GoogLeNet, no big fully-connected-layers:
instead use **global average pooling** and a single linear layer
at the end

Uses the same aggressive **stem** as GoogleNet to downsample the input 4x before applying residual blocks



Residual Networks

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear

ImageNet top-5 error: 10.92

ResNet-34:

Stem: 1 conv layer

Stage 1: 3 res. block = 6 conv

Stage 2: 4 res. block = 8 conv

Stage 3: 6 res. block = 12 conv

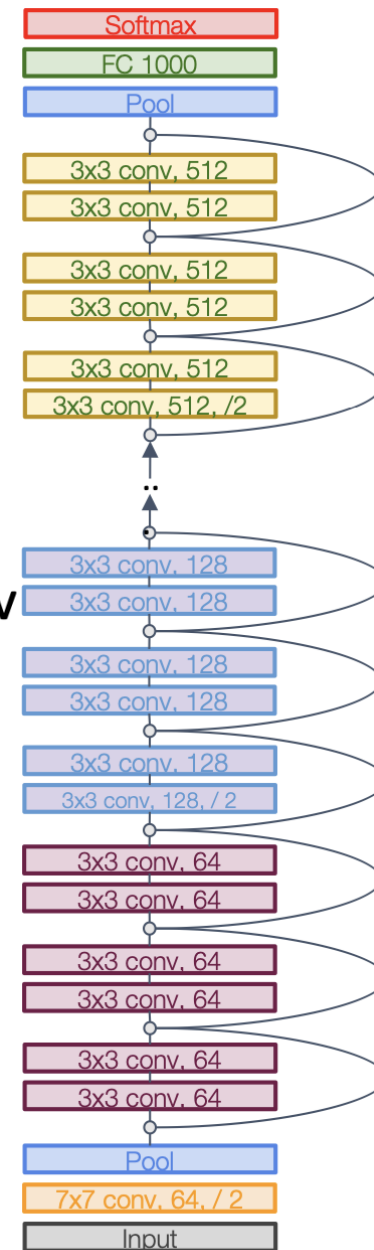
Stage 4: 3 res. block = 6 conv

Linear

ImageNet top-5 error: 8.58

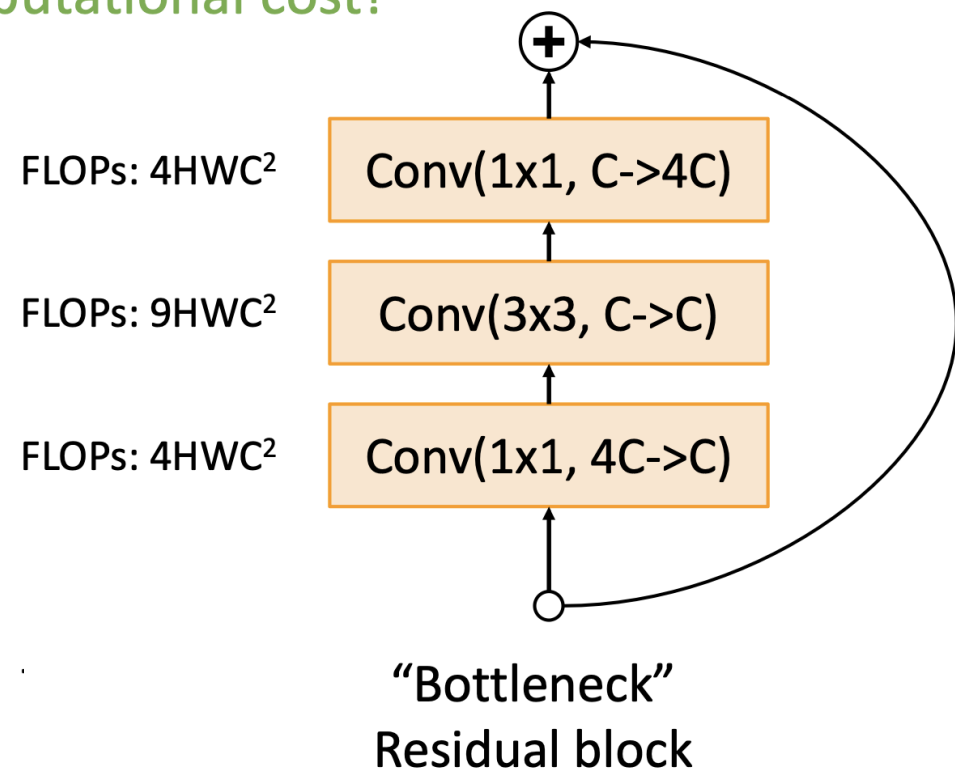
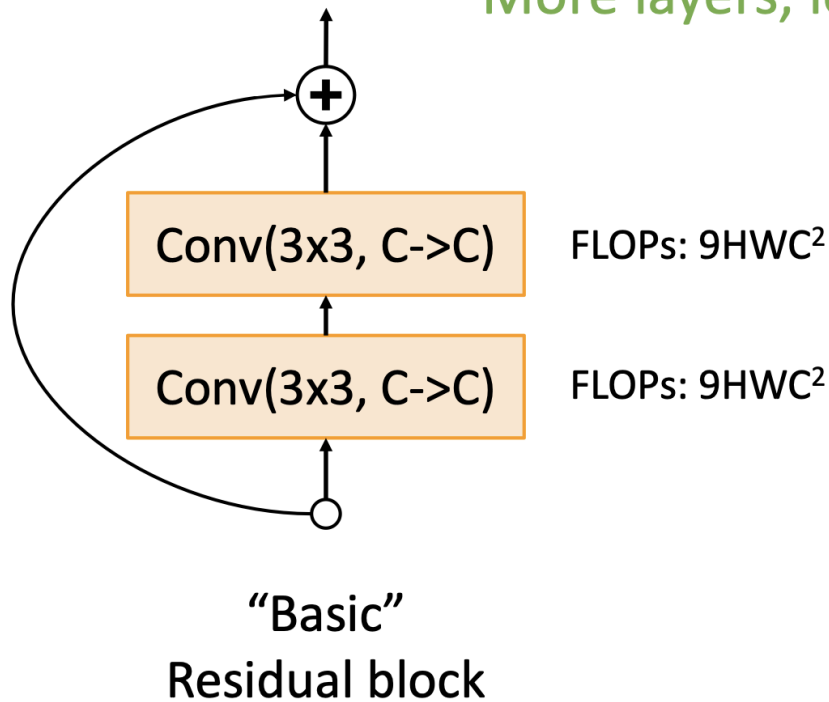
VGG-16:

ImageNet top-5 error: 9.62



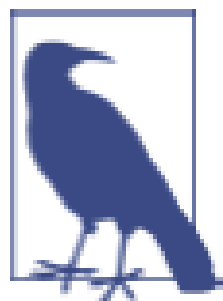
Residual Networks: Bottleneck Block

More layers, less computational cost!



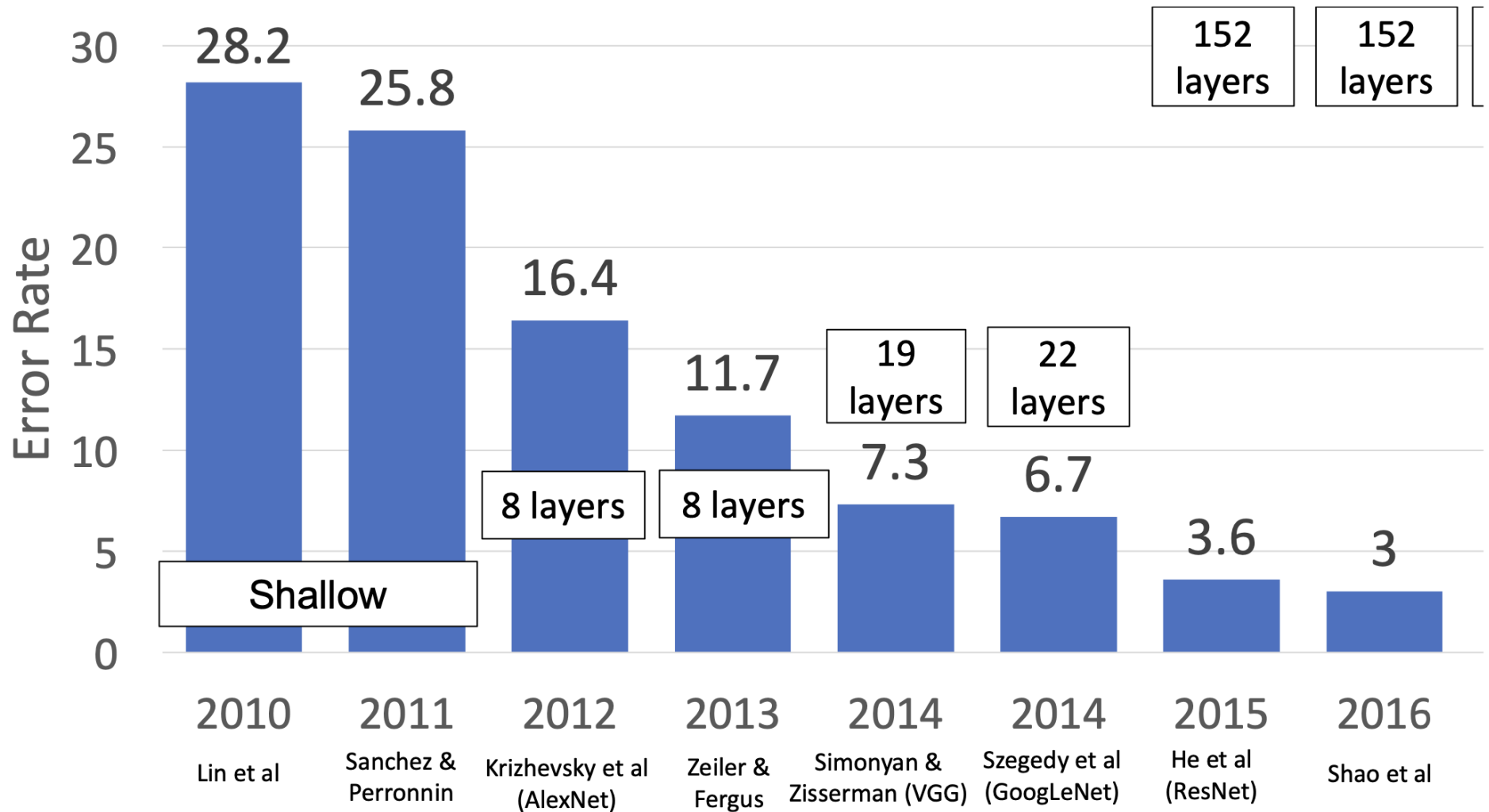
ResNet - 2015 winner

ResNets deeper than that, such as ResNet-152, use slightly different residual units. Instead of two 3×3 convolutional layers with (say) 256 feature maps, they use three convolutional layers: first a 1×1 convolutional layer with just 64 feature maps (4 times less), which acts as a bottleneck layer (as discussed already), then a 3×3 layer with 64 feature maps, and finally another 1×1 convolutional layer with 256 feature maps (4 times 64) that restores the original depth. ResNet-152 contains three such RUs that output 256 maps, then 8 RUs with 512 maps, a whopping 36 RUs with 1,024 maps, and finally 3 RUs with 2,048 maps.

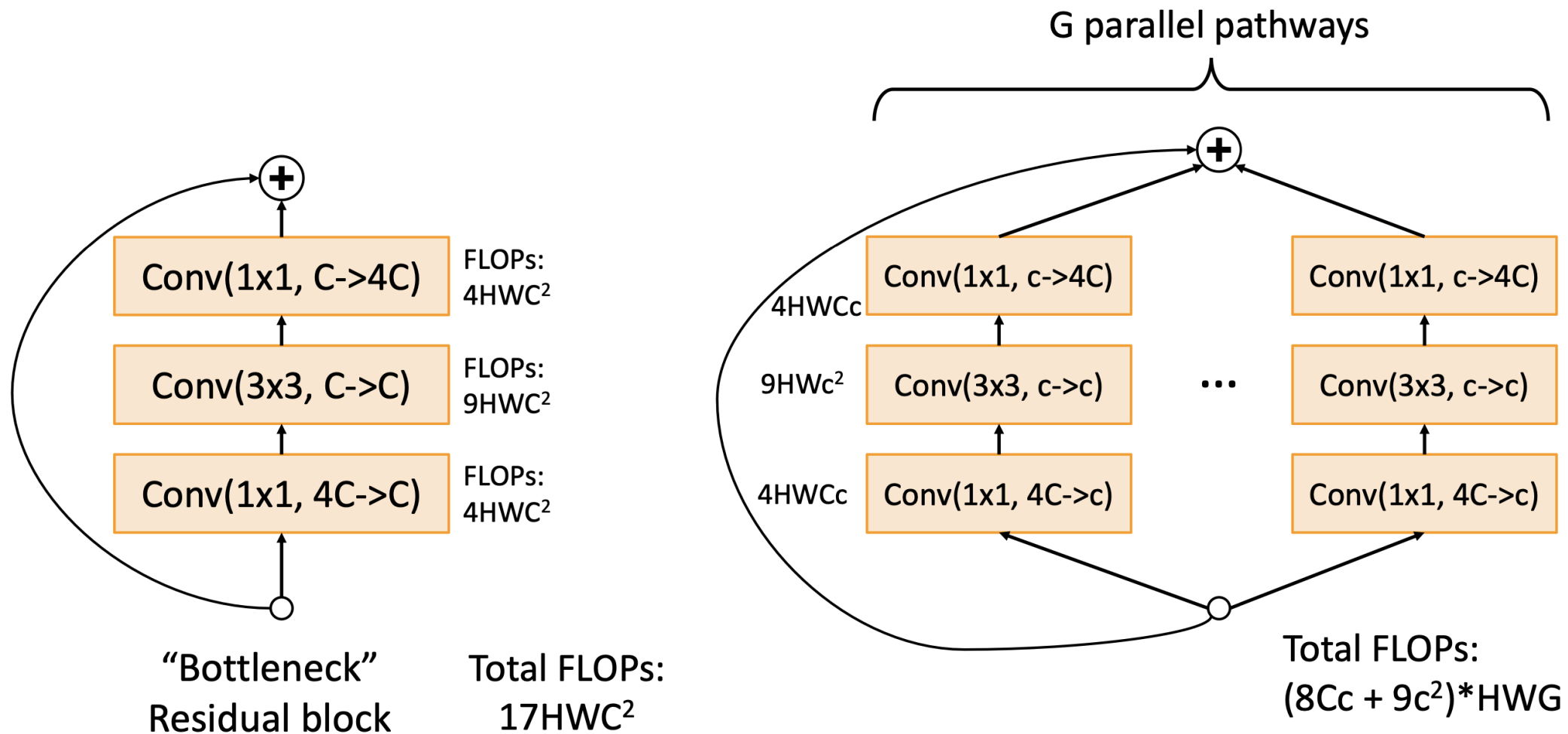


Google's **Inception-v4**¹⁶ architecture merged the ideas of GoogLeNet and ResNet and achieved close to 3% top-5 error rate on ImageNet classification.

ImageNet Classification Challenge



Improving ResNets: ResNeXt



Xception – F. Chollet, Depthwise Separable Convolutions:

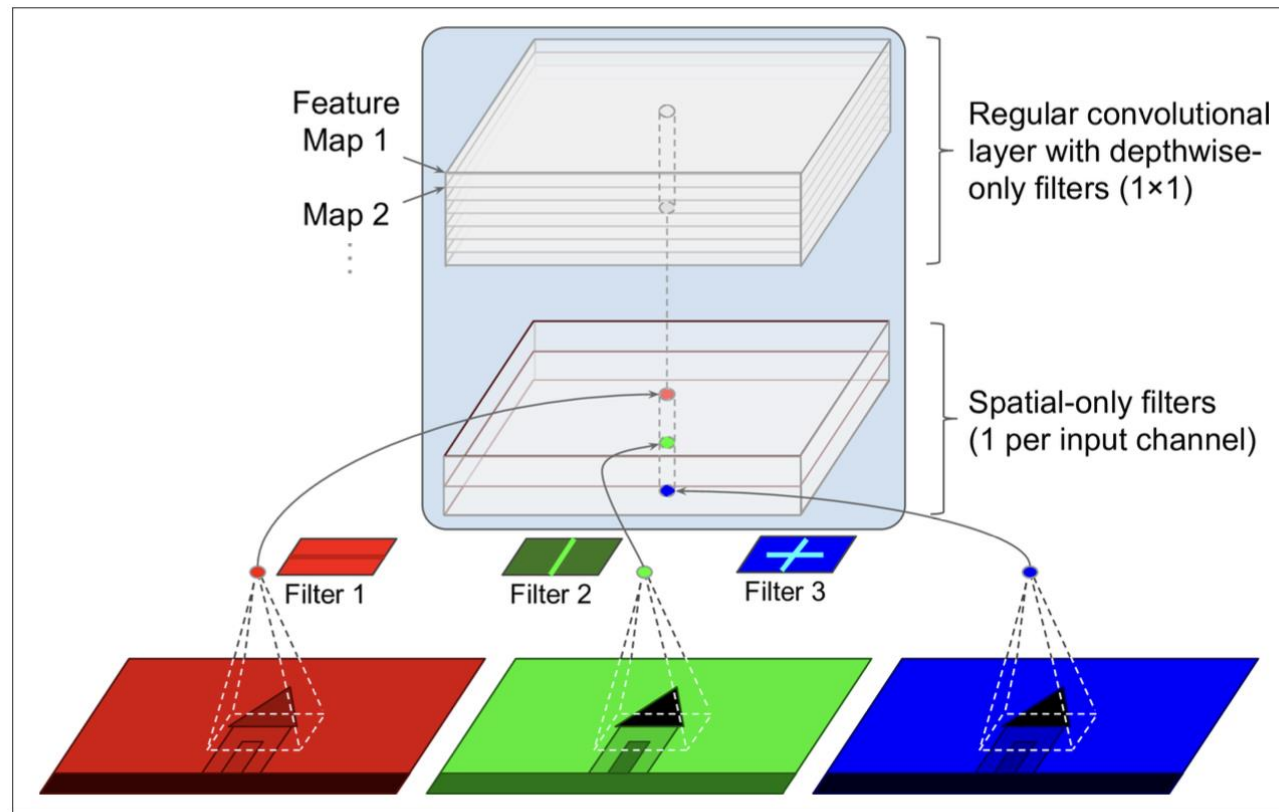


Figure 14-19. Depthwise Separable Convolutional Layer

- The primary innovation in Xception is the use of depthwise separable convolutions.
- A standard convolutional layer combines spatial and channel-wise operations into a single step. In contrast, depthwise separable convolutions break this into two steps:
 - **Depthwise Convolution:** Applies a single convolutional filter per input channel.
 - **Pointwise Convolution:** Uses a 1x1 convolution to combine the outputs of the depthwise convolution across channels.
- This separation reduces the number of parameters and computations, making the network more efficient without compromising performance.

Xception - F. Chollet

- Google's huge vision task: 350 million images and 17,000 classes (JFT dataset)
- Xception outperforms inception-v3 on this task

Architecture:

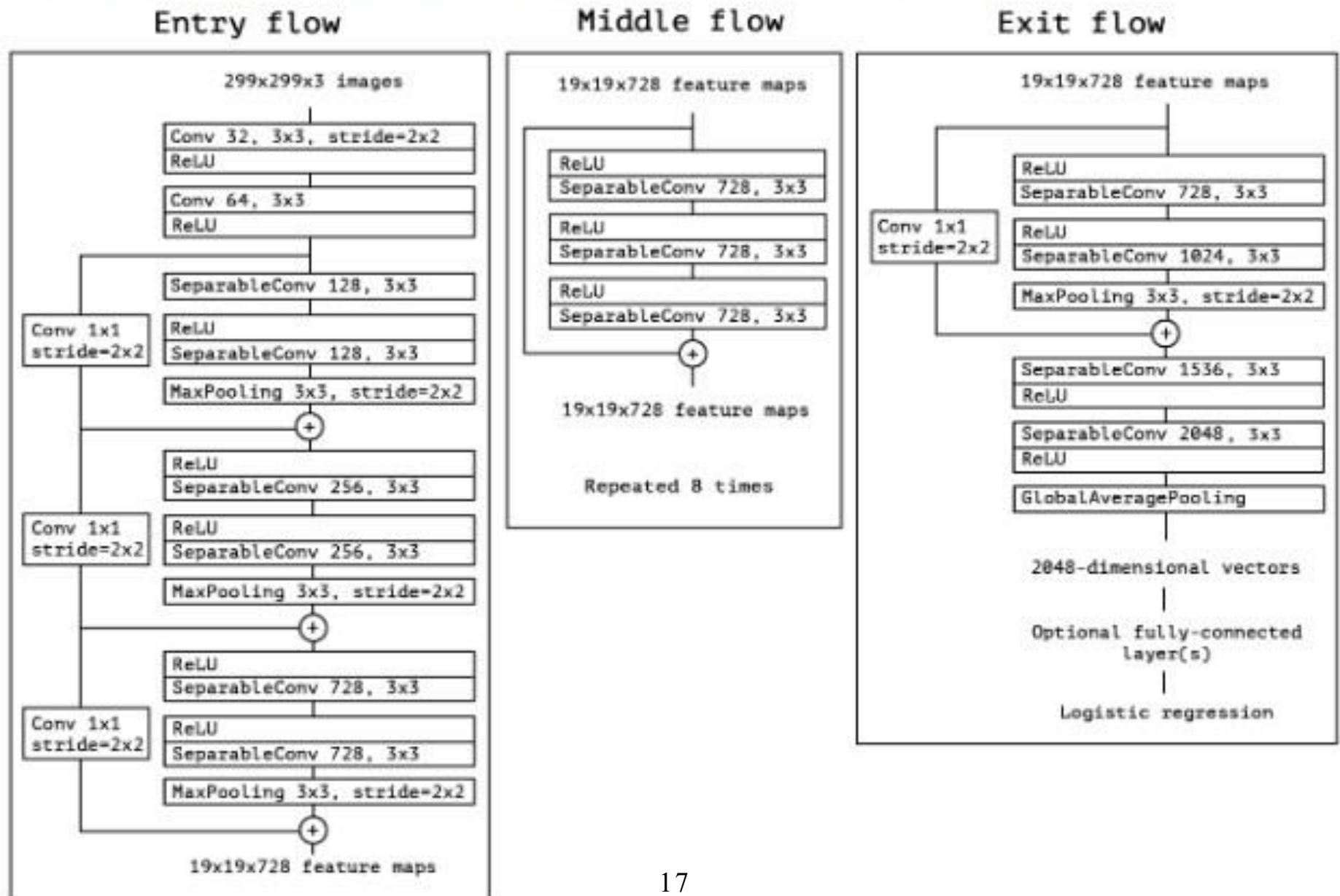
- The Xception architecture replaces the standard Inception modules with depthwise separable convolutions.
- It consists of a series of convolutional layers followed by depthwise separable convolution layers and pointwise convolution layers.
- The architecture ends with fully connected layers for classification.

Advantages:

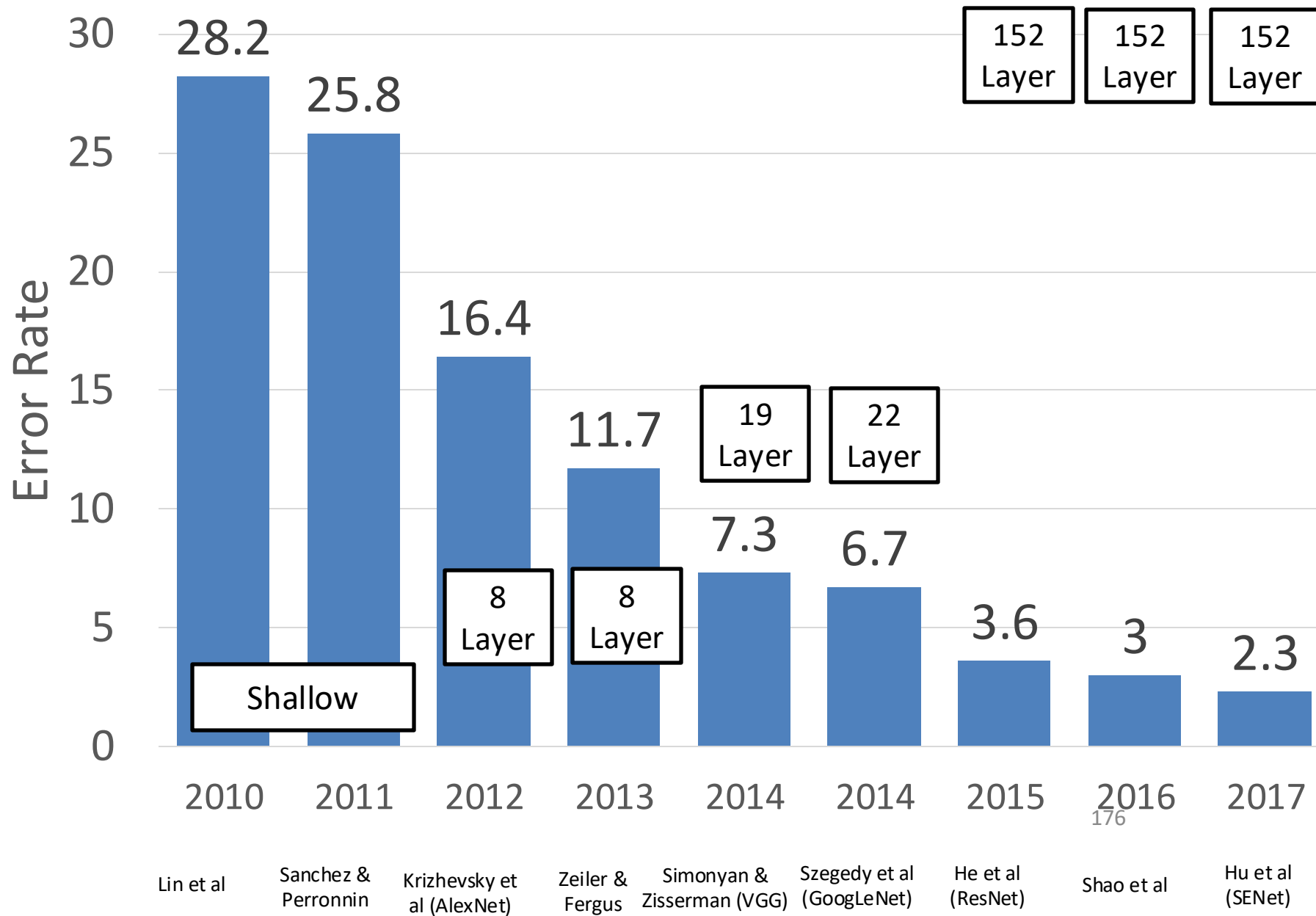
- Efficiency:** Fewer parameters and lower computational cost compared to traditional convolutions.
- Performance:** Achieves state-of-the-art results on several image classification benchmarks.
- Flexibility:** The model can be easily adapted and extended for different tasks.

Xception - F. Chollet

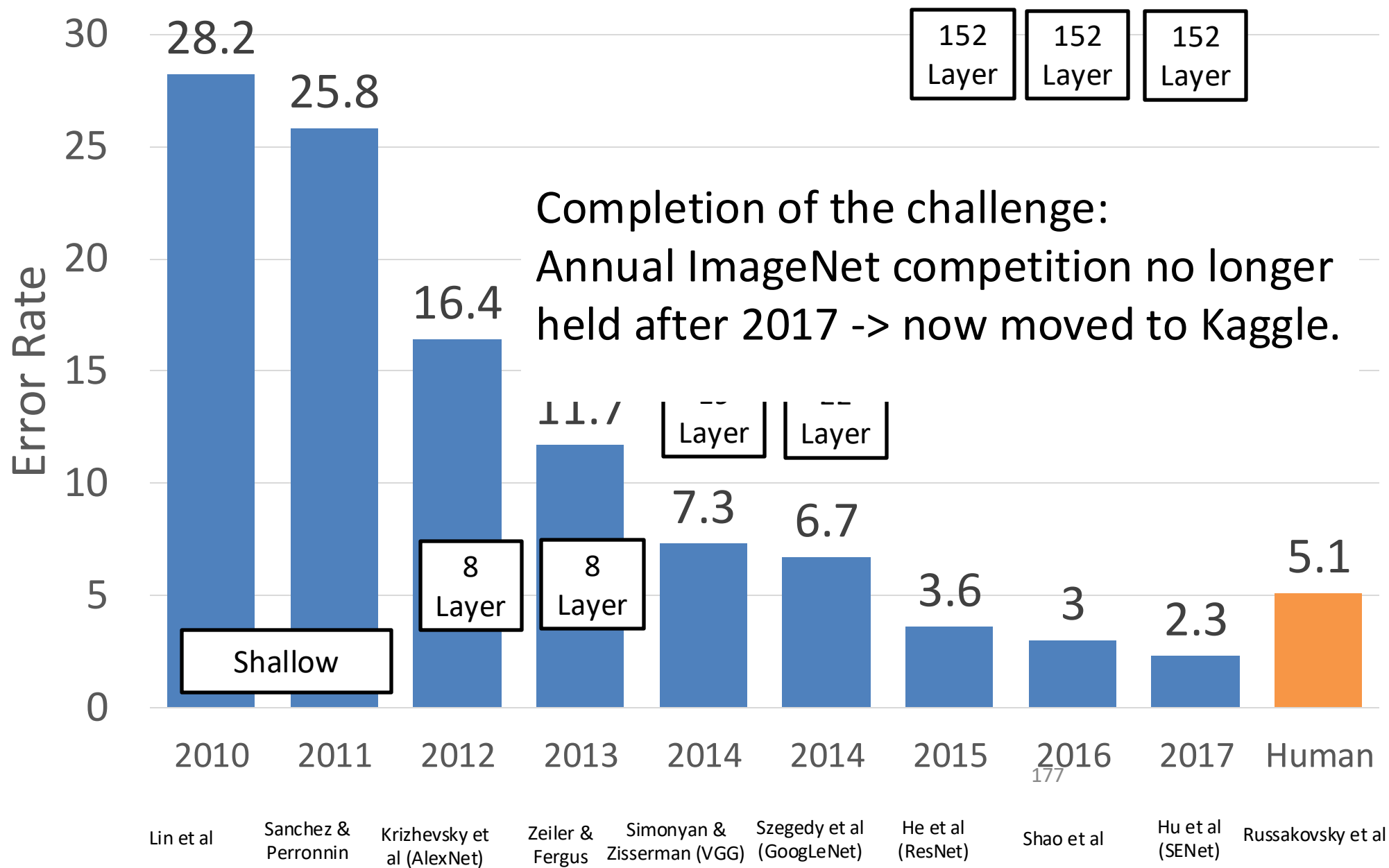
Figure 5. The Xception architecture: the data first goes through the entry flow, then through the middle flow which is repeated eight times, and finally through the exit flow. Note that all Convolution and SeparableConvolution layers are followed by batch normalization [7] (not included in the diagram). All SeparableConvolution layers use a depth multiplier of 1 (no depth expansion).



ImageNet Classification Challenge



ImageNet Classification Challenge

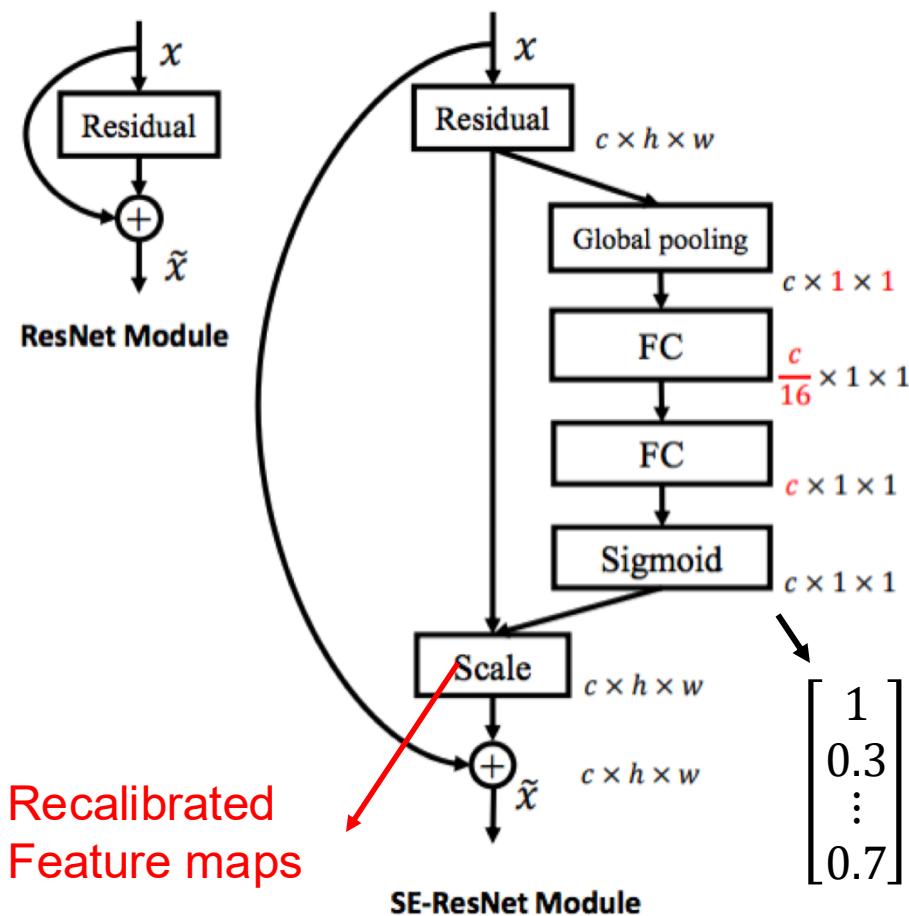


Squeeze-and-Excitation Networks, SENet

Adds a "Squeeze-and-excite" branch to each residual block that performs global pooling, full-connected layers, and multiplies back onto feature map

Adds **global context** to each residual block!

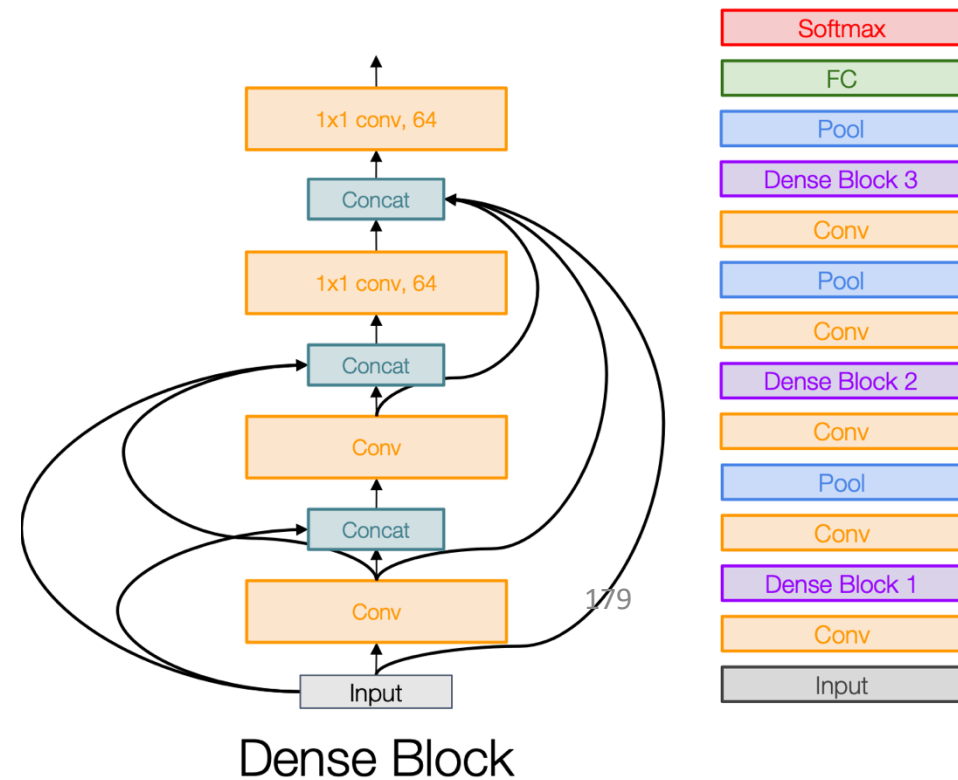
Won ILSVRC 2017 with ResNeXt-152-SE



Densely Connected Neural Networks

Dense blocks where each layer is connected to every other layer in feedforward fashion

Alleviates vanishing gradient, strengthens feature propagation, encourages feature reuse



MobileNets: Tiny Networks (For Mobile Devices)

Similar to Xception, MobileNet also uses depthwise separable convolutions to reduce the computational cost and number of parameters.

Also related:

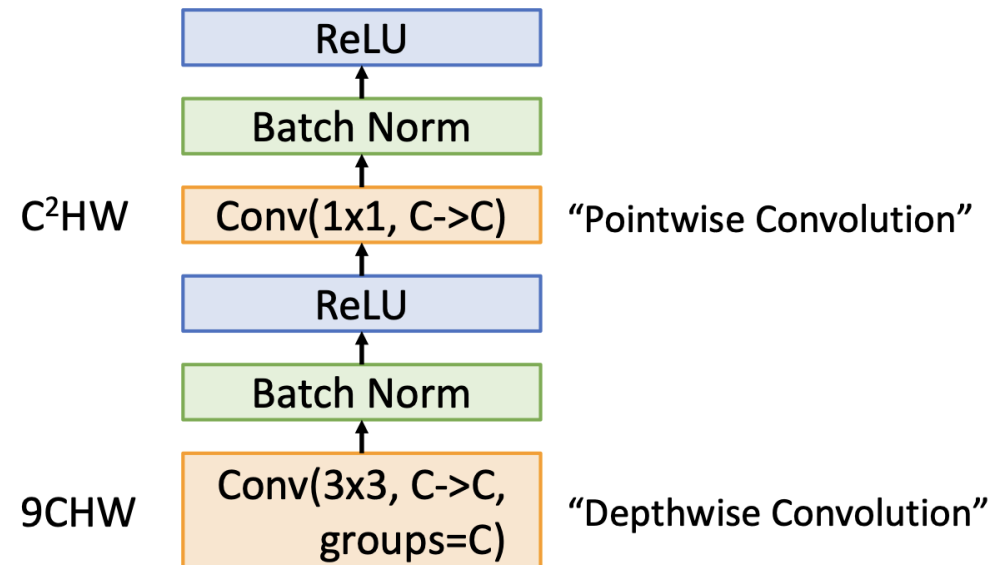
ShuffleNet: Zhang et al, CVPR 2018

MobileNetV2: Sandler et al, CVPR 2018

ShuffleNetV2: Ma et al, ECCV 2018

Depthwise Separable Convolution

Total cost: $(9C + C^2)HW$



Which Architecture should I use?

Don't be a hero. For most problems you should use an off-the-shelf architecture; don't try to design your own!

If you just care about accuracy, **ResNet-50** or **ResNet-101** are great choices

If you want an efficient network (real-time, run on mobile, etc) try **MobileNets** and **ShuffleNets**

CNN Architectures Summary

Early work (AlexNet -> ZFNet -> VGG) shows that **bigger networks work better**

GoogLeNet one of the first to focus on **efficiency** (aggressive stem, 1x1 bottleneck convolutions, global avg pool instead of FC layers)

ResNet showed us how to train extremely deep networks – limited only by GPU memory! Started to show diminishing returns as networks got bigger

After ResNet: **Efficient networks** became central: how can we improve the accuracy without increasing the complexity?

Lots of **tiny networks** aimed at mobile devices: MobileNet, ShuffleNet, etc

Neural Architecture Search promises to automate architecture design

TF Documentation References - Data API and Preprocessing

- loading a pandas dataframe into the data API

https://www.tensorflow.org/tutorials/load_data/pandas_dataframe

- about keras preprocessing layers

https://www.tensorflow.org/guide/keras/preprocessing_layers

- transferring feature columns (TF1 artifact) to keras preprocessing layers

https://www.tensorflow.org/guide/migrate/migrating_feature_columns

- build tensorflow input pipelines

<https://www.tensorflow.org/guide/data?hl=en>

- keras util: "FeatureSpace" supposed to be one stop utility for preprocessing and encoding structured data

https://www.tensorflow.org/api_docs/python/tf/keras/utils/FeatureSpace?hl=en

Next:
Transfer
Learning